

Mid-term Exam (Graph Neural Networks - Fall 2026)

해설지 ENG + KOR

Full Name:

Student ID:

Note: Students should write in English for evaluation.

1회차

ENG)

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1$, $P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 노드 1과 5에는 이름표가 주어져 있으며($P(Y_1)=1$, $P(Y_5)=0$), 나머지 노드는 이름표가 없고 0.5 로 초기화한다.

관계형 분류기는 이름표가 없는 각 노드를 이웃 노드들의 부류 확률에 대한 가중 평균으로 갱신한다. 분모는 해당 노드에 연결된 간선 가중치의 합이다.

- 관계형 분류가 전제하는 핵심 가정과 그 한계 하나를 서술하시오.
- 1차 반복 후 이름표가 없는 각 노드의 $P(Y_v)$ 를 계산하시오.
- 2차 반복 후의 $P(Y_v)$ 를 계산하시오. 단, 직전 반복의 값만 사용한다.

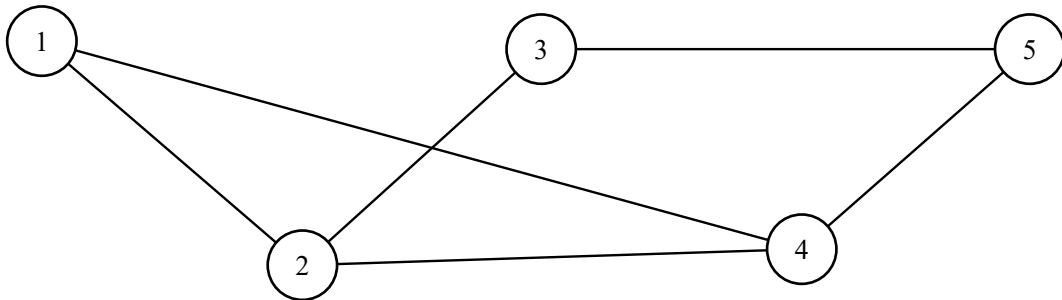


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.

KOR)

a) 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.

- 관계형 분류는 동질성을 가정해요. 이어진 노드끼리는 같은 이름표를 가지는 경향이 있다는 뜻이에요.

- 이름표 없는 노드를 이웃들의 확률의 가중 평균으로 거듭 바꿔요.

- 한계는 그래프 구조만 쓰고 노드의 특징은 무시한다는 것이에요.

ENG) Write the initial values: node 1 is 1, node 5 is 0, the rest start at 0.5.

KOR) 처음 값을 적는다. 이름표가 붙은 1번은 1, 5번은 0, 나머지는 0.5 로 시작한다.

ENG) First iteration. Only the unlabelled nodes are replaced by the mean of their neighbours.

KOR) 1차 반복. 이름표가 없는 노드만 이웃의 평균으로 바꾼다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

ENG) Node 3: neighbours 2, 5. $(0.5 + 0)/2 = 0.25$

KOR) 3번: 이웃 2, 5. $(0.5 + 0)/2 = 0.25$

ENG) Node 4: neighbours 1, 2, 5. $(1 + 0.5 + 0)/3 = 0.5$

KOR) 4번: 이웃 1, 2, 5. $(1 + 0.5 + 0)/3 = 0.5$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) b) $P(Y1) = 1$, $P(Y2) = 0.6667$, $P(Y3) = 0.25$, $P(Y4) = 0.5$, $P(Y5) = 0$

ENG) Second iteration. Use only the values from the first iteration, not the ones just computed.

KOR) 2차 반복. 방금 구한 값이 아니라 1차 반복의 값만 쓴다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.25 + 0.5)/3 = 0.5833$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.25 + 0.5)/3 = 0.5833$

ENG) Node 3: neighbours 2, 5. $(0.6667 + 0)/2 = 0.3333$

KOR) 3번: 이웃 2, 5. $(0.6667 + 0)/2 = 0.3333$

ENG) Node 4: neighbours 1, 2, 5. $(1 + 0.6667 + 0)/3 = 0.5556$

KOR) 4번: 이웃 1, 2, 5. $(1 + 0.6667 + 0)/3 = 0.5556$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) c) $P(Y1) = 1$, $P(Y2) = 0.5833$, $P(Y3) = 0.3333$, $P(Y4) = 0.5556$, $P(Y5) = 0$

ENG)

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{C, D\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

KOR)

인접행렬 A 와 노드 특징 벡터 $h^{(0)}$ 가 주어진 무방향 그래프 G 를 고려한다.

FastGCN 은 GCN 한 층의 전체 이웃 합을 층 단위 몬테카를로 추정으로 대체한다. 각 층에서 t_l 개의 노드를 중요도 분포 $q(u) \propto \|A(:, u)\|^2$ 로부터 한 번 추출하며, 평균은 이웃 수가 아니라 추출 개수 t_l 로 나눈다.

- 층 단위 추출이 GraphSAGE 의 노드 단위 추출과 어떻게 다른지 서술하시오.
- 다섯 노드에 대한 중요도 분포 $q(u)$ 를 계산하시오.
- 주어진 추출 집합($t_l = 2$)에 대하여 $h^{(1)}(A)$ 의 추정값을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ -2 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 뽑기는 노드마다 제 이웃을 뽑아서, 층이 깊어질수록 비용이 지수로 커져요.
 - FastGCN 은 층마다 정해진 개수를 한 번만 뽑고, 그 층의 모든 노드가 같이 써요.
 - 그래서 비용이 지수가 아니라 층 수에 비례해서만 늘어요.

ENG) Degrees: $d(A)=4, d(B)=2, d(C)=2, d(D)=3, d(E)=1$

KOR) 차수를 센다. $d(A)=4, d(B)=2, d(C)=2, d(D)=3, d(E)=1$

ENG) Divide each row by its degree. $\text{Atilde} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0.3333, 0, 0]; [1, 0, 0, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0.3333, 0, 0]; [1, 0, 0, 0, 0]$ (가로줄 합이 모두 1)

ENG) $q(u)$ uses the squared norm of column u of $\tilde{A}$, not of a row.

KOR) $q(u)$ 는 $\tilde{A}$ 의 **세로줄** 제곱합이다. 가로줄이 아니다.

ENG) Column A: squared sum = 1.6111

KOR) A 칸의 제곱합 = 1.6111

ENG) Column B: squared sum = 0.1736

KOR) B 칸의 제곱합 = 0.1736

ENG) Column C: squared sum = 0.1736

KOR) C 칸의 제곱합 = 0.1736

ENG) Column D: squared sum = 0.5625

KOR) D 칸의 제곱합 = 0.5625

ENG) Column E: squared sum = 0.0625

KOR) E 칸의 제곱합 = 0.0625

ENG) Divide by the total 2.5833 to turn them into probabilities.

KOR) 전체 합 2.5833 로 나눠 확률로 만든다.

ENG) b) $q(A) = 0.6237, q(B) = 0.0672, q(C) = 0.0672, q(D) = 0.2177, q(E) = 0.0242$

ENG) For (c) look only at the sampled nodes: read $\tilde{A}(A, u)$ for each.

KOR) c) 는 뽑힌 노드만 본다. A 가로줄에서 그 칸의 $\tilde{A}$ 값을 읽는다.

ENG) $\tilde{A}(A, C) h(C) = 0.25 \times 0 = 0$

KOR) $\tilde{A}(A, C) \times h(C) = 0.25 \times 0 = 0$

ENG) $\tilde{A}(A, D) h(D) = 0.25 \times -2 = -0.5$

KOR) $\tilde{A}(A, D) \times h(D) = 0.25 \times -2 = -0.5$

ENG) Add them and divide by the number sampled (2), not by the degree: $(0 + -0.5)/2 = -0.25$

KOR) 더한 뒤 이웃 수가 아니라 **뽑은 개수 2** 로 나눈다. $(0 + -0.5)/2 = -0.25$

ENG) ReLU clips the negatives: $-0.25 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.25 \rightarrow 0$

ENG) c) $h_1(A) = 0$

ENG)

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = E$.
- Calculate $E_{p(u_j | E)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(E)$.

KOR)

문제 11 과 동일한 그래프와 특징 벡터를 고려한다.

ASGCN 은 GCN 한 층을 확률 분포에 대한 기댓값으로 다시 쓴다. 여기서 $N(v_i) = \sum_j \tilde{A}_{ij}$ 는 정규화 계수이고, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$ 는 부모 노드 v_i 가 주어졌을 때 노드 u_j 를 추출할 확률이다.

- ASGCN 의 제안 분포가 FastGCN 과 달리 무엇에 의존하는지 서술하시오.
- 주어진 부모 노드에 대한 $N(v_i)$ 와 $p(u_j | v_i)$ 를 계산하시오.
- 해당 기댓값과 그 결과로 얻어지는 $h^{(1)}$ 을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ -1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - FastGCN 의 뽑기 확률은 미리 정해져 있고 부모 노드와 상관이 없어요.
 - ASGCN 은 뽑기 확률을 윗층 노드에 맞춰 정해요.
 - 그래서 뽑은 노드가 거의 다 쓸모 있고, 추정값의 흔들림이 줄어요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=1$, $d(D)=3$, $d(E)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=1$, $d(D)=3$, $d(E)=3$

ENG) Divide each row by its degree. Atilde = $[0, 0.5, 0, 0.5, 0]$; $[0.3333, 0, 0, 0.3333, 0.3333]$; $[0, 0, 0, 0, 1]$; $[0.3333, 0.3333, 0, 0, 0.3333]$; $[0, 0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0, 0, 1]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) The parent node is E, so only its row of Atilde matters.

KOR) 부모 노드는 E 다. Atilde 의 그 줄만 본다.

ENG) $N(E) = \text{sum of that row} = 1$. With row normalisation it is 1.

KOR) $N(E) = \text{그 가로줄의 합} = 1$. 행 정규화라 1 이다.

ENG) $p(B|E) = \text{Atilde}(E,B) / N = 0.3333$

KOR) $p(B|E) = \text{Atilde}(E,B) / N = 0.3333 / 1 = 0.3333$

ENG) $p(C|E) = \text{Atilde}(E,C) / N = 0.3333$

KOR) $p(C|E) = \text{Atilde}(E,C) / N = 0.3333 / 1 = 0.3333$

ENG) $p(D|E) = \text{Atilde}(E,D) / N = 0.3333$

KOR) $p(D|E) = \text{Atilde}(E,D) / N = 0.3333 / 1 = 0.3333$

ENG) b) $N(E) = 1$, $p(B|E) = 0.3333$, $p(C|E) = 0.3333$, $p(D|E) = 0.3333$

ENG) The expectation is the sum of probability times value: $0.3333 \times -1 + 0.3333 \times 3 + 0.3333 \times 2 = 1.3333$

KOR) 기댓값은 확률 곱하기 값의 합이다. $0.3333 \times -1 + 0.3333 \times 3 + 0.3333 \times 2 = 1.3333$

ENG) $N = 1$ and $W = 1$, so the value is unchanged. It equals the plain neighbour mean.

KOR) $N = 1$, $W = 1$ 이라 곱해도 그대로다. 결국 이웃 평균과 같은 값이다.

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $E = 1.3333$, $h1(E) = 1.3333$

ENG)

4. (APPPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

APPPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

KOR)

인접행렬 A 와 다층 퍼셉트론이 산출한 예측 행렬 H 가 주어졌다.

APPPNP 는 순간이동 확률 α 를 사용하여 예측을 전파한다. 전파 단계 사이에는 비선형 함수를 적용하지 않는다.

- 개인화 PageRank 의 순간이동 기법을 설명하고, 그것이 과평활을 막는 이유를 서술하시오.
- $\tilde{A}$ 와 $Z^{(1)}$ 을 계산하시오.
- $Z^{(2)}$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 퍼뜨리는 걸음마다 확률 알파로 출발 노드로 되돌아가요. 이것이 순간이동 장치예요.
 - 매 걸음 일정 비율이 되돌아오니까 분포가 출발점 근처에 머물러요.
 - 그래서 많이 퍼뜨려도 과평활이 안 생기고, 퍼뜨리는 데에 추가 파라미터도 없어요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$

ENG) Start from $Z_0 = H = [0, 1, 0, 0]$

KOR) $Z_0 = H$ 로 시작한다. $[0, 1, 0, 0]$

ENG) Propagate one step: $\tilde{Z}_0 = [0.5, 0, 0, 0.5]$

KOR) 한 걸음 퍼뜨린다. $\tilde{Z}_0 = [0.5, 0, 0, 0.5]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[0.4, 0, 0, 0.4]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[0.4, 0, 0, 0.4]$

ENG) Add $\alpha H = [0, 0.2, 0, 0]$. This is the teleport term.

KOR) $\alpha H = 0.2 \times [0, 1, 0, 0] = [0, 0.2, 0, 0]$ 를 더한다. 이것이 순간이동이다.

ENG) $Z_1 = [0.4, 0.2, 0, 0.4]$

ENG) One more step: $\tilde{Z}_1 = [0.1, 0.4, 0.4, 0.1]$

KOR) 한 걸음 더. $\tilde{Z}_1 = [0.1, 0.4, 0.4, 0.1]$

ENG) Multiply by $(1-\alpha)$ and add αH **again**. It is added at every step.

KOR) $(1-\alpha)$ 를 곱하고 **αH 를 또** 더한다. 매 걸음 더하는 것이 핵심이다.

ENG) c) $Z_2 = [0.08, 0.52, 0.32, 0.08]$

ENG)

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.3$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

KOR)

인접행렬 A 와 신경망이 산출한 은닉 특징 $H^{(0)}$ 이 주어졌다.

GPR-GNN 은 특징 추출과 위상 전파를 분리한다. $\tilde{A}$ 를 반복 적용하여 $H^{(k)}$ 를 얻고, 학습된 가중치 γ_k 로 가중합한다. 본 문항에서는 $K = 2$ 이며 $\gamma_0, \gamma_1, \gamma_2$ 가 주어진다.

- GPR-GNN 이 APPNP 와 달리 할 수 있는 것은 무엇이며, γ_k 가 음수가 될 수 있다는 점이 이질적 그래프에서 왜 중요한지 서술하시오.
- $H^{(1)}$ 과 $H^{(2)}$ 를 계산하시오.
- Z 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ 2 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - APPNP 는 계수를 알파 곱하기 (1-알파)의 k제곱으로 못 박아요. 늘 양수이고 줄어들어요.
 - GPR-GNN 은 그 무게를 학습하고, 음수도 될 수 있어요.
 - 음수 무게는 그 걸음을 빼요. 그래서 이질 그래프에서 고주파 필터처럼 동작할 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) H_0 stays as it is: $[4, -2, 2, 2]$

KOR) H_0 은 그대로 둔다. $[4, -2, 2, 2]$

ENG) $H_1 = \text{Atilde } H_0 = [0, 3, 3, 0]$

ENG) $H_2 = \text{Atilde } H_1 = [3, 0, 0, 3]$

ENG) b) $H_1 = [0, 3, 3, 0]$, $H_2 = [3, 0, 0, 3]$

ENG) Now take the weighted sum with gamma. There is no ReLU during propagation.

KOR) 이제 gamma 로 가중합한다. 전파 중에 ReLU 는 없다.

ENG) $\text{gamma}_0 H_0 = 0.7 \times [4, -2, 2, 2] = [2.8, -1.4, 1.4, 1.4]$

KOR) $\text{gamma}_0 \times H_0 = 0.7 \times [4, -2, 2, 2] = [2.8, -1.4, 1.4, 1.4]$

ENG) $\text{gamma}_1 H_1 = 0.3 \times [0, 3, 3, 0] = [0, 0.9, 0.9, 0]$

KOR) $\text{gamma}_1 \times H_1 = 0.3 \times [0, 3, 3, 0] = [0, 0.9, 0.9, 0]$

ENG) $\text{gamma}_2 H_2 = -0.2 \times [3, 0, 0, 3] = [-0.6, 0, 0, -0.6]$

KOR) $\text{gamma}_2 \times H_2 = -0.2 \times [3, 0, 0, 3] = [-0.6, 0, 0, -0.6]$

ENG) gamma_2 is negative, so that hop is subtracted. Dropping the sign gives a wrong answer.

KOR) gamma_2 가 음수라 그 걸음은 빼는 효과다. 부호를 빼뜨리면 틀린다.

ENG) Add the three vectors element-wise.

KOR) 세 벡터를 자리별로 더한다.

ENG) c) $Z = [2.2, -0.5, 2.3, 0.8]$

ENG)

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

MixHop 한 층은 인접행렬의 여러 거듭제곱 $\hat{A}^j$ ($j \in P = \{0, 1, 2\}$)를 각각 계산한 뒤 열 방향으로 이어 붙인다. 본 문항에서 가중치 행렬은 모두 1로 둔다.

- 두 걸음 델타 연산자에 관한 MixHop의 정리 1을 서술하고, 보통의 GCN이 층을 무한히 쌓아도 이를 표현할 수 없는 이유를 설명하시오.
- $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$, $\hat{A}^2 H^{(0)}$ 을 계산하시오.
- $H^{(1)}$ 과 델타 연산자 $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -3 \\ 3 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 정리 1은 보통 GCN이 두 걸음 델타 연산자를 표현할 수 없다고 말해요. 층을 무한히 쌓아도요.
 - GCN 층을 쌓으면 A 에 1제곱 곱하기 W 꼴로 뭉쳐져서, 중간 걸음들이 따로 남지 않아요.
 - MixHop은 거듭제곱마다 따로 구해서 나란히 붙여요. 그래서 뒤 층이 하나에서 다른 하나를 뺄 수 있어요.

ENG) Degrees: $d(A)=1$, $d(B)=3$, $d(C)=2$, $d(D)=2$

KOR) 차수를 센다. $d(A)=1$, $d(B)=3$, $d(C)=2$, $d(D)=2$

ENG) Divide each row by its degree. $\text{Atilde} = [0, 1, 0, 0]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0, 0.5, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 1, 0, 0]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) A^0 is the identity, so it is just $H0: [1, -3, 3, -1]$

KOR) $A0$ 은 단위행렬이라 $H0$ 그대로다. $[1, -3, 3, -1]$

ENG) $A^1 H0 = [-3, 1, -2, 0]$ (one hop, the neighbour mean)

KOR) $A1 H0 = [-3, 1, -2, 0]$ (한 걸음, 이웃 평균)

ENG) $A^2 H0 = \text{Atilde}(\text{Atilde } H0) = [1, -1.6667, 0.5, -0.5]$ (two hops)

KOR) $A2 H0 = \text{Atilde}(\text{Atilde } H0) = [1, -1.6667, 0.5, -0.5]$ (두 걸음)

ENG) ReLU is applied to each block separately, not after concatenation.

KOR) 조각마다 따로 ReLU 를 건다. 이어 붙인 뒤가 아니다.

ENG) After ReLU: $[1, 0, 3, 0] \parallel [0, 1, 0, 0] \parallel [1, 0, 0.5, 0]$

KOR) ReLU 뒤: $[1, 0, 3, 0] \parallel [0, 1, 0, 0] \parallel [1, 0, 0.5, 0]$

ENG) The delta operator subtracts the two-hop block from the one-hop block.

KOR) 델타 연산자는 한 걸음에서 두 걸음을 뺀 것이다.

ENG) b) $A0H = [1, -3, 3, -1]$, $A1H = [-3, 1, -2, 0]$, $A2H = [1, -1.6667, 0.5, -0.5]$

ENG) c) $H1 = \text{concatenation of the three blocks: } [1, 0, 3, 0] \parallel [0, 1, 0, 0] \parallel [1, 0, 0.5, 0]$

KOR) c) $H1 = \text{세 조각을 이어 붙여요: } [1, 0, 3, 0] \parallel [0, 1, 0, 0] \parallel [1, 0, 0.5, 0]$

ENG) $\Delta = [-1, 1, -0.5, 0]$

ENG)

7. (PairNorm) After one GCN layer the node features of a four-node graph are PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

KOR)

GCN 한 층을 통과한 뒤의 노드 특징 행렬 X 가 주어졌다. 행은 노드, 열은 특징 차원이다.

PairNorm 은 중심화 단계와 크기 조정 단계로 정규화한다. 중심화는 노드들에 대한 평균을 빼는 것이고, 크기 조정은 중심화된 벡터들의 노름 제곱 평균의 제곱근으로 나누는 것이다. $s = 1, n = 4$ 로 한다.

- 노드 단위 과평활과 특징 단위 과평활을 각각 설명하고, PairNorm 이 층에 걸쳐 일정하게 유지하는 양이 무엇인지 서술하시오.
- 중심화된 특징을 계산하시오.
- 크기 조정의 분모와 최종 x_i 를 계산하시오.

$$X = \begin{bmatrix} 2 & -2 \\ 4 & -1 \\ 4 & 3 \\ 3 & 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 과평활은 이어진 노드들의 특징이 너무 비슷해지는 것이에요.
 - 특징 단위 과평활은 그래프 전체에서 모든 노드 특징이 구별이 안 되게 되는 것이에요.
 - PairNorm 은 층을 지나도 노드 사이 거리 제곱의 합을 일정하게 지켜요.

ENG) First take the mean over the nodes, separately for each feature column.

KOR) 먼저 노드들에 대한 평균을 구한다. 특징 차원마다 따로 구한다.

ENG) Column 1: $(2 + 4 + 4 + 3)/4 = 3.25$

KOR) 첫 칸: $(2 + 4 + 4 + 3)/4 = 3.25$

ENG) Column 2: $(-2 + -1 + 3 + 2)/4 = 0.5$

KOR) 둘째 칸: $(-2 + -1 + 3 + 2)/4 = 0.5$

ENG) Subtract the mean from every row. Check: each column now sums to zero.

KOR) 가로줄마다 평균을 뺀다. 검산: 각 칸의 합이 0 이 된다.

ENG) Node 1: squared norm = 7.8125

KOR) 노드 1 의 길이 제곱 = 7.8125

ENG) Node 2: squared norm = 2.8125

KOR) 노드 2 의 길이 제곱 = 2.8125

ENG) Node 3: squared norm = 6.8125

KOR) 노드 3 의 길이 제곱 = 6.8125

ENG) Node 4: squared norm = 2.3125

KOR) 노드 4 의 길이 제곱 = 2.3125

ENG) Mean = $(7.8125 + 2.8125 + 6.8125 + 2.3125)/4 = 4.9375$

KOR) 평균 = $(7.8125 + 2.8125 + 6.8125 + 2.3125)/4 = 4.9375$

ENG) Take the square root: $\sqrt{4.9375} = 2.222$. Divide every row by this single number.

KOR) 제곱근을 씌운다. $\sqrt{4.9375} = 2.222$. 이 **하나의 수** 로 모든 줄을 나눈다.

ENG) b) Mean = [3.25, 0.5], centered = [-1.25, -2.5]; [0.75, -1.5]; [0.75, 2.5]; [-0.25, 1.5]

KOR) b) 평균 = [3.25, 0.5], 중심을 뺀 것 = [-1.25, -2.5]; [0.75, -1.5]; [0.75, 2.5]; [-0.25, 1.5]

ENG) c) Scaling denominator = 2.222, $x = [-0.5625, -1.1251]; [0.3375, -0.6751]; [0.3375, 1.1251]; [-0.1125, 0.6751]$

KOR) c) 분모 = 2.222, $x = [-0.5625, -1.1251]; [0.3375, -0.6751]; [0.3375, 1.1251]; [-0.1125, 0.6751]$

ENG)

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (B, C) and (A, C) are dropped. Recalculate $H^{(1)}$.

KOR)

노드 A, B, C, D 로 이루어진 완전그래프 K_4 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

DropEdge 는 매 에폭마다 각 무방향 간선을 확률 $1-p$ 로 독립적으로 유지한다. 이때 $A^{(t)}$ 와 $D^{(t)}$ 는 살아남은 간선만으로 다시 구성한다. $p = 1/3$ 으로 한다.

- DropEdge 가 목표로 하는 두 가지 문제를 쓰고, Dropout 과 어떻게 다른지 서술하시오.
- 살아남은 간선 수의 기댓값과, 간선을 제거하지 않은 원래 그래프에서의 $H^{(1)}$ 을 계산하시오.
- 주어진 두 간선이 제거되었을 때의 $H^{(1)}$ 을 다시 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -3 \\ 3 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - DropEdge 는 과적합과 과평활을 노려요.
 - 판마다 다른 부분 그래프를 써요. 그래프에 자료를 늘리는 효과가 나요.
 - Dropout 은 특징 값을 0 으로 꺼요. DropEdge 는 인접행렬의 선을 지워요.

ENG) K_4 has 6 edges. Each survives with probability $1 - 1/3 = 2/3$, so the expectation is 4.

KOR) K_4 는 간선이 6 개다. 남을 확률이 $1 - 1/3 = 2/3$ 이므로 기댓값은 $6 \times 2/3 = 4$ 개다.

ENG) First the full graph: every node has degree 3.

KOR) 먼저 원래 그래프. 모든 노드의 차수가 3 다.

ENG) A: neighbours B, C, D. $(-3 + 3 + 2)/3 = 0.6667$

KOR) A: 이웃 B, C, D. $(-3 + 3 + 2)/3 = 0.6667$

ENG) B: neighbours A, C, D. $(3 + 3 + 2)/3 = 2.6667$

KOR) B: 이웃 A, C, D. $(3 + 3 + 2)/3 = 2.6667$

ENG) C: neighbours A, B, D. $(3 + -3 + 2)/3 = 0.6667$

KOR) C: 이웃 A, B, D. $(3 + -3 + 2)/3 = 0.6667$

ENG) D: neighbours A, B, C. $(3 + -3 + 3)/3 = 1$

KOR) D: 이웃 A, B, C. $(3 + -3 + 3)/3 = 1$

ENG) Now drop the two given edges and **recount the degrees**. This is where most mistakes happen.

KOR) 이제 주어진 두 간선을 지운다. **차수를 다시 센다**. 여기서 거의 다 틀린다.

ENG) Degrees after dropping: $d(A)=2$, $d(B)=2$, $d(C)=1$, $d(D)=3$

KOR) 지운 뒤 차수: $d(A)=2$, $d(B)=2$, $d(C)=1$, $d(D)=3$

ENG) A: neighbours B, D. $(-3 + 2)/2 = -0.5$

KOR) A: 이웃 B, D. $(-3 + 2)/2 = -0.5$

ENG) B: neighbours A, D. $(3 + 2)/2 = 2.5$

KOR) B: 이웃 A, D. $(3 + 2)/2 = 2.5$

ENG) C: neighbours D. $(2)/1 = 2$

KOR) C: 이웃 D. $(2)/1 = 2$

ENG) D: neighbours A, B, C. $(3 + -3 + 3)/3 = 1$

KOR) D: 이웃 A, B, C. $(3 + -3 + 3)/3 = 1$

ENG) b) Expected number of surviving edges = 4, $H1 = [0.6667, 2.6667, 0.6667, 1]$

KOR) b) 남는 선 수의 기댓값 = 4, $H1 = [0.6667, 2.6667, 0.6667, 1]$

ENG) c) $H1 = [0, 2.5, 2, 1]$

ENG)

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

KOR)

다음 그림의 무방향 그래프 G 를 고려한다. 임의보행이 노드 A 에서 노드 B 로 이동하여 현재 B 에 있으며, $p = 2, q = 0.5$ 이다.

B 에서 다음 노드 x 로의 비정규화 전이확률은 $\pi_{Bx} = \alpha_{pq}(A, x)$ 이며, α 는 직전 노드 A 로부터의 최단경로 거리에 따라 결정된다.

- BFS 와 DFS 를 설명하고, q 가 작을 때 보행이 둘 중 어느 쪽처럼 동작하는지 서술하시오.
- 노드 B 에서 이동 가능한 모든 노드를 열거하고 비정규화 전이확률 π_{Bx} 를 계산하시오.
- 정규화된 전이확률을 계산하시오.

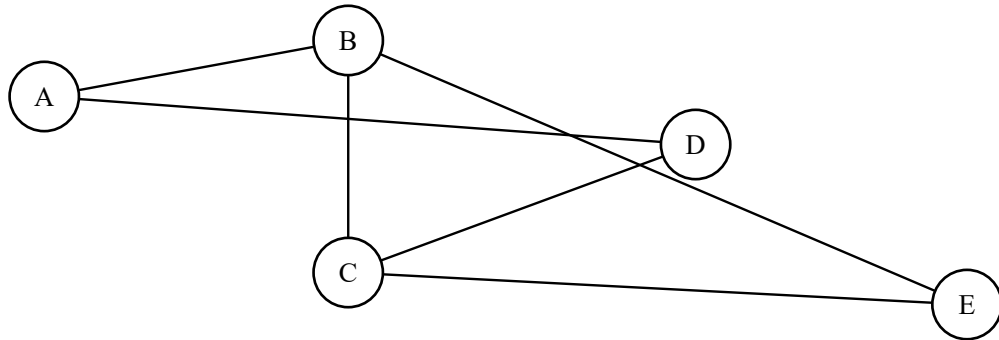


Figure 1. Graph G

풀이

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - BFS 는 바로 옆 이웃부터 훑어요. 그래서 임베딩이 노드의 구조적 역할을 담아요.
 - DFS 는 출발점에서 멀리 가요. 그래서 임베딩이 어느 동네 소속인지를 담아요.
 - q 가 작으면 $1/q$ 가 커져서 걸음이 바깥으로 나가요. DFS 처럼 움직여요.

ENG) The previous node is A and the walk is now at B. All distances are measured from A.

KOR) 직전 노드는 A, 지금 노드는 B 다. 거리는 모두 A 로부터 잴다.

ENG) A: $x = A$ is the previous node, $d = 0$, $\alpha = 1/p = 0.5$

KOR) A: $x = A$ 는 직전 노드라 $d = 0$. $\alpha = 1/p = 0.5$

ENG) C: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) C: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) E: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) E: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) Sum them: $0.5 + 2 + 2 = 4.5$

KOR) 세 값을 더한다. $0.5 + 2 + 2 = 4.5$

ENG) b) $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow E) = 2$

ENG) c) $P(A) = 0.1111$, $P(C) = 0.4444$, $P(E) = 0.4444$

ENG)

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = 1, u'_B = 0.5, u'_C = -1, u'_D = -0.5.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- What is second-order proximity, and how does it differ from first-order proximity?
- Calculate $p_2(A|C)$, $p_2(B|C)$, $p_2(C|C)$ and $p_2(D|C)$.

KOR)

노드 임베딩 u 와 맥락 임베딩 u' 가 주어진 무방향 그래프 G 를 고려한다.

2차 근접도는 $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$ 로 정의한다. 지수 함수 값은 문제에 주어진 표를 사용한다.

- 2차 근접도란 무엇이며 1차 근접도와 어떻게 다른지 서술하시오.
- 주어진 기준 노드에 대하여 네 노드의 p_2 를 계산하시오.

풀이

ENG)

- Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 1차 근접도는 선으로 직접 이어진 두 노드 사이의 비슷함이에요.
 - 2차 근접도는 이웃을 많이 공유하는 두 노드의 비슷함이에요. 선이 없어도 돼요.
 - 그래서 LINE 은 노드마다 벡터를 두 개 뒹요. 자기 벡터와 맥락 벡터예요.

ENG) The base node is C with $u(C) = -0.5$. The other side uses the context vector u' .

KOR) 기준 노드는 C 이고 $u(C) = -0.5$ 다. 상대는 맥락 벡터 u' 를 쓴다.

ENG) A: $u'(A) u(C) = 1 \times -0.5 = -0.5$, $\exp = 0.61$

KOR) A: $u'(A) \times u(C) = 1 \times -0.5 = -0.5$, $\exp = 0.61$

ENG) B: $u'(B) u(C) = 0.5 \times -0.5 = -0.25$, $\exp = 0.78$

KOR) B: $u'(B) \times u(C) = 0.5 \times -0.5 = -0.25$, $\exp = 0.78$

ENG) C: $u'(C) u(C) = -1 \times -0.5 = 0.5$, $\exp = 1.65$

KOR) C: $u'(C) \times u(C) = -1 \times -0.5 = 0.5$, $\exp = 1.65$

ENG) D: $u'(D) u(C) = -0.5 \times -0.5 = 0.25$, $\exp = 1.28$

KOR) D: $u'(D) \times u(C) = -0.5 \times -0.5 = 0.25$, $\exp = 1.28$

ENG) The denominator is the sum of all four: $0.61 + 0.78 + 1.65 + 1.28 = 4.32$

KOR) 분모는 넷을 모두 더한 값이다. $0.61 + 0.78 + 1.65 + 1.28 = 4.32$

ENG) b) $p_2(A|C) = 0.1412$, $p_2(B|C) = 0.1806$, $p_2(C|C) = 0.3819$, $p_2(D|C) = 0.2963$

ENG)

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCN 모형의 층 (k) 에서의 은닉층은 $H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)})$ 로 계산된다고 가정한다. 여기서 $\tilde{A} = D^{-1}A$ 는 정규화된 인접행렬이고 σ 는 ReLU 이다.

- GCN 한 층에서 정보가 어떻게 전파되는지 설명하시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서 GCN 모형의 출력을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 0 \\ 2 \\ 3 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 한 층에서 노드는 바로 옆 이웃의 특징을 모아 평균을 내요.
 - 행 정규화 행렬 $\tilde{A}$ 데는 D 의 역행렬 곱하기 A 이고, 그것이 평균을 내 줘요.
 - k 개 층을 지나면 노드는 k 걸음 안의 모든 노드 정보를 받아요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=4, d(D)=3, d(E)=1$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=4, d(D)=3, d(E)=1$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0.5, 0]; [0.25, 0.25, 0, 0.25, 0.25]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 0, 1, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0.5, 0]; [0.25, 0.25, 0, 0.25, 0.25]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 0, 1, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0.5, 0]; [0.25, 0.25, 0, 0.25, 0.25]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 0, 1, 0, 0]$

ENG) Multiply $\tilde{A}$ by H_0 . Each entry is the mean of that node's neighbours.

KOR) $\tilde{A}$ 를 H_0 에 곱한다. 노드마다 이웃의 평균이다.

ENG) A: neighbours C, D. $(2 + 3)/2 = 2.5$

KOR) A: 이웃 C, D. $(2 + 3)/2 = 2.5$

ENG) B: neighbours C, D. $(2 + 3)/2 = 2.5$

KOR) B: 이웃 C, D. $(2 + 3)/2 = 2.5$

ENG) C: neighbours A, B, D, E. $(2 + 0 + 3 + -2)/4 = 0.75$

KOR) C: 이웃 A, B, D, E. $(2 + 0 + 3 + -2)/4 = 0.75$

ENG) D: neighbours A, B, C. $(2 + 0 + 2)/3 = 1.3333$

KOR) D: 이웃 A, B, C. $(2 + 0 + 2)/3 = 1.3333$

ENG) E: neighbours C. $(2)/1 = 2$

KOR) E: 이웃 C. $(2)/1 = 2$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $H_1 = [2.5, 2.5, 0.75, 1.3333, 2]$

ENG)

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 각 노드 옆의 수가 초기 특징이다.

집계 함수가 평균인 GraphSAGE 모형에서, 노드 i 의 층 k 특징은 이웃의 평균을 자신의 특징에 이어 붙인 뒤 ReLU를 적용하여 갱신된다.

- GraphSAGE의 핵심 착상은 무엇이며 기존 GCN과 어떻게 다른지 서술하시오.
- $k = 1$ 에서 각 노드의 특징을 계산하시오.
- $k = 1$ 일 때 평균 전역 풀링으로 그래프 수준 임베딩 h_G 를 계산하시오.

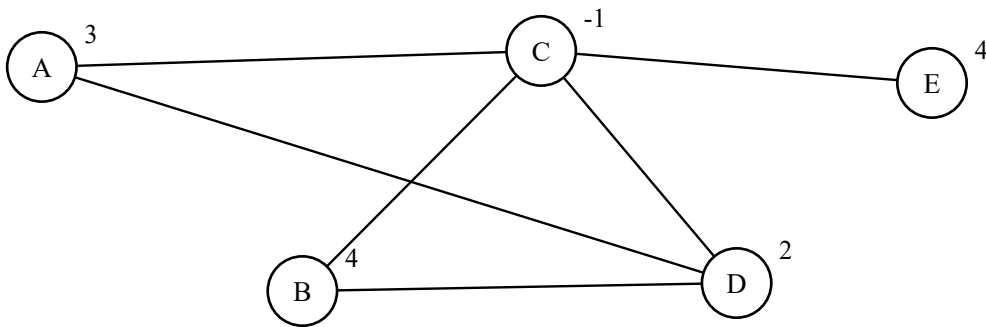


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE는 노드마다 고정된 벡터를 배우는 대신 모으는 함수를 배워요.
 - 이웃을 정해진 개수만 뽑아요. 그래서 아주 큰 그래프에서도 미니배치로 학습할 수 있어요.
 - GCN은 전이적이고 GraphSAGE는 귀납적이에요. 그래서 학습 때 못 본 노드도 벡터를 만들 수 있어요.

ENG) First compute the neighbour mean. The node itself is not included.

KOR) 먼저 이웃 평균을 구한다. 자기 자신은 넣지 않는다.

ENG) A: neighbours C, D. $(-1 + 2)/2 = 0.5$

KOR) A: 이웃 C, D. $(-1 + 2)/2 = 0.5$

ENG) B: neighbours C, D. $(-1 + 2)/2 = 0.5$

KOR) B: 이웃 C, D. $(-1 + 2)/2 = 0.5$

ENG) C: neighbours A, B, D, E. $(3 + 4 + 2 + 4)/4 = 3.25$

KOR) C: 이웃 A, B, D, E. $(3 + 4 + 2 + 4)/4 = 3.25$

ENG) D: neighbours A, B, C. $(3 + 4 + -1)/3 = 2$

KOR) D: 이웃 A, B, C. $(3 + 4 + -1)/3 = 2$

ENG) E: neighbours C. $(-1)/1 = -1$

KOR) E: 이웃 C. $(-1)/1 = -1$

ENG) Concatenate the node's own value with the neighbour mean. This is not an addition.

KOR) 자기 값을 앞에, 이웃 평균을 뒤에 이어 붙인다. 더하는 것이 아니다.

ENG) ReLU clips the negatives: $-1 \rightarrow 0$, $-1 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1 \rightarrow 0$, $-1 \rightarrow 0$

ENG) The graph embedding is the mean per position: first $12 / 5$, second $5.25 / 5$

KOR) 그래프 임베딩은 자리별 평균이다. 첫 자리 합 $12 / 5$, 둘째 자리 합 $5.25 / 5$

ENG) b) $h(A) = [3, 0.5]$, $h(B) = [4, 0.5]$, $h(C) = [0, 3.25]$, $h(D) = [2, 2]$, $h(E) = [4, 0]$

ENG) c) $hG = [2.6, 1.25]$

ENG)

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 는 두 개의 군집 G_1 과 G_2 를 포함하며, 각 노드 옆의 수가 초기 특징이다.

ClusterGCN 모형에서는 이웃 중 자신과 같은 군집에 속한 노드만 평균에 포함한다.

- 미니배치 학습이란 무엇이며 ClusterGCN 이 이를 필요로 하는 이유를 서술하시오.
- 층 $k = 1$ 에서 모든 노드의 출력 표현을 계산하시오.

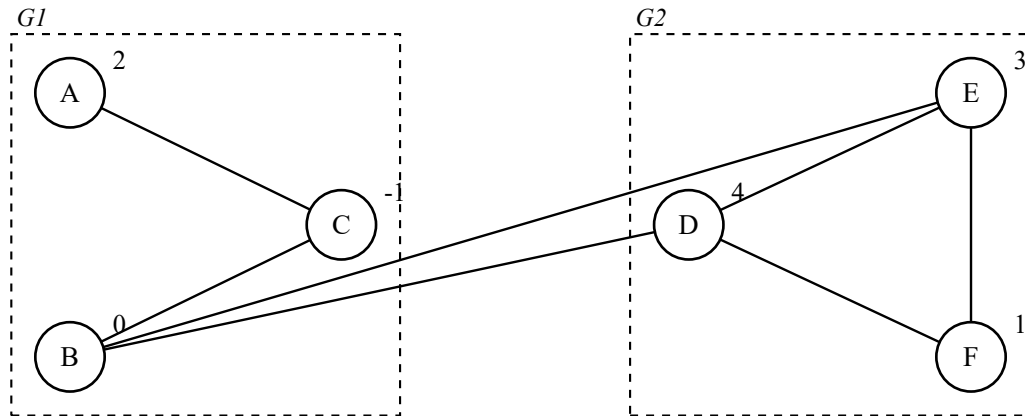


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 미니배치 학습은 한 번에 자료의 일부만 써서 파라미터를 고쳐요.
 - 큰 그래프는 메모리에 안 들어가요. 그래서 ClusterGCN 은 안쪽이 촘촘한 덩어리로 나뉘요.
 - 잘 나누면 선 대부분이 배치 안에 남아서 경계에서 잃는 정보가 적어요.

ENG) Edges that cross a group are treated as absent. Cross them out first.

KOR) 묶음을 넘는 간선은 없는 것으로 본다. 먼저 그 간선에 사선을 그어 지운다.

ENG) A: neighbours C. $(-1)/1 = -1$

KOR) A: 이웃 C. $(-1)/1 = -1$

ENG) B: neighbours C. $(-1)/1 = -1$

KOR) B: 이웃 C. $(-1)/1 = -1$

ENG) C: neighbours A, B. $(2 + 0)/2 = 1$

KOR) C: 이웃 A, B. $(2 + 0)/2 = 1$

ENG) D: neighbours E, F. $(3 + 1)/2 = 2$

KOR) D: 이웃 E, F. $(3 + 1)/2 = 2$

ENG) E: neighbours D, F. $(4 + 1)/2 = 2.5$

KOR) E: 이웃 D, F. $(4 + 1)/2 = 2.5$

ENG) F: neighbours D, E. $(4 + 3)/2 = 3.5$

KOR) F: 이웃 D, E. $(4 + 3)/2 = 3.5$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU to both entries.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다. ReLU 는 두 자리 모두에 걸린다.

ENG) b) $h(A) = [2, 0]$, $h(B) = [0, 0]$, $h(C) = [0, 1]$, $h(D) = [4, 2]$, $h(E) = [3, 2.5]$, $h(F) = [1, 3.5]$

ENG)

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 에는 추출된 부분그래프 G_1 이 주어져 있으며, 각 노드 옆의 수가 초기 특징이다.

GraphSAINT 모형에서는 이웃 중 같은 부분그래프에 속한 노드만 평균에 포함한다.

- GraphSAINT의 추출 전략이 노드 단위 또는 층 단위 추출 방식과 어떻게 다른지 서술하시오.
- 층 $k = 1$ 에서 노드 A, B, C, D의 출력 표현을 계산하시오.

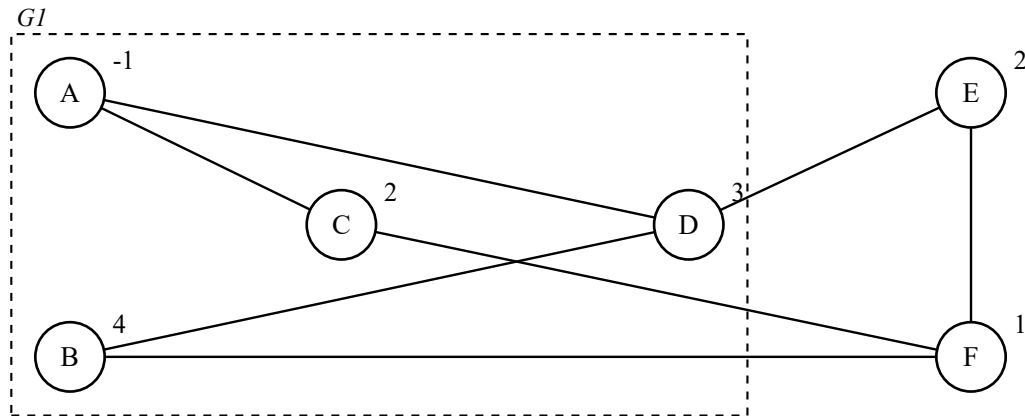


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE와 FastGCN은 층을 쌓으면서 뽑아요. 그래서 층마다 뽑기를 되풀이해요.
 - GraphSAINT는 부분 그래프를 먼저 뽑고 그 위에 GNN을 통째로 올려요.
 - 그래서 깊어져도 비용이 안 늘고, 정규화 계수로 뽑기의 치우침을 바로잡아요.

ENG) Edges that leave the sampled subgraph are discarded.

KOR) 뽑힌 부분그래프 밖으로 나가는 간선은 버린다.

ENG) A: neighbours C, D. $(2 + 3)/2 = 2.5$

KOR) A: 이웃 C, D. $(2 + 3)/2 = 2.5$

ENG) B: neighbours D. $(3)/1 = 3$

KOR) B: 이웃 D. $(3)/1 = 3$

ENG) C: neighbours A. $(-1)/1 = -1$

KOR) C: 이웃 A. $(-1)/1 = -1$

ENG) D: neighbours A, B. $(-1 + 4)/2 = 1.5$

KOR) D: 이웃 A, B. $(-1 + 4)/2 = 1.5$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다.

ENG) b) $h(A) = [0, 2.5]$, $h(B) = [4, 3]$, $h(C) = [2, 0]$, $h(D) = [3, 1.5]$

ENG)

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

JK 신경망의 층 (k) 출력은 $\sigma(\tilde{A} H^{(0)})$ 부터 $\sigma(\tilde{A} H^{(k-1)})$ 까지의 원소별 최댓값으로 계산된다고 가정한다.

- Jumping Knowledge 신경망이 깊은 GCN 의 과평활 문제를 어떻게 완화하는지 설명하시오.
- $\tilde{A}$ 와 $H^{(1)}$ 을 계산하시오.
- 층 $k = 2$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 에서는 노드가 이웃 평균을 거듭 내서 표현들이 다 비슷해져요. 이것이 과평활이에요.
 - JKNet 은 모든 층의 표현을 들고 있다가 끝에서 최댓값, 이어 붙이기, LSTM 으로 합쳐요.
 - 알맞은 깊이는 노드마다 달라요. 그래서 노드마다 자기에게 맞는 깊이를 고를 수 있어요.

ENG) Degrees: $d(A)=3, d(B)=3, d(C)=2, d(D)=3, d(E)=1$

KOR) 차수를 센다. $d(A)=3, d(B)=3, d(C)=2, d(D)=3, d(E)=1$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333];$

$[0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 1, 0, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 1, 0, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0]; [0, 1, 0, 0, 0]$

ENG) First build $H1 = \text{ReLU}(\tilde{A} H0)$

KOR) 먼저 $H1$ 을 만든다. $H1 = \text{ReLU}(\tilde{A} H0)$

ENG) A: neighbours B, C, D. $(2 + -3 + 2)/3 = 0.3333$

KOR) A: 이웃 B, C, D. $(2 + -3 + 2)/3 = 0.3333$

ENG) B: neighbours A, D, E. $(-2 + 2 + 1)/3 = 0.3333$

KOR) B: 이웃 A, D, E. $(-2 + 2 + 1)/3 = 0.3333$

ENG) C: neighbours A, D. $(-2 + 2)/2 = 0$

KOR) C: 이웃 A, D. $(-2 + 2)/2 = 0$

ENG) D: neighbours A, B, C. $(-2 + 2 + -3)/3 = -1$

KOR) D: 이웃 A, B, C. $(-2 + 2 + -3)/3 = -1$

ENG) E: neighbours B. $(2)/1 = 2$

KOR) E: 이웃 B. $(2)/1 = 2$

ENG) ReLU clips the negatives: $-1 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1 \rightarrow 0$

ENG) $H1 = [0.3333, 0.3333, 0, 0, 2]$

ENG) For $k = 2$ there are two terms. First: $\text{ReLU}(\tilde{A} H0) = [0.3333, 0.3333, 0, 0, 2]$

KOR) $k=2$ 이므로 조각이 두 개다. 첫째 $\text{ReLU}(\tilde{A} H0) = [0.3333, 0.3333, 0, 0, 2]$

ENG) Second: $\text{ReLU}(\tilde{A} H1) = [0.1111, 0.7778, 0.1667, 0.2222, 0.3333]$

KOR) 둘째 $\text{ReLU}(\tilde{A} H1) = [0.1111, 0.7778, 0.1667, 0.2222, 0.3333]$

ENG) The max is taken element-wise, not by choosing one whole vector.

KOR) max 는 가로줄마다 큰 쪽을 고른다. 벡터 하나를 고르는 것이 아니다.

ENG) c) $H2 = [0.3333, 0.7778, 0.1667, 0.2222, 2]$

ENG)

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.5$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCNII 모형의 층 (k) 출력은 $H^{(k)} = \sigma[((1-\beta)I_n) \cdot ((1-\alpha)\tilde{A} H^{(k-1)} + \alpha H^{(0)})]$ 로 계산된다고 가정한다. I_n 은 단위행렬이며 α 와 β 는 문제에 주어진 값을 사용한다.

- GCNII 가 GCN 의 깊이 한계를 극복하기 위해 도입한 두 가지 주요 구성 요소를 쓰시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -1 \\ 2 \\ -3 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 두 장치는 초기 잔차 연결과 항등 사상이에요.
 - 초기 잔차 연결은 층마다 입력 $H^{(0)}$ 의 일부를 알파만큼 다시 더해요.
 - 항등 사상은 가중치 행렬을 단위행렬에 가깝게 뒹요. 그래서 깊어져도 신호가 안 망가져요.

ENG) Degrees: $d(A)=3$, $d(B)=1$, $d(C)=2$, $d(D)=3$, $d(E)=3$

KOR) 차수를 센다. $d(A)=3$, $d(B)=1$, $d(C)=2$, $d(D)=3$, $d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0, 0, 1]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0, 0, 1]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0, 0, 1]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$

ENG) Start with the inner bracket. $H_0 = [-0.3333, 0, 0, 1.6667, -0.3333]$

KOR) 안쪽 괄호부터 푼다. $H_0 = [-0.3333, 0, 0, 1.6667, -0.3333]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[-0.2667, 0, 0, 1.3333, -0.2667]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[-0.2667, 0, 0, 1.3333, -0.2667]$

ENG) Add $\alpha H_0 = 0.2 \times [3, -1, 2, -3, 0] = [0.6, -0.2, 0.4, -0.6, 0]$

KOR) $\alpha H_0 = 0.2 \times [3, -1, 2, -3, 0] = [0.6, -0.2, 0.4, -0.6, 0]$ 를 더한다.

ENG) Inner bracket = $[0.3333, -0.2, 0.4, 0.7333, -0.2667]$

KOR) 안쪽 괄호 = $[0.3333, -0.2, 0.4, 0.7333, -0.2667]$

ENG) Multiply by the outer $(1-\beta) = 0.5$. Dropping this factor scales the answer.

KOR) 바깥의 $(1-\beta) = 0.5$ 를 곱한다. 이 계수를 빼뜨리면 답이 배로 커진다.

ENG) ReLU clips the negatives: $-0.1 \rightarrow 0, -0.1333 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.1 \rightarrow 0, -0.1333 \rightarrow 0$

ENG) c) $H_1 = [0.1667, 0, 0.2, 0.3667, 0]$

ENG)

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows: Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

잔차 연결을 갖는 DeepGCN 의 층 (k) 출력은 $H^{(k)} = \sigma(\tilde{A} H^{(k-1)}) + H^{(k-1)}$ 로 계산된다고 가정한다. 덧셈은 ReLU 적용 이후에 이루어진다.

- 잔차 연결이 GCN 을 더 깊게 쌓을 수 있게 하는 이유를 설명하시오.
- 층 $k = 1$ 과 $k = 2$ 에서의 출력 표현을 계산하시오.
- 잔차 연결이 없는 보통의 GCN 출력과 $H^{(1)}$ 을 비교하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 3 \\ 3 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 은 기울기 소실과 과평활을 겪어요. 그래서 층을 더하면 오히려 나빠져요.
 - 잔차 연결은 입력을 그대로 앞으로 보내요. 그래서 층은 바뀌는 양만 배우면 돼요.
 - 바꿀 게 없으면 층은 0 을 내보내면 돼요. 기울기도 그 지름길로 되돌아 흘러요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.3333, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.3333, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) Atilde $H_0 = [2, 2, 1, 1.6667]$

ENG) Apply ReLU first: $[2, 2, 1, 1.6667]$

KOR) ReLU 를 먼저 건다. $[2, 2, 1, 1.6667]$

ENG) Then add H_0 . The addition sits outside the ReLU, so negatives can survive.

KOR) 그 다음에 H_0 을 더한다. 더하기는 ReLU 바깥이라 음수가 살아남을 수 있다.

ENG) For $k = 2$, feed H_1 back in and add H_1 , not H_0 .

KOR) $k=2$ 에서는 H_1 을 다시 넣고, 더하는 것도 H_0 이 아니라 H_1 이다.

ENG) Atilde $H_1 = [3.3333, 3.3333, 2.8889, 3.3333]$, after ReLU $[3.3333, 3.3333, 2.8889, 3.3333]$

KOR) Atilde $H_1 = [3.3333, 3.3333, 2.8889, 3.3333]$, ReLU 뒤 $[3.3333, 3.3333, 2.8889, 3.3333]$

ENG) b) $H_1 = [1, 5, 4, 2.6667]$

ENG) $H_2 = [4.3333, 8.3333, 6.8889, 6]$

ENG) c) Plain GCN without residual: $H_1 = [2, 2, 1, 1.6667]$ (the node's own value and negatives disappear)

KOR) c) 잔차 없는 GCN 의 $H_1 = [2, 2, 1, 1.6667]$ (잔차가 없으면 자기 값과 음수가 사라져요)

2회차

ENG)

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 노드 1과 5에는 이름표가 주어져 있으며($P(Y_1)=1, P(Y_5)=0$), 나머지 노드는 이름표가 없고 0.5로 초기화한다.

관계형 분류기는 이름표가 없는 각 노드를 이웃 노드들의 부류 확률에 대한 가중 평균으로 갱신한다. 분모는 해당 노드에 연결된 간선 가중치의 합이다.

- 관계형 분류가 전제하는 핵심 가정과 그 한계 하나를 서술하시오.
- 1차 반복 후 이름표가 없는 각 노드의 $P(Y_v)$ 를 계산하시오.
- 2차 반복 후의 $P(Y_v)$ 를 계산하시오. 단, 직전 반복의 값만 사용한다.

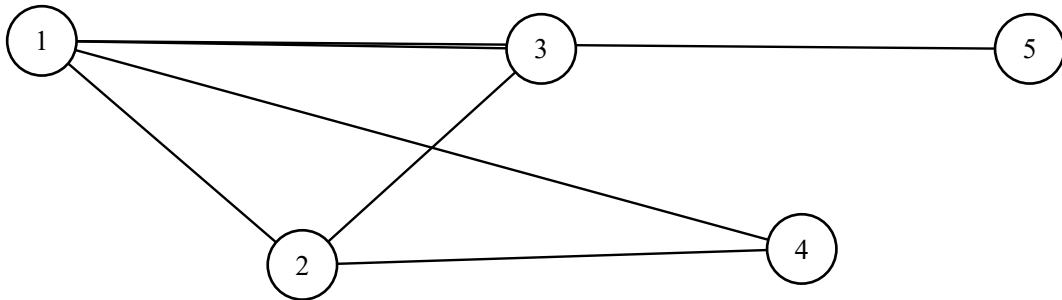


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 관계형 분류는 동질성을 가정해요. 이어진 노드끼리는 같은 이름표를 가지는 경향이 있다는 뜻이에요.
 - 이름표 없는 노드를 이웃들의 확률의 가중 평균으로 거듭 바뀌요.
 - 한계는 그래프 구조만 쓰고 노드의 특징은 무시한다는 것이에요.

ENG) Write the initial values: node 1 is 1, node 5 is 0, the rest start at 0.5.

KOR) 처음 값을 적는다. 이름표가 붙은 1번은 1, 5번은 0, 나머지는 0.5 로 시작한다.

ENG) First iteration. Only the unlabelled nodes are replaced by the mean of their neighbours.

KOR) 1차 반복. 이름표가 없는 노드만 이웃의 평균으로 바꾼다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

ENG) Node 3: neighbours 1, 2. $(1 + 0.5)/2 = 0.75$

KOR) 3번: 이웃 1, 2. $(1 + 0.5)/2 = 0.75$

ENG) Node 4: neighbours 1, 2. $(1 + 0.5)/2 = 0.75$

KOR) 4번: 이웃 1, 2. $(1 + 0.5)/2 = 0.75$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) b) $P(Y1) = 1$, $P(Y2) = 0.6667$, $P(Y3) = 0.75$, $P(Y4) = 0.75$, $P(Y5) = 0$

ENG) Second iteration. Use only the values from the first iteration, not the ones just computed.

KOR) 2차 반복. 방금 구한 값이 아니라 1차 반복의 값만 쓴다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.75 + 0.75)/3 = 0.8333$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.75 + 0.75)/3 = 0.8333$

ENG) Node 3: neighbours 1, 2. $(1 + 0.6667)/2 = 0.8333$

KOR) 3번: 이웃 1, 2. $(1 + 0.6667)/2 = 0.8333$

ENG) Node 4: neighbours 1, 2. $(1 + 0.6667)/2 = 0.8333$

KOR) 4번: 이웃 1, 2. $(1 + 0.6667)/2 = 0.8333$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) c) $P(Y1) = 1$, $P(Y2) = 0.8333$, $P(Y3) = 0.8333$, $P(Y4) = 0.8333$, $P(Y5) = 0$

ENG)

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{D, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

KOR)

인접행렬 A 와 노드 특징 벡터 $h^{(0)}$ 가 주어진 무방향 그래프 G 를 고려한다.

FastGCN 은 GCN 한 층의 전체 이웃 합을 층 단위 몬테카를로 추정으로 대체한다. 각 층에서 t_l 개의 노드를 중요도 분포 $q(u) \propto \|A(:, u)\|^2$ 로부터 한 번 추출하며, 평균은 이웃 수가 아니라 추출 개수 t_l 로 나눈다.

- 층 단위 추출이 GraphSAGE 의 노드 단위 추출과 어떻게 다른지 서술하시오.
- 다섯 노드에 대한 중요도 분포 $q(u)$ 를 계산하시오.
- 주어진 추출 집합($t_l = 2$)에 대하여 $h^{(1)}(A)$ 의 추정값을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ -2 \\ 3 \\ 1 \\ -1 \end{bmatrix}$$

풀이**ENG)**

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 뽑기는 노드마다 제 이웃을 뽑아서, 층이 깊어질수록 비용이 지수로 커져요.
 - FastGCN 은 층마다 정해진 개수를 한 번만 뽑고, 그 층의 모든 노드가 같이 써요.
 - 그래서 비용이 지수가 아니라 층 수에 비례해서만 늘어요.

ENG) Degrees: $d(A)=3, d(B)=3, d(C)=1, d(D)=3, d(E)=2$

KOR) 차수를 센다. $d(A)=3, d(B)=3, d(C)=1, d(D)=3, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) $q(u)$ uses the squared norm of column u of $\tilde{A}$, not of a row.

KOR) $q(u)$ 는 $\tilde{A}$ 의 **세로줄** 제곱합이다. 가로줄이 아니다.

ENG) Column A: squared sum = 1.2222

KOR) A 칸의 제곱합 = 1.2222

ENG) Column B: squared sum = 0.4722

KOR) B 칸의 제곱합 = 0.4722

ENG) Column C: squared sum = 0.1111

KOR) C 칸의 제곱합 = 0.1111

ENG) Column D: squared sum = 0.4722

KOR) D 칸의 제곱합 = 0.4722

ENG) Column E: squared sum = 0.2222

KOR) E 칸의 제곱합 = 0.2222

ENG) Divide by the total 2.5 to turn them into probabilities.

KOR) 전체 합 2.5 로 나눠 확률로 만든다.

ENG) b) $q(A) = 0.4889, q(B) = 0.1889, q(C) = 0.0444, q(D) = 0.1889, q(E) = 0.0889$

ENG) For (c) look only at the sampled nodes: read $\tilde{A}(A, u)$ for each.

KOR) c) 는 뽑힌 노드만 본다. A 가로줄에서 그 칸의 $\tilde{A}$ 값을 읽는다.

ENG) $\tilde{A}(A, D) h(D) = 0.3333 \times 1 = 0.3333$

KOR) $\tilde{A}(A, D) \times h(D) = 0.3333 \times 1 = 0.3333$

ENG) $\tilde{A}(A, E) h(E) = 0 \times 1 = 0$

KOR) $\tilde{A}(A, E) \times h(E) = 0 \times 1 = 0$

ENG) Add them and divide by the number sampled (2), not by the degree: $(0.3333 + 0)/2 = 0.1667$

KOR) 더한 뒤 이웃 수가 아니라 **뽑은 개수 2** 로 나눈다. $(0.3333 + 0)/2 = 0.1667$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $h_1(A) = 0.1667$

ENG)

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = C$.
- Calculate $E_{p(u_j | C)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(C)$.

KOR)

문제 11 과 동일한 그래프와 특징 벡터를 고려한다.

ASGCN 은 GCN 한 층을 확률 분포에 대한 기댓값으로 다시 쓴다. 여기서 $N(v_i) = \sum_j \tilde{A}_{ij}$ 는 정규화 계수이고, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$ 는 부모 노드 v_i 가 주어졌을 때 노드 u_j 를 추출할 확률이다.

- ASGCN 의 제안 분포가 FastGCN 과 달리 무엇에 의존하는지 서술하시오.
- 주어진 부모 노드에 대한 $N(v_i)$ 와 $p(u_j | v_i)$ 를 계산하시오.
- 해당 기댓값과 그 결과로 얻어지는 $h^{(1)}$ 을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 4 \\ -1 \\ 2 \\ 2 \\ 1 \end{bmatrix}$$

풀이**ENG)**

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - FastGCN 의 뽑기 확률은 미리 정해져 있고 부모 노드와 상관이 없어요.
 - ASGCN 은 뽑기 확률을 윗층 노드에 맞춰 정해요.
 - 그래서 뽑은 노드가 거의 다 쓸모 있고, 추정값의 흔들림이 줄어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$, $d(E)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$, $d(E)=2$

ENG) Divide each row by its degree. Atilde = $[0, 0.5, 0.5, 0, 0]$; $[0.5, 0, 0, 0.5, 0]$; $[0.3333, 0, 0, 0.3333, 0.3333]$; $[0, 0.3333, 0.3333, 0, 0.3333]$; $[0, 0, 0.5, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.3333, 0.3333, 0, 0.3333]; [0, 0, 0.5, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) The parent node is C, so only its row of $\tilde{A}$ matters.

KOR) 부모 노드는 C 다. $\tilde{A}$ 의 그 줄만 본다.

ENG) $N(C) = \text{sum of that row} = 1$. With row normalisation it is 1.

KOR) $N(C) = \text{그 가로줄의 합} = 1$. 행 정규화라 1 이다.

ENG) $p(A|C) = \tilde{A}(C,A) / N = 0.3333$

KOR) $p(A|C) = \tilde{A}(C,A) / N = 0.3333 / 1 = 0.3333$

ENG) $p(D|C) = \tilde{A}(C,D) / N = 0.3333$

KOR) $p(D|C) = \tilde{A}(C,D) / N = 0.3333 / 1 = 0.3333$

ENG) $p(E|C) = \tilde{A}(C,E) / N = 0.3333$

KOR) $p(E|C) = \tilde{A}(C,E) / N = 0.3333 / 1 = 0.3333$

ENG) b) $N(C) = 1$, $p(A|C) = 0.3333$, $p(D|C) = 0.3333$, $p(E|C) = 0.3333$

ENG) The expectation is the sum of probability times value: $0.3333 \times 4 + 0.3333 \times 2 + 0.3333 \times 1 = 2.3333$

KOR) 기댓값은 확률 곱하기 값의 합이다. $0.3333 \times 4 + 0.3333 \times 2 + 0.3333 \times 1 = 2.3333$

ENG) $N = 1$ and $W = 1$, so the value is unchanged. It equals the plain neighbour mean.

KOR) $N = 1$, $W = 1$ 이라 곱해도 그대로다. 결국 이웃 평균과 같은 값이다.

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $E = 2.3333$, $h1(C) = 2.3333$

ENG)

4. (APPPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

APPPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.5$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

KOR)

인접행렬 A 와 다층 퍼셉트론이 산출한 예측 행렬 H 가 주어졌다.

APPPNP 는 순간이동 확률 α 를 사용하여 예측을 전파한다. 전파 단계 사이에는 비선형 함수를 적용하지 않는다.

- 개인화 PageRank 의 순간이동 기법을 설명하고, 그것이 과평활을 막는 이유를 서술하시오.
- $\tilde{A}$ 와 $Z^{(1)}$ 을 계산하시오.
- $Z^{(2)}$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 퍼뜨리는 걸음마다 확률 알파로 출발 노드로 되돌아가요. 이것이 순간이동 장치예요.
 - 매 걸음 일정 비율이 되돌아오니까 분포가 출발점 근처에 머물러요.
 - 그래서 많이 퍼뜨려도 과평활이 안 생기고, 퍼뜨리는 데에 추가 파라미터도 없어요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=3, d(D)=1$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=3, d(D)=1$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0]$

ENG) Start from $Z_0 = H = [0, 0, 0, 1]$

KOR) $Z_0 = H$ 로 시작한다. $[0, 0, 0, 1]$

ENG) Propagate one step: $\tilde{Z}_0 = [0, 0, 0.3333, 0]$

KOR) 한 걸음 퍼뜨린다. $\tilde{Z}_0 = [0, 0, 0.3333, 0]$

ENG) Multiply by $(1-\alpha) = 0.5$: $[0, 0, 0.1667, 0]$

KOR) $(1-\alpha) = 0.5$ 를 곱한다. $[0, 0, 0.1667, 0]$

ENG) Add $\alpha H = [0, 0, 0, 0.5]$. This is the teleport term.

KOR) $\alpha H = 0.5 \times [0, 0, 0, 1] = [0, 0, 0, 0.5]$ 를 더한다. 이것이 순간이동이다.

ENG) $Z_1 = [0, 0, 0.1667, 0.5]$

ENG) One more step: $\tilde{Z}_1 = [0.0833, 0.0833, 0.1667, 0.1667]$

KOR) 한 걸음 더. $\tilde{Z}_1 = [0.0833, 0.0833, 0.1667, 0.1667]$

ENG) Multiply by $(1-\alpha)$ and add αH **again**. It is added at every step.

KOR) $(1-\alpha)$ 를 곱하고 **αH 를 또** 더한다. 매 걸음 더하는 것이 핵심이다.

ENG) c) $Z_2 = [0.0417, 0.0417, 0.0833, 0.5833]$

ENG)

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.3$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

KOR)

인접행렬 A 와 신경망이 산출한 은닉 특징 $H^{(0)}$ 이 주어졌다.

GPR-GNN 은 특징 추출과 위상 전파를 분리한다. $\tilde{A}$ 를 반복 적용하여 $H^{(k)}$ 를 얻고, 학습된 가중치 γ_k 로 가중합한다. 본 문항에서는 $K = 2$ 이며 $\gamma_0, \gamma_1, \gamma_2$ 가 주어진다.

- GPR-GNN 이 APPNP 와 달리 할 수 있는 것은 무엇이며, γ_k 가 음수가 될 수 있다는 점이 이질적 그래프에서 왜 중요한지 서술하시오.
- $H^{(1)}$ 과 $H^{(2)}$ 를 계산하시오.
- Z 를 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 0 \\ -1 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - APPNP 는 계수를 알파 곱하기 (1-알파)의 k제곱으로 못 박아요. 늘 양수이고 줄어들어요.
 - GPR-GNN 은 그 무게를 학습하고, 음수도 될 수 있어요.
 - 음수 무게는 그 걸음을 빼요. 그래서 이질 그래프에서 고주파 필터처럼 동작할 수 있어요.

ENG) Degrees: d(A)=2, d(B)=1, d(C)=2, d(D)=3

KOR) 차수를 센다. d(A)=2, d(B)=1, d(C)=2, d(D)=3

ENG) Divide each row by its degree. Atilde = [0, 0, 0.5, 0.5]; [0, 0, 0, 1]; [0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0] (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. Atilde = [0, 0, 0.5, 0.5]; [0, 0, 0, 1]; [0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0] (가로줄 합이 모두 1)

ENG) H_0 stays as it is: $[3, 0, -1, -2]$

KOR) H_0 은 그대로 둔다. $[3, 0, -1, -2]$

ENG) $H_1 = \text{Atilde } H_0 = [-1.5, -2, 0.5, 0.6667]$

ENG) $H_2 = \text{Atilde } H_1 = [0.5833, 0.6667, -0.4167, -1]$

ENG) b) $H_1 = [-1.5, -2, 0.5, 0.6667]$, $H_2 = [0.5833, 0.6667, -0.4167, -1]$

ENG) Now take the weighted sum with gamma. There is no ReLU during propagation.

KOR) 이제 gamma 로 가중합한다. 전파 중에 ReLU 는 없다.

ENG) $\text{gamma}_0 H_0 = 0.7 \times [3, 0, -1, -2] = [2.1, 0, -0.7, -1.4]$

KOR) $\text{gamma}_0 \times H_0 = 0.7 \times [3, 0, -1, -2] = [2.1, 0, -0.7, -1.4]$

ENG) $\text{gamma}_1 H_1 = 0.3 \times [-1.5, -2, 0.5, 0.6667] = [-0.45, -0.6, 0.15, 0.2]$

KOR) $\text{gamma}_1 \times H_1 = 0.3 \times [-1.5, -2, 0.5, 0.6667] = [-0.45, -0.6, 0.15, 0.2]$

ENG) $\text{gamma}_2 H_2 = -0.2 \times [0.5833, 0.6667, -0.4167, -1] = [-0.1167, -0.1333, 0.0833, 0.2]$

KOR) $\text{gamma}_2 \times H_2 = -0.2 \times [0.5833, 0.6667, -0.4167, -1] = [-0.1167, -0.1333, 0.0833, 0.2]$

ENG) gamma_2 is negative, so that hop is subtracted. Dropping the sign gives a wrong answer.

KOR) gamma_2 가 음수라 그 걸음은 빼는 효과다. 부호를 빼뜨리면 틀린다.

ENG) Add the three vectors element-wise.

KOR) 세 벡터를 자리별로 더한다.

ENG) c) $Z = [1.5333, -0.7333, -0.4667, -1]$

ENG)

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

MixHop 한 층은 인접행렬의 여러 거듭제곱 $\hat{A}^j$ ($j \in P = \{0, 1, 2\}$)를 각각 계산한 뒤 열 방향으로 이어 붙인다.

본 문항에서 가중치 행렬은 모두 1 로 둔다.

a) 두 걸음 델타 연산자에 관한 MixHop 의 정리 1 을 서술하고, 보통의 GCN 이 층을 무한히 쌓아도 이를 표현할 수 없는 이유를 설명하시오.

b) $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$, $\hat{A}^2 H^{(0)}$ 을 계산하시오.

c) $H^{(1)}$ 과 델타 연산자 $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 정리 1 은 보통 GCN 이 두 걸음 델타 연산자를 표현할 수 없다고 말해요. 층을 무한히 쌓아도요.
 - GCN 층을 쌓으면 A 에 l 제곱 곱하기 W 꼴로 뭉쳐져서, 중간 걸음들이 따로 남지 않아요.
 - MixHop 은 거듭제곱마다 따로 구해서 나란히 붙여요. 그래서 뒤 층이 하나에서 다른 하나를 뺄 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=1$, $d(D)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=1$, $d(D)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0, 0, 1]; [0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0, 0, 1]; [0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) A^0 is the identity, so it is just $H_0: [2, 3, 4, -1]$

KOR) A_0 은 단위행렬이라 H_0 그대로다. $[2, 3, 4, -1]$

ENG) $A^1 H_0 = [1, 0.5, -1, 3]$ (one hop, the neighbour mean)

KOR) $A_1 H_0 = [1, 0.5, -1, 3]$ (한 걸음, 이웃 평균)

ENG) $A^2 H_0 = \tilde{A}(\tilde{A} H_0) = [1.75, 2, 3, 0.1667]$ (two hops)

KOR) $A_2 H_0 = \tilde{A}(\tilde{A} H_0) = [1.75, 2, 3, 0.1667]$ (두 걸음)

ENG) ReLU is applied to each block separately, not after concatenation.

KOR) 조각마다 따로 ReLU 를 건다. 이어 붙인 뒤가 아니다.

ENG) After ReLU: $[2, 3, 4, 0] \parallel [1, 0.5, 0, 3] \parallel [1.75, 2, 3, 0.1667]$

KOR) ReLU 뒤: $[2, 3, 4, 0] \parallel [1, 0.5, 0, 3] \parallel [1.75, 2, 3, 0.1667]$

ENG) The delta operator subtracts the two-hop block from the one-hop block.

KOR) 델타 연산자는 한 걸음에서 두 걸음을 뺀 것이다.

ENG) b) $A_0 H = [2, 3, 4, -1]$, $A_1 H = [1, 0.5, -1, 3]$, $A_2 H = [1.75, 2, 3, 0.1667]$

ENG) c) $H_1 =$ concatenation of the three blocks: $[2, 3, 4, 0] \parallel [1, 0.5, 0, 3] \parallel [1.75, 2, 3, 0.1667]$

KOR) c) $H_1 =$ 세 조각을 이어 붙여요: $[2, 3, 4, 0] \parallel [1, 0.5, 0, 3] \parallel [1.75, 2, 3, 0.1667]$

ENG) $\Delta = [-0.75, -1.5, -3, 2.8333]$

ENG)

7. (PairNorm) After one GCN layer the node features of a four-node graph are PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

KOR)

GCN 한 층을 통과한 뒤의 노드 특징 행렬 X 가 주어졌다. 행은 노드, 열은 특징 차원이다.

PairNorm 은 중심화 단계와 크기 조정 단계로 정규화한다. 중심화는 노드들에 대한 평균을 빼는 것이고, 크기 조정은 중심화된 벡터들의 노름 제곱 평균의 제곱근으로 나누는 것이다. $s = 1, n = 4$ 로 한다.

- 노드 단위 과평활과 특징 단위 과평활을 각각 설명하고, PairNorm 이 층에 걸쳐 일정하게 유지하는 양이 무엇인지 서술하시오.
- 중심화된 특징을 계산하시오.
- 크기 조정의 분모와 최종 x_i 를 계산하시오.

$$X = \begin{bmatrix} 0 & 4 \\ 1 & 3 \\ 1 & -2 \\ -2 & 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 과평활은 이어진 노드들의 특징이 너무 비슷해지는 것이에요.
 - 특징 단위 과평활은 그래프 전체에서 모든 노드 특징이 구별이 안 되게 되는 것이에요.
 - PairNorm 은 층을 지나도 노드 사이 거리 제곱의 합을 일정하게 지켜요.

ENG) First take the mean over the nodes, separately for each feature column.

KOR) 먼저 노드들에 대한 평균을 구한다. 특징 차원마다 따로 구한다.

ENG) Column 1: $(0 + 1 + 1 + -2)/4 = 0$

KOR) 첫 칸: $(0 + 1 + 1 + -2)/4 = 0$

ENG) Column 2: $(4 + 3 + -2 + 1)/4 = 1.5$

KOR) 둘째 칸: $(4 + 3 + -2 + 1)/4 = 1.5$

ENG) Subtract the mean from every row. Check: each column now sums to zero.

KOR) 가로줄마다 평균을 뺀다. 검산: 각 칸의 합이 0 이 된다.

ENG) Node 1: squared norm = 6.25

KOR) 노드 1 의 길이 제곱 = 6.25

ENG) Node 2: squared norm = 3.25

KOR) 노드 2 의 길이 제곱 = 3.25

ENG) Node 3: squared norm = 13.25

KOR) 노드 3 의 길이 제곱 = 13.25

ENG) Node 4: squared norm = 4.25

KOR) 노드 4 의 길이 제곱 = 4.25

ENG) Mean = $(6.25 + 3.25 + 13.25 + 4.25)/4 = 6.75$

KOR) 평균 = $(6.25 + 3.25 + 13.25 + 4.25)/4 = 6.75$

ENG) Take the square root: $\sqrt{6.75} = 2.5981$. Divide every row by this single number.

KOR) 제곱근을 씌운다. $\sqrt{6.75} = 2.5981$. 이 **하나의 수** 로 모든 줄을 나눈다.

ENG) b) Mean = [0, 1.5], centered = [0, 2.5]; [1, 1.5]; [1, -3.5]; [-2, -0.5]

KOR) b) 평균 = [0, 1.5], 중심을 뺀 것 = [0, 2.5]; [1, 1.5]; [1, -3.5]; [-2, -0.5]

ENG) c) Scaling denominator = 2.5981, $x = [0, 0.9623]; [0.3849, 0.5774]; [0.3849, -1.3472]; [-0.7698, -0.1925]$

KOR) c) 분모 = 2.5981, $x = [0, 0.9623]; [0.3849, 0.5774]; [0.3849, -1.3472]; [-0.7698, -0.1925]$

ENG)

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (C, D) and (A, C) are dropped. Recalculate $H^{(1)}$.

KOR)

노드 A, B, C, D 로 이루어진 완전그래프 K_4 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

DropEdge 는 매 에폭마다 각 무방향 간선을 확률 $1-p$ 로 독립적으로 유지한다. 이때 $A^{(t)}$ 와 $D^{(t)}$ 는 살아남은 간선만으로 다시 구성한다. $p = 1/3$ 으로 한다.

- DropEdge 가 목표로 하는 두 가지 문제를 쓰고, Dropout 과 어떻게 다른지 서술하시오.
- 살아남은 간선 수의 기댓값과, 간선을 제거하지 않은 원래 그래프에서의 $H^{(1)}$ 을 계산하시오.
- 주어진 두 간선이 제거되었을 때의 $H^{(1)}$ 을 다시 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -2 \\ 2 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - DropEdge 는 과적합과 과평활을 노려요.
 - 판마다 다른 부분 그래프를 써요. 그래프에 자료를 늘리는 효과가 나요.
 - Dropout 은 특징 값을 0 으로 꺼요. DropEdge 는 인접행렬의 선을 지워요.

ENG) K_4 has 6 edges. Each survives with probability $1 - 1/3 = 2/3$, so the expectation is 4.

KOR) K_4 는 간선이 6 개다. 남을 확률이 $1 - 1/3 = 2/3$ 이므로 기댓값은 $6 \times 2/3 = 4$ 개다.

ENG) First the full graph: every node has degree 3.

KOR) 먼저 원래 그래프. 모든 노드의 차수가 3 다.

ENG) A: neighbours B, C, D. $(-2 + 2 + 2)/3 = 0.6667$

KOR) A: 이웃 B, C, D. $(-2 + 2 + 2)/3 = 0.6667$

ENG) B: neighbours A, C, D. $(0 + 2 + 2)/3 = 1.3333$

KOR) B: 이웃 A, C, D. $(0 + 2 + 2)/3 = 1.3333$

ENG) C: neighbours A, B, D. $(0 + -2 + 2)/3 = 0$

KOR) C: 이웃 A, B, D. $(0 + -2 + 2)/3 = 0$

ENG) D: neighbours A, B, C. $(0 + -2 + 2)/3 = 0$

KOR) D: 이웃 A, B, C. $(0 + -2 + 2)/3 = 0$

ENG) Now drop the two given edges and **recount the degrees**. This is where most mistakes happen.

KOR) 이제 주어진 두 간선을 지운다. **차수를 다시 센다**. 여기서 거의 다 틀린다.

ENG) Degrees after dropping: $d(A)=2$, $d(B)=3$, $d(C)=1$, $d(D)=2$

KOR) 지운 뒤 차수: $d(A)=2$, $d(B)=3$, $d(C)=1$, $d(D)=2$

ENG) A: neighbours B, D. $(-2 + 2)/2 = 0$

KOR) A: 이웃 B, D. $(-2 + 2)/2 = 0$

ENG) B: neighbours A, C, D. $(0 + 2 + 2)/3 = 1.3333$

KOR) B: 이웃 A, C, D. $(0 + 2 + 2)/3 = 1.3333$

ENG) C: neighbours B. $(-2)/1 = -2$

KOR) C: 이웃 B. $(-2)/1 = -2$

ENG) D: neighbours A, B. $(0 + -2)/2 = -1$

KOR) D: 이웃 A, B. $(0 + -2)/2 = -1$

ENG) b) Expected number of surviving edges = 4, $H1 = [0.6667, 1.3333, 0, 0]$

KOR) b) 남는 선 수의 기댓값 = 4, $H1 = [0.6667, 1.3333, 0, 0]$

ENG) c) $H1 = [0, 1.3333, 0, 0]$

ENG)

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

KOR)

다음 그림의 무방향 그래프 G 를 고려한다. 임의보행이 노드 A 에서 노드 B 로 이동하여 현재 B 에 있으며, $p = 2, q = 0.5$ 이다.

B 에서 다음 노드 x 로의 비정규화 전이확률은 $\pi_{Bx} = \alpha_{pq}(A, x)$ 이며, α 는 직전 노드 A 로부터의 최단경로 거리에 따라 결정된다.

- BFS 와 DFS 를 설명하고, q 가 작을 때 보행이 둘 중 어느 쪽처럼 동작하는지 서술하시오.
- 노드 B 에서 이동 가능한 모든 노드를 열거하고 비정규화 전이확률 π_{Bx} 를 계산하시오.
- 정규화된 전이확률을 계산하시오.

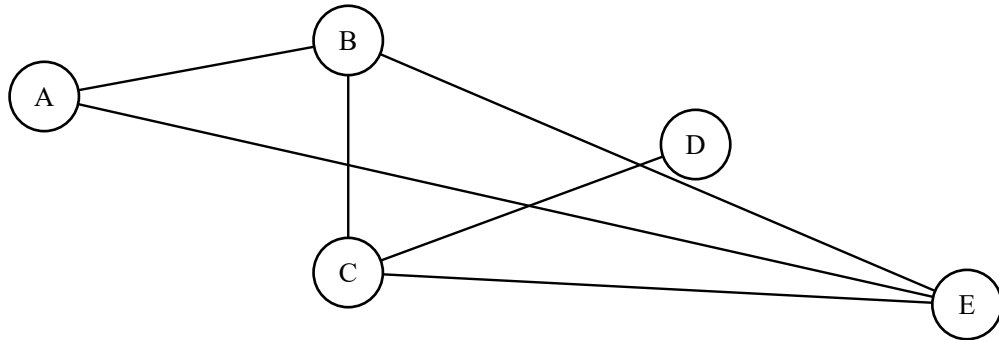


Figure 1. Graph G

풀이

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - BFS 는 바로 옆 이웃부터 훑어요. 그래서 임베딩이 노드의 구조적 역할을 담아요.
 - DFS 는 출발점에서 멀리 가요. 그래서 임베딩이 어느 동네 소속인지를 담아요.
 - q 가 작으면 $1/q$ 가 커져서 걸음이 바깥으로 나가요. DFS 처럼 움직여요.

ENG) The previous node is A and the walk is now at B. All distances are measured from A.

KOR) 직전 노드는 A, 지금 노드는 B 다. 거리는 모두 A 로부터 잴다.

ENG) A: $x = A$ is the previous node, $d = 0$, $\alpha = 1/p = 0.5$

KOR) A: $x = A$ 는 직전 노드라 $d = 0$. $\alpha = 1/p = 0.5$

ENG) C: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) C: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) E: x is also a neighbour of A, $d = 1$, $\alpha = 1 = 1$

KOR) E: A 와도 이웃이라 $d = 1$. $\alpha = 1 = 1$

ENG) Sum them: $0.5 + 2 + 1 = 3.5$

KOR) 세 값을 더한다. $0.5 + 2 + 1 = 3.5$

ENG) b) $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow E) = 1$

ENG) c) $P(A) = 0.1429$, $P(C) = 0.5714$, $P(E) = 0.2857$

ENG)

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = 1, u'_C = 0.5, u'_D = -1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- What is second-order proximity, and how does it differ from first-order proximity?
- Calculate $p_2(A|C)$, $p_2(B|C)$, $p_2(C|C)$ and $p_2(D|C)$.

KOR)

노드 임베딩 u 와 맥락 임베딩 u' 가 주어진 무방향 그래프 G 를 고려한다.

2차 근접도는 $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$ 로 정의한다. 지수 함수 값은 문제에 주어진 표를 사용한다.

- 2차 근접도란 무엇이며 1차 근접도와 어떻게 다른지 서술하시오.
- 주어진 기준 노드에 대하여 네 노드의 p_2 를 계산하시오.

풀이

ENG)

- Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 1차 근접도는 선으로 직접 이어진 두 노드 사이의 비슷함이에요.
 - 2차 근접도는 이웃을 많이 공유하는 두 노드의 비슷함이에요. 선이 없어도 돼요.
 - 그래서 LINE 은 노드마다 벡터를 두 개 뒀요. 자기 벡터와 맥락 벡터예요.

ENG) The base node is C with $u(C) = -0.5$. The other side uses the context vector u' .

KOR) 기준 노드는 C 이고 $u(C) = -0.5$ 다. 상대는 맥락 벡터 u' 를 쓴다.

ENG) A: $u'(A) u(C) = -0.5 \times -0.5 = 0.25$, $\exp = 1.28$

KOR) A: $u'(A) \times u(C) = -0.5 \times -0.5 = 0.25$, $\exp = 1.28$

ENG) B: $u'(B) u(C) = 1 \times -0.5 = -0.5$, $\exp = 0.61$

KOR) B: $u'(B) \times u(C) = 1 \times -0.5 = -0.5$, $\exp = 0.61$

ENG) C: $u'(C) u(C) = 0.5 \times -0.5 = -0.25$, $\exp = 0.78$

KOR) C: $u'(C) \times u(C) = 0.5 \times -0.5 = -0.25$, $\exp = 0.78$

ENG) D: $u'(D) u(C) = -1 \times -0.5 = 0.5$, $\exp = 1.65$

KOR) D: $u'(D) \times u(C) = -1 \times -0.5 = 0.5$, $\exp = 1.65$

ENG) The denominator is the sum of all four: $1.28 + 0.61 + 0.78 + 1.65 = 4.32$

KOR) 분모는 넷을 모두 더한 값이다. $1.28 + 0.61 + 0.78 + 1.65 = 4.32$

ENG) b) $p_2(A|C) = 0.2963$, $p_2(B|C) = 0.1412$, $p_2(C|C) = 0.1806$, $p_2(D|C) = 0.3819$

ENG)

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCN 모형의 층 (k) 에서의 은닉층은 $H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)})$ 로 계산된다고 가정한다. 여기서 $\tilde{A} = D^{-1}A$ 는 정규화된 인접행렬이고 σ 는 ReLU 이다.

- GCN 한 층에서 정보가 어떻게 전파되는지 설명하시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서 GCN 모형의 출력을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ -2 \\ 2 \\ -2 \\ 4 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 한 층에서 노드는 바로 옆 이웃의 특징을 모아 평균을 내요.
 - 행 정규화 행렬 $\tilde{A}$ 데는 D 의 역행렬 곱하기 A 이고, 그것이 평균을 내 줘요.
 - k 개 층을 지나면 노드는 k 걸음 안의 모든 노드 정보를 받아요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=3, d(D)=2, d(E)=3$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=3, d(D)=2, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0.5, 0, 0.5]; [0, 0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.5, 0.5, 0, 0]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0.5, 0, 0.5]; [0, 0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) b) $\text{Atilde} = [0, 0.5, 0.5, 0, 0]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0.5, 0, 0.5]; [0, 0.3333, 0.3333, 0.3333, 0]$

ENG) Multiply Atilde by H0. Each entry is the mean of that node's neighbours.

KOR) Atilde 를 H0 에 곱한다. 노드마다 이웃의 평균이다.

ENG) A: neighbours B, C. $(-2 + 2)/2 = 0$

KOR) A: 이웃 B, C. $(-2 + 2)/2 = 0$

ENG) B: neighbours A, E. $(-1 + 4)/2 = 1.5$

KOR) B: 이웃 A, E. $(-1 + 4)/2 = 1.5$

ENG) C: neighbours A, D, E. $(-1 + -2 + 4)/3 = 0.3333$

KOR) C: 이웃 A, D, E. $(-1 + -2 + 4)/3 = 0.3333$

ENG) D: neighbours C, E. $(2 + 4)/2 = 3$

KOR) D: 이웃 C, E. $(2 + 4)/2 = 3$

ENG) E: neighbours B, C, D. $(-2 + 2 + -2)/3 = -0.6667$

KOR) E: 이웃 B, C, D. $(-2 + 2 + -2)/3 = -0.6667$

ENG) ReLU clips the negatives: $-0.6667 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.6667 \rightarrow 0$

ENG) c) $H1 = [0, 1.5, 0.3333, 3, 0]$

ENG)

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 각 노드 옆의 수가 초기 특징이다.

집계 함수가 평균인 GraphSAGE 모형에서, 노드 i 의 층 k 특징은 이웃의 평균을 자신의 특징에 이어 붙인 뒤 ReLU를 적용하여 갱신된다.

- GraphSAGE의 핵심 착상은 무엇이며 기존 GCN과 어떻게 다른지 서술하시오.
- $k = 1$ 에서 각 노드의 특징을 계산하시오.
- $k = 1$ 일 때 평균 전역 풀링으로 그래프 수준 임베딩 h_G 를 계산하시오.

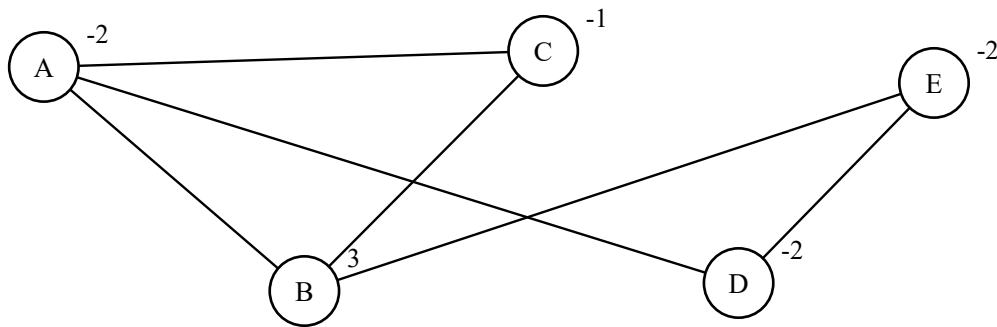


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE는 노드마다 고정된 벡터를 배우는 대신 모으는 함수를 배워요.
 - 이웃을 정해진 개수만 뽑아요. 그래서 아주 큰 그래프에서도 미니배치로 학습할 수 있어요.
 - GCN은 전이적이고 GraphSAGE는 귀납적이에요. 그래서 학습 때 못 본 노드도 벡터를 만들 수 있어요.

ENG) First compute the neighbour mean. The node itself is not included.

KOR) 먼저 이웃 평균을 구한다. 자기 자신은 넣지 않는다.

ENG) A: neighbours B, C, D. $(3 + -1 + -2)/3 = 0$

KOR) A: 이웃 B, C, D. $(3 + -1 + -2)/3 = 0$

ENG) B: neighbours A, C, E. $(-2 + -1 + -2)/3 = -1.6667$

KOR) B: 이웃 A, C, E. $(-2 + -1 + -2)/3 = -1.6667$

ENG) C: neighbours A, B. $(-2 + 3)/2 = 0.5$

KOR) C: 이웃 A, B. $(-2 + 3)/2 = 0.5$

ENG) D: neighbours A, E. $(-2 + -2)/2 = -2$

KOR) D: 이웃 A, E. $(-2 + -2)/2 = -2$

ENG) E: neighbours B, D. $(3 + -2)/2 = 0.5$

KOR) E: 이웃 B, D. $(3 + -2)/2 = 0.5$

ENG) Concatenate the node's own value with the neighbour mean. This is not an addition.

KOR) 자기 값을 앞에, 이웃 평균을 뒤에 이어 붙인다. 더하는 것이 아니다.

ENG) ReLU clips the negatives: $-2 \rightarrow 0$, $-1.6667 \rightarrow 0$, $-1 \rightarrow 0$, $-2 \rightarrow 0$, $-2 \rightarrow 0$, $-2 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-2 \rightarrow 0$, $-1.6667 \rightarrow 0$, $-1 \rightarrow 0$, $-2 \rightarrow 0$, $-2 \rightarrow 0$, $-2 \rightarrow 0$

ENG) The graph embedding is the mean per position: first $-4 / 5$, second $-2.6667 / 5$

KOR) 그래프 임베딩은 자리별 평균이다. 첫 자리 합 $-4 / 5$, 둘째 자리 합 $-2.6667 / 5$

ENG) b) $h(A) = [0, 0]$, $h(B) = [3, 0]$, $h(C) = [0, 0.5]$, $h(D) = [0, 0]$, $h(E) = [0, 0.5]$

ENG) c) $hG = [0.6, 0.2]$

ENG)

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 는 두 개의 군집 G_1 과 G_2 를 포함하며, 각 노드 옆의 수가 초기 특징이다.

ClusterGCN 모형에서는 이웃 중 자신과 같은 군집에 속한 노드만 평균에 포함한다.

- 미니배치 학습이란 무엇이며 ClusterGCN 이 이를 필요로 하는 이유를 서술하시오.
- 층 $k = 1$ 에서 모든 노드의 출력 표현을 계산하시오.

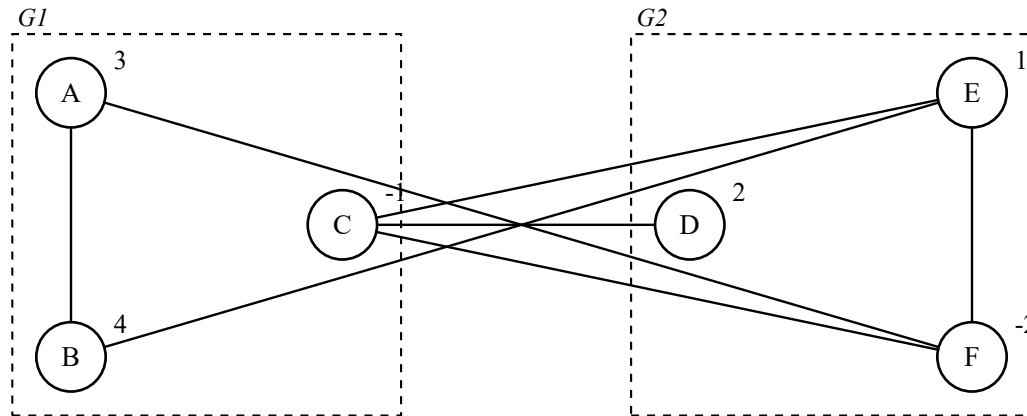


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 미니배치 학습은 한 번에 자료의 일부만 써서 파라미터를 고쳐요.
 - 큰 그래프는 메모리에 안 들어가요. 그래서 ClusterGCN 은 안쪽이 촘촘한 덩어리로 나뉘요.
 - 잘 나누면 선 대부분이 배치 안에 남아서 경계에서 잃는 정보가 적어요.

ENG) Edges that cross a group are treated as absent. Cross them out first.

KOR) 묶음을 넘는 간선은 없는 것으로 본다. 먼저 그 간선에 사선을 그어 지운다.

ENG) A: neighbours B. $(4)/1 = 4$

KOR) A: 이웃 B. $(4)/1 = 4$

ENG) B: neighbours A. $(3)/1 = 3$

KOR) B: 이웃 A. $(3)/1 = 3$

ENG) C: no neighbour inside the group, so the mean is 0.

KOR) C: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) D: no neighbour inside the group, so the mean is 0.

KOR) D: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) E: neighbours F. $(-2)/1 = -2$

KOR) E: 이웃 F. $(-2)/1 = -2$

ENG) F: neighbours E. $(1)/1 = 1$

KOR) F: 이웃 E. $(1)/1 = 1$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU to both entries.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다. ReLU 는 두 자리 모두에 걸린다.

ENG) b) $h(A) = [3, 4]$, $h(B) = [4, 3]$, $h(C) = [0, 0]$, $h(D) = [2, 0]$, $h(E) = [1, 0]$, $h(F) = [0, 1]$

ENG)

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 에는 추출된 부분그래프 G_1 이 주어져 있으며, 각 노드 옆의 수가 초기 특징이다.

GraphSAINT 모형에서는 이웃 중 같은 부분그래프에 속한 노드만 평균에 포함한다.

- GraphSAINT의 추출 전략이 노드 단위 또는 층 단위 추출 방식과 어떻게 다른지 서술하시오.
- 층 $k = 1$ 에서 노드 A, B, C, D의 출력 표현을 계산하시오.

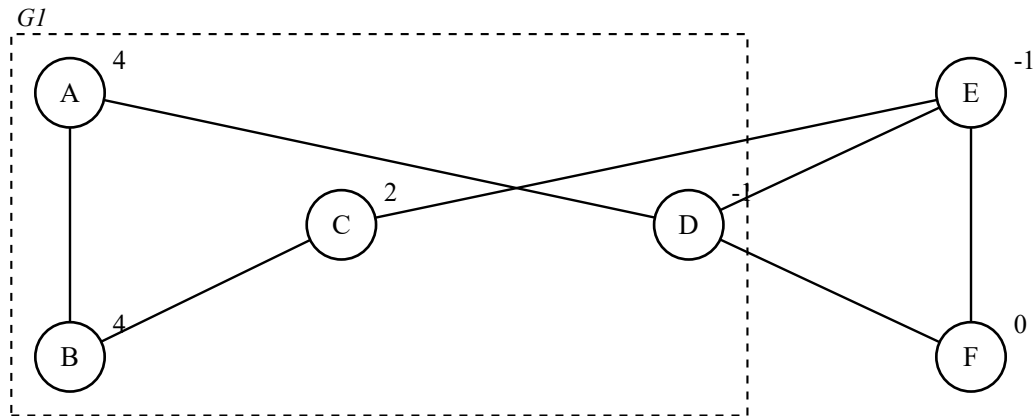


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE와 FastGCN은 층을 쌓으면서 뽑아요. 그래서 층마다 뽑기를 되풀이해요.
 - GraphSAINT는 부분 그래프를 먼저 뽑고 그 위에 GNN을 통째로 올려요.
 - 그래서 깊어져도 비용이 안 늘고, 정규화 계수로 뽑기의 치우침을 바로잡아요.

ENG) Edges that leave the sampled subgraph are discarded.

KOR) 뽑힌 부분그래프 밖으로 나가는 간선은 버린다.

ENG) A: neighbours B, D. $(4 + -1)/2 = 1.5$

KOR) A: 이웃 B, D. $(4 + -1)/2 = 1.5$

ENG) B: neighbours A, C. $(4 + 2)/2 = 3$

KOR) B: 이웃 A, C. $(4 + 2)/2 = 3$

ENG) C: neighbours B. $(4)/1 = 4$

KOR) C: 이웃 B. $(4)/1 = 4$

ENG) D: neighbours A. $(4)/1 = 4$

KOR) D: 이웃 A. $(4)/1 = 4$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다.

ENG) b) $h(A) = [4, 1.5]$, $h(B) = [4, 3]$, $h(C) = [2, 4]$, $h(D) = [0, 4]$

ENG)

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

JK 신경망의 층 (k) 출력은 $\sigma(\tilde{A} H^{(0)})$ 부터 $\sigma(\tilde{A} H^{(k-1)})$ 까지의 원소별 최댓값으로 계산된다고 가정한다.

- Jumping Knowledge 신경망이 깊은 GCN 의 과평활 문제를 어떻게 완화하는지 설명하시오.
- $\tilde{A}$ 와 $H^{(1)}$ 을 계산하시오.
- 층 $k = 2$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 0 \\ -3 \\ -1 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 에서는 노드가 이웃 평균을 거듭 내서 표현들이 다 비슷해져요. 이것이 과평활이에요.
 - JKNet 은 모든 층의 표현을 들고 있다가 끝에서 최댓값, 이어 붙이기, LSTM 으로 합쳐요.
 - 알맞은 깊이는 노드마다 달라요. 그래서 노드마다 자기에게 맞는 깊이를 고를 수 있어요.

ENG) Degrees: $d(A)=3, d(B)=2, d(C)=1, d(D)=3, d(E)=3$

KOR) 차수를 센다. $d(A)=3, d(B)=2, d(C)=1, d(D)=3, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]$

ENG) First build $H1 = \text{ReLU}(\tilde{A}H0)$

KOR) 먼저 $H1$ 을 만든다. $H1 = \text{ReLU}(\tilde{A}H0)$

ENG) A: neighbours B, D, E. $(0 + -1 + -1)/3 = -0.6667$

KOR) A: 이웃 B, D, E. $(0 + -1 + -1)/3 = -0.6667$

ENG) B: neighbours A, D. $(3 + -1)/2 = 1$

KOR) B: 이웃 A, D. $(3 + -1)/2 = 1$

ENG) C: neighbours E. $(-1)/1 = -1$

KOR) C: 이웃 E. $(-1)/1 = -1$

ENG) D: neighbours A, B, E. $(3 + 0 + -1)/3 = 0.6667$

KOR) D: 이웃 A, B, E. $(3 + 0 + -1)/3 = 0.6667$

ENG) E: neighbours A, C, D. $(3 + -3 + -1)/3 = -0.3333$

KOR) E: 이웃 A, C, D. $(3 + -3 + -1)/3 = -0.3333$

ENG) ReLU clips the negatives: $-0.6667 \rightarrow 0, -1 \rightarrow 0, -0.3333 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.6667 \rightarrow 0, -1 \rightarrow 0, -0.3333 \rightarrow 0$

ENG) $H1 = [0, 1, 0, 0.6667, 0]$

ENG) For $k = 2$ there are two terms. First: $\text{ReLU}(\tilde{A}H0) = [0, 1, 0, 0.6667, 0]$

KOR) $k=2$ 이므로 조각이 두 개다. 첫째 $\text{ReLU}(\tilde{A}H0) = [0, 1, 0, 0.6667, 0]$

ENG) Second: $\text{ReLU}(\tilde{A}H1) = [0.5556, 0.3333, 0, 0.3333, 0.2222]$

KOR) 둘째 $\text{ReLU}(\tilde{A}H1) = [0.5556, 0.3333, 0, 0.3333, 0.2222]$

ENG) The max is taken element-wise, not by choosing one whole vector.

KOR) max 는 가로줄마다 큰 쪽을 고른다. 벡터 하나를 고르는 것이 아니다.

ENG) c) $H2 = [0.5556, 1, 0, 0.6667, 0.2222]$

ENG)

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCNII 모형의 층 (k) 출력은 $H^{(k)} = \sigma[((1-\beta)I_n) \cdot ((1-\alpha)\tilde{A} H^{(k-1)} + \alpha H^{(0)})]$ 로 계산된다고 가정한다. I_n 은 단위행렬이며 α 와 β 는 문제에 주어진 값을 사용한다.

- GCNII 가 GCN 의 깊이 한계를 극복하기 위해 도입한 두 가지 주요 구성 요소를 쓰시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -3 \\ -1 \\ 1 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 두 장치는 초기 잔차 연결과 항등 사상이에요.
 - 초기 잔차 연결은 층마다 입력 $H^{(0)}$ 의 일부를 알파만큼 다시 더해요.
 - 항등 사상은 가중치 행렬을 단위행렬에 가깝게 뒹요. 그래서 깊어져도 신호가 안 망가져요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=3$, $d(E)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=3$, $d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0, 0.5, 0.5]; [0, 0, 0.5, 0, 0.5]; [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0, 0.5, 0.5]; [0, 0, 0.5, 0, 0.5]; [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0, 0, 0.5, 0.5]; [0, 0, 0.5, 0, 0.5]; [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$

ENG) Start with the inner bracket. $\tilde{H}_0 = [0, -1, -1, -0.6667, -0.6667]$

KOR) 안쪽 괄호부터 푼다. $\tilde{H}_0 = [0, -1, -1, -0.6667, -0.6667]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[0, -0.8, -0.8, -0.5333, -0.5333]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[0, -0.8, -0.8, -0.5333, -0.5333]$

ENG) Add $\alpha \tilde{H}_0 = 0.2 \times [0, -3, -1, 1, -1] = [0, -0.6, -0.2, 0.2, -0.2]$

KOR) $\alpha \tilde{H}_0 = 0.2 \times [0, -3, -1, 1, -1] = [0, -0.6, -0.2, 0.2, -0.2]$ 를 더한다.

ENG) Inner bracket = $[0, -1.4, -1, -0.3333, -0.7333]$

KOR) 안쪽 괄호 = $[0, -1.4, -1, -0.3333, -0.7333]$

ENG) Multiply by the outer $(1-\beta) = 0.6$. Dropping this factor scales the answer.

KOR) 바깥의 $(1-\beta) = 0.6$ 를 곱한다. 이 계수를 빼뜨리면 답이 배로 커진다.

ENG) ReLU clips the negatives: $-0.84 \rightarrow 0, -0.6 \rightarrow 0, -0.2 \rightarrow 0, -0.44 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.84 \rightarrow 0, -0.6 \rightarrow 0, -0.2 \rightarrow 0, -0.44 \rightarrow 0$

ENG) c) $H_1 = [0, 0, 0, 0, 0]$

ENG)

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

잔차 연결을 갖는 DeepGCN 의 층 (k) 출력은 $H^{(k)} = \sigma(\tilde{A} H^{(k-1)}) + H^{(k-1)}$ 로 계산된다고 가정한다. 덧셈은 ReLU 적용 이후에 이루어진다.

- 잔차 연결이 GCN 을 더 깊게 쌓을 수 있게 하는 이유를 설명하시오.
- 층 $k = 1$ 과 $k = 2$ 에서의 출력 표현을 계산하시오.
- 잔차 연결이 없는 보통의 GCN 출력과 $H^{(1)}$ 을 비교하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 0 \\ 3 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 은 기울기 소실과 과평활을 겪어요. 그래서 층을 더하면 오히려 나빠져요.
 - 잔차 연결은 입력을 그대로 앞으로 보내요. 그래서 층은 바뀌는 양만 배우면 돼요.
 - 바꿀 게 없으면 층은 0 을 내보내면 돼요. 기울기도 그 지름길로 되돌아 흘러요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) $\tilde{H}_0 = [0.5, 0.6667, 0.5, 0.3333]$

ENG) Apply ReLU first: $[0.5, 0.6667, 0.5, 0.3333]$

KOR) ReLU 를 **먼저** 건다. $[0.5, 0.6667, 0.5, 0.3333]$

ENG) Then add $\tilde{H}_0$. The addition sits outside the ReLU, so negatives can survive.

KOR) 그 다음에 $\tilde{H}_0$ 을 더한다. 더하기는 ReLU 바깥이라 음수가 살아남을 수 있다.

ENG) For $k = 2$, feed H_1 back in and add H_1 , not $\tilde{H}_0$.

KOR) $k=2$ 에서는 H_1 을 다시 넣고, 더하는 것도 $\tilde{H}_0$ 이 아니라 H_1 이다.

ENG) $\tilde{H}_1 = [1, 1.1111, 1, 0.8889]$, after ReLU $[1, 1.1111, 1, 0.8889]$

KOR) $\tilde{H}_1 = [1, 1.1111, 1, 0.8889]$, ReLU 뒤 $[1, 1.1111, 1, 0.8889]$

ENG) b) $H_1 = [-1.5, 0.6667, 3.5, 1.3333]$

ENG) $H_2 = [-0.5, 1.7778, 4.5, 2.2222]$

ENG) c) Plain GCN without residual: $H_1 = [0.5, 0.6667, 0.5, 0.3333]$ (the node's own value and negatives disappear)

KOR) c) 잔차 없는 GCN 의 $H_1 = [0.5, 0.6667, 0.5, 0.3333]$ (잔차가 없으면 자기 값과 음수가 사라져요)

3회차

ENG)

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 노드 1과 5에는 이름표가 주어져 있으며($P(Y_1)=1, P(Y_5)=0$), 나머지 노드는 이름표가 없고 0.5로 초기화한다.

관계형 분류기는 이름표가 없는 각 노드를 이웃 노드들의 부류 확률에 대한 가중 평균으로 갱신한다. 분모는 해당 노드에 연결된 간선 가중치의 합이다.

- 관계형 분류가 전제하는 핵심 가정과 그 한계 하나를 서술하시오.
- 1차 반복 후 이름표가 없는 각 노드의 $P(Y_v)$ 를 계산하시오.
- 2차 반복 후의 $P(Y_v)$ 를 계산하시오. 단, 직전 반복의 값만 사용한다.

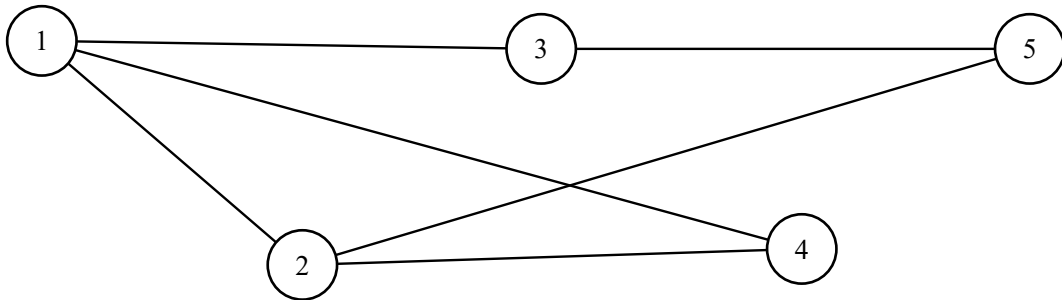


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 관계형 분류는 동질성을 가정해요. 이어진 노드끼리는 같은 이름표를 가지는 경향이 있다는 뜻이에요.
 - 이름표 없는 노드를 이웃들의 확률의 가중 평균으로 거듭 바뀌요.
 - 한계는 그래프 구조만 쓰고 노드의 특징은 무시한다는 것이에요.

ENG) Write the initial values: node 1 is 1, node 5 is 0, the rest start at 0.5.

KOR) 처음 값을 적는다. 이름표가 붙은 1번은 1, 5번은 0, 나머지는 0.5로 시작한다.

ENG) First iteration. Only the unlabelled nodes are replaced by the mean of their neighbours.

KOR) 1차 반복. 이름표가 없는 노드만 이웃의 평균으로 바꾼다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 4, 5. $(1 + 0.5 + 0)/3 = 0.5$

KOR) 2번: 이웃 1, 4, 5. $(1 + 0.5 + 0)/3 = 0.5$

ENG) Node 3: neighbours 1, 5. $(1 + 0)/2 = 0.5$

KOR) 3번: 이웃 1, 5. $(1 + 0)/2 = 0.5$

ENG) Node 4: neighbours 1, 2. $(1 + 0.5)/2 = 0.75$

KOR) 4번: 이웃 1, 2. $(1 + 0.5)/2 = 0.75$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) b) $P(Y1) = 1, P(Y2) = 0.5, P(Y3) = 0.5, P(Y4) = 0.75, P(Y5) = 0$

ENG) Second iteration. Use only the values from the first iteration, not the ones just computed.

KOR) 2차 반복. 방금 구한 값이 아니라 1차 반복의 값만 쓴다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 4, 5. $(1 + 0.75 + 0)/3 = 0.5833$

KOR) 2번: 이웃 1, 4, 5. $(1 + 0.75 + 0)/3 = 0.5833$

ENG) Node 3: neighbours 1, 5. $(1 + 0)/2 = 0.5$

KOR) 3번: 이웃 1, 5. $(1 + 0)/2 = 0.5$

ENG) Node 4: neighbours 1, 2. $(1 + 0.5)/2 = 0.75$

KOR) 4번: 이웃 1, 2. $(1 + 0.5)/2 = 0.75$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) c) $P(Y1) = 1, P(Y2) = 0.5833, P(Y3) = 0.5, P(Y4) = 0.75, P(Y5) = 0$

ENG)

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{B, C\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

KOR)

인접행렬 A 와 노드 특징 벡터 $h^{(0)}$ 가 주어진 무방향 그래프 G 를 고려한다.

FastGCN 은 GCN 한 층의 전체 이웃 합을 층 단위 몬테카를로 추정으로 대체한다. 각 층에서 t_l 개의 노드를 중요도 분포 $q(u) \propto \|A(:, u)\|^2$ 로부터 한 번 추출하며, 평균은 이웃 수가 아니라 추출 개수 t_l 로 나눈다.

- 층 단위 추출이 GraphSAGE 의 노드 단위 추출과 어떻게 다른지 서술하시오.
- 다섯 노드에 대한 중요도 분포 $q(u)$ 를 계산하시오.
- 주어진 추출 집합($t_l = 2$)에 대하여 $h^{(1)}(A)$ 의 추정값을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 1 \\ 0 \\ -2 \\ -1 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 뽑기는 노드마다 제 이웃을 뽑아서, 층이 깊어질수록 비용이 지수로 커져요.
 - FastGCN 은 층마다 정해진 개수를 한 번만 뽑고, 그 층의 모든 노드가 같이 써요.
 - 그래서 비용이 지수가 아니라 층 수에 비례해서만 늘어요.

ENG) Degrees: $d(A)=1, d(B)=3, d(C)=2, d(D)=3, d(E)=3$

KOR) 차수를 센다. $d(A)=1, d(B)=3, d(C)=2, d(D)=3, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 1, 0, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0, 0.5, 0.5]; [0.3333, 0.3333, 0, 0.3333, 0]; [0, 0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 1, 0, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0, 0.5, 0.5]; [0, 0.3333, 0.3333, 0, 0.3333]; [0, 0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) $q(u)$ uses the squared norm of column u of $\tilde{A}$, not of a row.

KOR) $q(u)$ 는 $\tilde{A}$ 의 **세로줄** 제곱합이다. 가로줄이 아니다.

ENG) Column A: squared sum = 0.1111

KOR) A 칸의 제곱합 = 0.1111

ENG) Column B: squared sum = 1.2222

KOR) B 칸의 제곱합 = 1.2222

ENG) Column C: squared sum = 0.2222

KOR) C 칸의 제곱합 = 0.2222

ENG) Column D: squared sum = 0.4722

KOR) D 칸의 제곱합 = 0.4722

ENG) Column E: squared sum = 0.4722

KOR) E 칸의 제곱합 = 0.4722

ENG) Divide by the total 2.5 to turn them into probabilities.

KOR) 전체 합 2.5 로 나눠 확률로 만든다.

ENG) b) $q(A) = 0.0444, q(B) = 0.4889, q(C) = 0.0889, q(D) = 0.1889, q(E) = 0.1889$

ENG) For (c) look only at the sampled nodes: read $\tilde{A}(A, u)$ for each.

KOR) c) 는 뽑힌 노드만 본다. A 가로줄에서 그 칸의 $\tilde{A}$ 값을 읽는다.

ENG) $\tilde{A}(A, B) h(B) = 1 \times 0 = 0$

KOR) $\tilde{A}(A, B) \times h(B) = 1 \times 0 = 0$

ENG) $\tilde{A}(A, C) h(C) = 0 \times -2 = 0$

KOR) $\tilde{A}(A, C) \times h(C) = 0 \times -2 = 0$

ENG) Add them and divide by the number sampled (2), not by the degree: $(0 + 0)/2 = 0$

KOR) 더한 뒤 이웃 수가 아니라 **뽑은 개수 2** 로 나눈다. $(0 + 0)/2 = 0$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $h_1(A) = 0$

ENG)

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = C$.
- Calculate $E_{p(u_j | C)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(C)$.

KOR)

문제 11 과 동일한 그래프와 특징 벡터를 고려한다.

ASGCN 은 GCN 한 층을 확률 분포에 대한 기댓값으로 다시 쓴다. 여기서 $N(v_i) = \sum_j \tilde{A}_{ij}$ 는 정규화 계수이고, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$ 는 부모 노드 v_i 가 주어졌을 때 노드 u_j 를 추출할 확률이다.

- ASGCN 의 제안 분포가 FastGCN 과 달리 무엇에 의존하는지 서술하시오.
- 주어진 부모 노드에 대한 $N(v_i)$ 와 $p(u_j | v_i)$ 를 계산하시오.
- 해당 기댓값과 그 결과로 얻어지는 $h^{(1)}$ 을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -1 \\ -1 \\ 4 \\ 1 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - FastGCN 의 뽑기 확률은 미리 정해져 있고 부모 노드와 상관이 없어요.
 - ASGCN 은 뽑기 확률을 윗층 노드에 맞춰 정해요.
 - 그래서 뽑은 노드가 거의 다 쓸모 있고, 추정값의 흔들림이 줄어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=2$, $d(E)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=2$, $d(E)=3$

ENG) Divide each row by its degree. Atilde = $[0, 0, 0.5, 0, 0.5]$; $[0, 0, 0, 0.5, 0.5]$; $[0.3333, 0, 0, 0.3333, 0.3333]$; $[0, 0.5, 0.5, 0, 0]$; $[0.3333, 0.3333, 0.3333, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0, 0.5]; [0, 0, 0, 0.5, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$ (가로줄 합이 모두 1)

ENG) The parent node is C, so only its row of $\tilde{A}$ matters.

KOR) 부모 노드는 C 다. $\tilde{A}$ 의 그 줄만 본다.

ENG) $N(C) = \text{sum of that row} = 1$. With row normalisation it is 1.

KOR) $N(C) = \text{그 가로줄의 합} = 1$. 행 정규화라 1 이다.

ENG) $p(A|C) = \tilde{A}(C,A) / N = 0.3333$

KOR) $p(A|C) = \tilde{A}(C,A) / N = 0.3333 / 1 = 0.3333$

ENG) $p(D|C) = \tilde{A}(C,D) / N = 0.3333$

KOR) $p(D|C) = \tilde{A}(C,D) / N = 0.3333 / 1 = 0.3333$

ENG) $p(E|C) = \tilde{A}(C,E) / N = 0.3333$

KOR) $p(E|C) = \tilde{A}(C,E) / N = 0.3333 / 1 = 0.3333$

ENG) b) $N(C) = 1$, $p(A|C) = 0.3333$, $p(D|C) = 0.3333$, $p(E|C) = 0.3333$

ENG) The expectation is the sum of probability times value: $0.3333 \times -1 + 0.3333 \times 1 + 0.3333 \times -2 = -0.6667$

KOR) 기댓값은 확률 곱하기 값의 합이다. $0.3333 \times -1 + 0.3333 \times 1 + 0.3333 \times -2 = -0.6667$

ENG) $N = 1$ and $W = 1$, so the value is unchanged. It equals the plain neighbour mean.

KOR) $N = 1$, $W = 1$ 이라 곱해도 그대로다. 결국 이웃 평균과 같은 값이다.

ENG) ReLU clips the negatives: $-0.6667 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.6667 \rightarrow 0$

ENG) c) $E = -0.6667$, $h1(C) = 0$

ENG)

4. (APPP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

APPP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.5$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

KOR)

인접행렬 A 와 다층 퍼셉트론이 산출한 예측 행렬 H 가 주어졌다.

APPP 는 순간이동 확률 α 를 사용하여 예측을 전파한다. 전파 단계 사이에는 비선형 함수를 적용하지 않는다.

- 개인화 PageRank 의 순간이동 기법을 설명하고, 그것이 과평활을 막는 이유를 서술하시오.
- $\tilde{A}$ 와 $Z^{(1)}$ 을 계산하시오.
- $Z^{(2)}$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 퍼뜨리는 걸음마다 확률 알파로 출발 노드로 되돌아가요. 이것이 순간이동 장치예요.
 - 매 걸음 일정 비율이 되돌아오니까 분포가 출발점 근처에 머물러요.
 - 그래서 많이 퍼뜨려도 과평활이 안 생기고, 퍼뜨리는 데에 추가 파라미터도 없어요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$ (가로 줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$

ENG) Start from $Z_0 = H = [0, 0, 0, 1]$

KOR) $Z_0 = H$ 로 시작한다. $[0, 0, 0, 1]$

ENG) Propagate one step: $\tilde{Z}_0 = [0.5, 0.5, 0, 0]$

KOR) 한 걸음 퍼뜨린다. $\tilde{Z}_0 = [0.5, 0.5, 0, 0]$

ENG) Multiply by $(1-\alpha) = 0.5$: $[0.25, 0.25, 0, 0]$

KOR) $(1-\alpha) = 0.5$ 를 곱한다. $[0.25, 0.25, 0, 0]$

ENG) Add $\alpha H = [0, 0, 0, 0.5]$. This is the teleport term.

KOR) $\alpha H = 0.5 \times [0, 0, 0, 1] = [0, 0, 0, 0.5]$ 를 더한다. 이것이 순간이동이다.

ENG) $Z_1 = [0.25, 0.25, 0, 0.5]$

ENG) One more step: $\tilde{Z}_1 = [0.25, 0.25, 0.25, 0.25]$

KOR) 한 걸음 더. $\tilde{Z}_1 = [0.25, 0.25, 0.25, 0.25]$

ENG) Multiply by $(1-\alpha)$ and add αH **again**. It is added at every step.

KOR) $(1-\alpha)$ 를 곱하고 **αH 를 또** 더한다. 매 걸음 더하는 것이 핵심이다.

ENG) c) $Z_2 = [0.125, 0.125, 0.125, 0.625]$

ENG)

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.5$, $\gamma_1 = 0.2$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

KOR)

인접행렬 A 와 신경망이 산출한 은닉 특징 $H^{(0)}$ 이 주어졌다.

GPR-GNN 은 특징 추출과 위상 전파를 분리한다. $\tilde{A}$ 를 반복 적용하여 $H^{(k)}$ 를 얻고, 학습된 가중치 γ_k 로 가중합한다. 본 문항에서는 $K = 2$ 이며 $\gamma_0, \gamma_1, \gamma_2$ 가 주어진다.

- GPR-GNN 이 APPNP 와 달리 할 수 있는 것은 무엇이며, γ_k 가 음수가 될 수 있다는 점이 이질적 그래프에서 왜 중요한지 서술하시오.
- $H^{(1)}$ 과 $H^{(2)}$ 를 계산하시오.
- Z 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ -1 \\ 4 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - APPNP 는 계수를 알파 곱하기 (1-알파)의 k제곱으로 못 박아요. 늘 양수이고 줄어들어요.
 - GPR-GNN 은 그 무게를 학습하고, 음수도 될 수 있어요.
 - 음수 무게는 그 걸음을 빼요. 그래서 이질 그래프에서 고주파 필터처럼 동작할 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.5, 0, 0.5, 0]; [0, 0.5, 0, 0.5]; [0.5, 0, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0, 0.5]; [0.5, 0, 0.5, 0]; [0, 0.5, 0, 0.5]; [0.5, 0, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) H_0 stays as it is: $[4, -2, -1, 4]$

KOR) H_0 은 그대로 둔다. $[4, -2, -1, 4]$

ENG) $H_1 = \text{Atilde } H_0 = [1, 1.5, 1, 1.5]$

ENG) $H_2 = \text{Atilde } H_1 = [1.5, 1, 1.5, 1]$

ENG) b) $H_1 = [1, 1.5, 1, 1.5]$, $H_2 = [1.5, 1, 1.5, 1]$

ENG) Now take the weighted sum with gamma. There is no ReLU during propagation.

KOR) 이제 gamma 로 가중합한다. 전파 중에 ReLU 는 없다.

ENG) $\text{gamma}_0 H_0 = 0.5 \times [4, -2, -1, 4] = [2, -1, -0.5, 2]$

KOR) $\text{gamma}_0 \times H_0 = 0.5 \times [4, -2, -1, 4] = [2, -1, -0.5, 2]$

ENG) $\text{gamma}_1 H_1 = 0.2 \times [1, 1.5, 1, 1.5] = [0.2, 0.3, 0.2, 0.3]$

KOR) $\text{gamma}_1 \times H_1 = 0.2 \times [1, 1.5, 1, 1.5] = [0.2, 0.3, 0.2, 0.3]$

ENG) $\text{gamma}_2 H_2 = -0.2 \times [1.5, 1, 1.5, 1] = [-0.3, -0.2, -0.3, -0.2]$

KOR) $\text{gamma}_2 \times H_2 = -0.2 \times [1.5, 1, 1.5, 1] = [-0.3, -0.2, -0.3, -0.2]$

ENG) gamma_2 is negative, so that hop is subtracted. Dropping the sign gives a wrong answer.

KOR) gamma_2 가 음수라 그 걸음은 빼는 효과다. 부호를 빼뜨리면 틀린다.

ENG) Add the three vectors element-wise.

KOR) 세 벡터를 자리별로 더한다.

ENG) c) $Z = [1.9, -0.9, -0.6, 2.1]$

ENG)

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

MixHop 한 층은 인접행렬의 여러 거듭제곱 $\hat{A}^j$ ($j \in P = \{0, 1, 2\}$)를 각각 계산한 뒤 열 방향으로 이어 붙인다. 본 문항에서 가중치 행렬은 모두 1로 둔다.

- 두 걸음 델타 연산자에 관한 MixHop의 정리 1을 서술하고, 보통의 GCN이 층을 무한히 쌓아도 이를 표현할 수 없는 이유를 설명하시오.
- $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$, $\hat{A}^2 H^{(0)}$ 을 계산하시오.
- $H^{(1)}$ 과 델타 연산자 $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ -2 \\ 1 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 정리 1은 보통 GCN이 두 걸음 델타 연산자를 표현할 수 없다고 말해요. 층을 무한히 쌓아도요.
 - GCN 층을 쌓으면 A 에 1제곱 곱하기 W 꼴로 뭉쳐져서, 중간 걸음들이 따로 남지 않아요.
 - MixHop은 거듭제곱마다 따로 구해서 나란히 붙여요. 그래서 뒤 층이 하나에서 다른 하나를 뺄 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=1$, $d(C)=2$, $d(D)=3$

KOR) 차수를 센다. $d(A)=2$, $d(B)=1$, $d(C)=2$, $d(D)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0, 1]; [0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0, 1]; [0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) A^0 is the identity, so it is just $H_0: [2, -2, 1, -1]$

KOR) A_0 은 단위행렬이라 H_0 그대로다. $[2, -2, 1, -1]$

ENG) $A^1 H_0 = [0, -1, 0.5, 0.3333]$ (one hop, the neighbour mean)

KOR) $A_1 H_0 = [0, -1, 0.5, 0.3333]$ (한 걸음, 이웃 평균)

ENG) $A^2 H_0 = \tilde{A}(\tilde{A} H_0) = [0.4167, 0.3333, 0.1667, -0.1667]$ (two hops)

KOR) $A_2 H_0 = \tilde{A}(\tilde{A} H_0) = [0.4167, 0.3333, 0.1667, -0.1667]$ (두 걸음)

ENG) ReLU is applied to each block separately, not after concatenation.

KOR) 조각마다 따로 ReLU 를 건다. 이어 붙인 뒤가 아니다.

ENG) After ReLU: $[2, 0, 1, 0] \parallel [0, 0, 0.5, 0.3333] \parallel [0.4167, 0.3333, 0.1667, 0]$

KOR) ReLU 뒤: $[2, 0, 1, 0] \parallel [0, 0, 0.5, 0.3333] \parallel [0.4167, 0.3333, 0.1667, 0]$

ENG) The delta operator subtracts the two-hop block from the one-hop block.

KOR) 델타 연산자는 한 걸음에서 두 걸음을 뺀 것이다.

ENG) b) $A_0 H = [2, -2, 1, -1]$, $A_1 H = [0, -1, 0.5, 0.3333]$, $A_2 H = [0.4167, 0.3333, 0.1667, -0.1667]$

ENG) c) $H_1 =$ concatenation of the three blocks: $[2, 0, 1, 0] \parallel [0, 0, 0.5, 0.3333] \parallel [0.4167, 0.3333, 0.1667, 0]$

KOR) c) $H_1 =$ 세 조각을 이어 붙여요: $[2, 0, 1, 0] \parallel [0, 0, 0.5, 0.3333] \parallel [0.4167, 0.3333, 0.1667, 0]$

ENG) $\Delta = [-0.4167, -0.3333, 0.3333, 0.3333]$

ENG)

7. (PairNorm) After one GCN layer the node features of a four-node graph are PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

KOR)

GCN 한 층을 통과한 뒤의 노드 특징 행렬 X 가 주어졌다. 행은 노드, 열은 특징 차원이다.

PairNorm 은 중심화 단계와 크기 조정 단계로 정규화한다. 중심화는 노드들에 대한 평균을 빼는 것이고, 크기 조정은 중심화된 벡터들의 노름 제곱 평균의 제곱근으로 나누는 것이다. $s = 1, n = 4$ 로 한다.

- 노드 단위 과평활과 특징 단위 과평활을 각각 설명하고, PairNorm 이 층에 걸쳐 일정하게 유지하는 양이 무엇인지 서술하시오.
- 중심화된 특징을 계산하시오.
- 크기 조정의 분모와 최종 x_i 를 계산하시오.

$$X = \begin{bmatrix} 4 & 0 \\ 1 & 3 \\ 1 & -1 \\ -1 & 4 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 과평활은 이어진 노드들의 특징이 너무 비슷해지는 것이에요.
 - 특징 단위 과평활은 그래프 전체에서 모든 노드 특징이 구별이 안 되게 되는 것이에요.
 - PairNorm 은 층을 지나도 노드 사이 거리 제곱의 합을 일정하게 지켜요.

ENG) First take the mean over the nodes, separately for each feature column.

KOR) 먼저 노드들에 대한 평균을 구한다. 특징 차원마다 따로 구한다.

ENG) Column 1: $(4 + 1 + 1 + -1)/4 = 1.25$

KOR) 첫 칸: $(4 + 1 + 1 + -1)/4 = 1.25$

ENG) Column 2: $(0 + 3 + -1 + 4)/4 = 1.5$

KOR) 둘째 칸: $(0 + 3 + -1 + 4)/4 = 1.5$

ENG) Subtract the mean from every row. Check: each column now sums to zero.

KOR) 가로줄마다 평균을 뺀다. 검산: 각 칸의 합이 0 이 된다.

ENG) Node 1: squared norm = 9.8125

KOR) 노드 1 의 길이 제곱 = 9.8125

ENG) Node 2: squared norm = 2.3125

KOR) 노드 2 의 길이 제곱 = 2.3125

ENG) Node 3: squared norm = 6.3125

KOR) 노드 3 의 길이 제곱 = 6.3125

ENG) Node 4: squared norm = 11.3125

KOR) 노드 4 의 길이 제곱 = 11.3125

ENG) Mean = $(9.8125 + 2.3125 + 6.3125 + 11.3125)/4 = 7.4375$

KOR) 평균 = $(9.8125 + 2.3125 + 6.3125 + 11.3125)/4 = 7.4375$

ENG) Take the square root: $\sqrt{7.4375} = 2.7272$. Divide every row by this single number.

KOR) 제곱근을 씌운다. $\sqrt{7.4375} = 2.7272$. 이 **하나의 수** 로 모든 줄을 나눈다.

ENG) b) Mean = [1.25, 1.5], centered = [2.75, -1.5]; [-0.25, 1.5]; [-0.25, -2.5]; [-2.25, 2.5]

KOR) b) 평균 = [1.25, 1.5], 중심을 뺀 것 = [2.75, -1.5]; [-0.25, 1.5]; [-0.25, -2.5]; [-2.25, 2.5]

ENG) c) Scaling denominator = 2.7272, $x = [1.0084, -0.55]; [-0.0917, 0.55]; [-0.0917, -0.9167]; [-0.825, 0.9167]$

KOR) c) 분모 = 2.7272, $x = [1.0084, -0.55]; [-0.0917, 0.55]; [-0.0917, -0.9167]; [-0.825, 0.9167]$

ENG)

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (A, C) and (B, D) are dropped. Recalculate $H^{(1)}$.

KOR)

노드 A, B, C, D 로 이루어진 완전그래프 K_4 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

DropEdge 는 매 에폭마다 각 무방향 간선을 확률 $1-p$ 로 독립적으로 유지한다. 이때 $A^{(t)}$ 와 $D^{(t)}$ 는 살아남은 간선만으로 다시 구성한다. $p = 1/3$ 으로 한다.

- DropEdge 가 목표로 하는 두 가지 문제를 쓰고, Dropout 과 어떻게 다른지 서술하시오.
- 살아남은 간선 수의 기댓값과, 간선을 제거하지 않은 원래 그래프에서의 $H^{(1)}$ 을 계산하시오.
- 주어진 두 간선이 제거되었을 때의 $H^{(1)}$ 을 다시 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -3 \\ 4 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - DropEdge 는 과적합과 과평활을 노려요.
 - 판마다 다른 부분 그래프를 써요. 그래프에 자료를 늘리는 효과가 나요.
 - Dropout 은 특징 값을 0 으로 꺼요. DropEdge 는 인접행렬의 선을 지워요.

ENG) K_4 has 6 edges. Each survives with probability $1 - 1/3 = 2/3$, so the expectation is 4.

KOR) K_4 는 간선이 6 개다. 남을 확률이 $1 - 1/3 = 2/3$ 이므로 기댓값은 $6 \times 2/3 = 4$ 개다.

ENG) First the full graph: every node has degree 3.

KOR) 먼저 원래 그래프. 모든 노드의 차수가 3 다.

ENG) A: neighbours B, C, D. $(-3 + 4 + 3)/3 = 1.3333$

KOR) A: 이웃 B, C, D. $(-3 + 4 + 3)/3 = 1.3333$

ENG) B: neighbours A, C, D. $(0 + 4 + 3)/3 = 2.3333$

KOR) B: 이웃 A, C, D. $(0 + 4 + 3)/3 = 2.3333$

ENG) C: neighbours A, B, D. $(0 + -3 + 3)/3 = 0$

KOR) C: 이웃 A, B, D. $(0 + -3 + 3)/3 = 0$

ENG) D: neighbours A, B, C. $(0 + -3 + 4)/3 = 0.3333$

KOR) D: 이웃 A, B, C. $(0 + -3 + 4)/3 = 0.3333$

ENG) Now drop the two given edges and **recount the degrees**. This is where most mistakes happen.

KOR) 이제 주어진 두 간선을 지운다. **차수를 다시 센다**. 여기서 거의 다 틀린다.

ENG) Degrees after dropping: $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

KOR) 지운 뒤 차수: $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

ENG) A: neighbours B, D. $(-3 + 3)/2 = 0$

KOR) A: 이웃 B, D. $(-3 + 3)/2 = 0$

ENG) B: neighbours A, C. $(0 + 4)/2 = 2$

KOR) B: 이웃 A, C. $(0 + 4)/2 = 2$

ENG) C: neighbours B, D. $(-3 + 3)/2 = 0$

KOR) C: 이웃 B, D. $(-3 + 3)/2 = 0$

ENG) D: neighbours A, C. $(0 + 4)/2 = 2$

KOR) D: 이웃 A, C. $(0 + 4)/2 = 2$

ENG) b) Expected number of surviving edges = 4, $H1 = [1.3333, 2.3333, 0, 0.3333]$

KOR) b) 남은 선 수의 기댓값 = 4, $H1 = [1.3333, 2.3333, 0, 0.3333]$

ENG) c) $H1 = [0, 2, 0, 2]$

ENG)

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

KOR)

다음 그림의 무방향 그래프 G 를 고려한다. 임의보행이 노드 A 에서 노드 B 로 이동하여 현재 B 에 있으며, $p = 2, q = 0.5$ 이다.

B 에서 다음 노드 x 로의 비정규화 전이확률은 $\pi_{Bx} = \alpha_{pq}(A, x)$ 이며, α 는 직전 노드 A 로부터의 최단경로 거리에 따라 결정된다.

- BFS 와 DFS 를 설명하고, q 가 작을 때 보행이 둘 중 어느 쪽처럼 동작하는지 서술하시오.
- 노드 B 에서 이동 가능한 모든 노드를 열거하고 비정규화 전이확률 π_{Bx} 를 계산하시오.
- 정규화된 전이확률을 계산하시오.

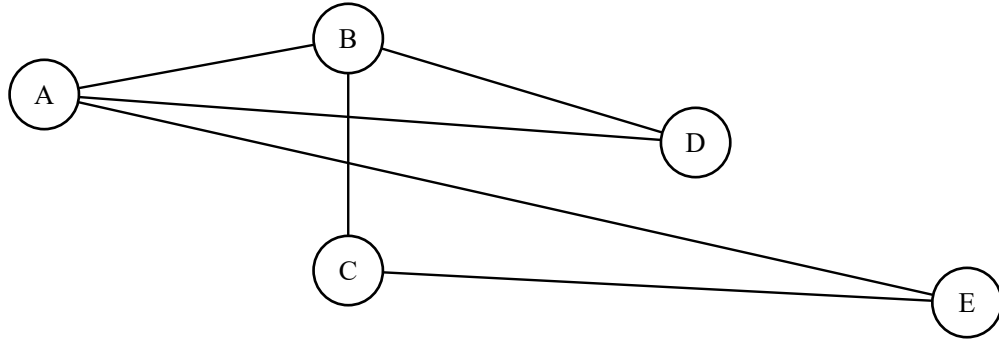


Figure 1. Graph G

풀이

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - BFS 는 바로 옆 이웃부터 훑어요. 그래서 임베딩이 노드의 구조적 역할을 담아요.
 - DFS 는 출발점에서 멀리 가요. 그래서 임베딩이 어느 동네 소속인지를 담아요.
 - q 가 작으면 $1/q$ 가 커져서 걸음이 바깥으로 나가요. DFS 처럼 움직여요.

ENG) The previous node is A and the walk is now at B. All distances are measured from A.

KOR) 직전 노드는 A, 지금 노드는 B 다. 거리는 모두 A 로부터 잴다.

ENG) A: $x = A$ is the previous node, $d = 0$, $\alpha = 1/p = 0.5$

KOR) A: $x = A$ 는 직전 노드라 $d = 0$. $\alpha = 1/p = 0.5$

ENG) C: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) C: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) D: x is also a neighbour of A, $d = 1$, $\alpha = 1 = 1$

KOR) D: A 와도 이웃이라 $d = 1$. $\alpha = 1 = 1$

ENG) Sum them: $0.5 + 2 + 1 = 3.5$

KOR) 세 값을 더한다. $0.5 + 2 + 1 = 3.5$

ENG) b) $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow D) = 1$

ENG) c) $P(A) = 0.1429$, $P(C) = 0.5714$, $P(D) = 0.2857$

ENG)

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = 0.5, u'_C = 1, u'_D = -1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- What is second-order proximity, and how does it differ from first-order proximity?
- Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

KOR)

노드 임베딩 u 와 맥락 임베딩 u' 가 주어진 무방향 그래프 G 를 고려한다.

2차 근접도는 $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$ 로 정의한다. 지수 함수 값은 문제에 주어진 표를 사용한다.

- 2차 근접도란 무엇이며 1차 근접도와 어떻게 다른지 서술하시오.
- 주어진 기준 노드에 대하여 네 노드의 p_2 를 계산하시오.

풀이

ENG)

- Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 1차 근접도는 선으로 직접 이어진 두 노드 사이의 비슷함이에요.
 - 2차 근접도는 이웃을 많이 공유하는 두 노드의 비슷함이에요. 선이 없어도 돼요.
 - 그래서 LINE 은 노드마다 벡터를 두 개 뒹요. 자기 벡터와 맥락 벡터예요.

ENG) The base node is B with $u(B) = 0.5$. The other side uses the context vector u' .

KOR) 기준 노드는 B 이고 $u(B) = 0.5$ 다. 상대는 맥락 벡터 u' 를 쓴다.

ENG) A: $u'(A) u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

KOR) A: $u'(A) \times u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

ENG) B: $u'(B) u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

KOR) B: $u'(B) \times u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

ENG) C: $u'(C) u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

KOR) C: $u'(C) \times u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

ENG) D: $u'(D) u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

KOR) D: $u'(D) \times u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

ENG) The denominator is the sum of all four: $0.78 + 1.28 + 1.65 + 0.61 = 4.32$

KOR) 분모는 넷을 모두 더한 값이다. $0.78 + 1.28 + 1.65 + 0.61 = 4.32$

ENG) b) $p_2(A|B) = 0.1806$, $p_2(B|B) = 0.2963$, $p_2(C|B) = 0.3819$, $p_2(D|B) = 0.1412$

ENG)

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCN 모형의 층 (k) 에서의 은닉층은 $H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)})$ 로 계산된다고 가정한다. 여기서 $\tilde{A} = D^{-1}A$ 는 정규화된 인접행렬이고 σ 는 ReLU 이다.

- GCN 한 층에서 정보가 어떻게 전파되는지 설명하시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서 GCN 모형의 출력을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -2 \\ -1 \\ 4 \\ 4 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 한 층에서 노드는 바로 옆 이웃의 특징을 모아 평균을 내요.
 - 행 정규화 행렬 $\tilde{A}$ 데는 D 의 역행렬 곱하기 A 이고, 그것이 평균을 내 줘요.
 - k 개 층을 지나면 노드는 k 걸음 안의 모든 노드 정보를 받아요.

ENG) Degrees: $d(A)=2, d(B)=3, d(C)=3, d(D)=1, d(E)=3$

KOR) 차수를 센다. $d(A)=2, d(B)=3, d(C)=3, d(D)=1, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0, 0.5]; [0, 0, 0.3333, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 1, 0, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0, 0.5, 0, 0.5]; [0, 0, 0.3333, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 1, 0, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\text{Atilde} = [0, 0, 0.5, 0, 0.5]; [0, 0, 0.3333, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 1, 0, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$

ENG) Multiply Atilde by H0. Each entry is the mean of that node's neighbours.

KOR) Atilde 를 H0 에 곱한다. 노드마다 이웃의 평균이다.

ENG) A: neighbours C, E. $(-1 + 4)/2 = 1.5$

KOR) A: 이웃 C, E. $(-1 + 4)/2 = 1.5$

ENG) B: neighbours C, D, E. $(-1 + 4 + 4)/3 = 2.3333$

KOR) B: 이웃 C, D, E. $(-1 + 4 + 4)/3 = 2.3333$

ENG) C: neighbours A, B, E. $(1 + -2 + 4)/3 = 1$

KOR) C: 이웃 A, B, E. $(1 + -2 + 4)/3 = 1$

ENG) D: neighbours B. $(-2)/1 = -2$

KOR) D: 이웃 B. $(-2)/1 = -2$

ENG) E: neighbours A, B, C. $(1 + -2 + -1)/3 = -0.6667$

KOR) E: 이웃 A, B, C. $(1 + -2 + -1)/3 = -0.6667$

ENG) ReLU clips the negatives: $-2 \rightarrow 0, -0.6667 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-2 \rightarrow 0, -0.6667 \rightarrow 0$

ENG) c) $H1 = [1.5, 2.3333, 1, 0, 0]$

ENG)

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 각 노드 옆의 수가 초기 특징이다.

집계 함수가 평균인 GraphSAGE 모형에서, 노드 i 의 층 k 특징은 이웃의 평균을 자신의 특징에 이어 붙인 뒤 ReLU를 적용하여 갱신된다.

- GraphSAGE의 핵심 착상은 무엇이며 기존 GCN과 어떻게 다른지 서술하시오.
- $k = 1$ 에서 각 노드의 특징을 계산하시오.
- $k = 1$ 일 때 평균 전역 풀링으로 그래프 수준 임베딩 h_G 를 계산하시오.

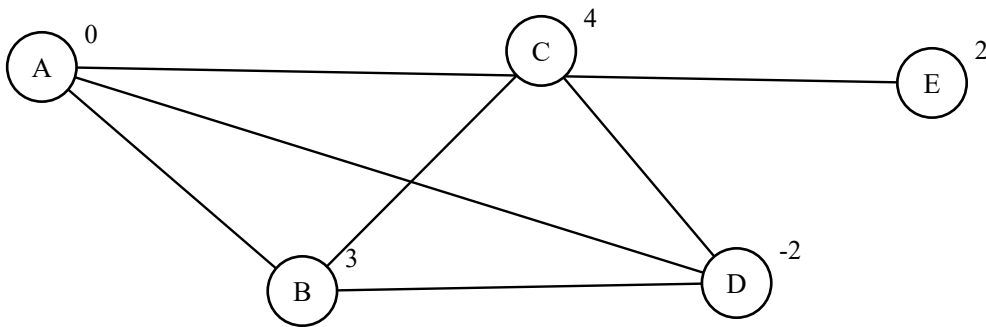


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE는 노드마다 고정된 벡터를 배우는 대신 모으는 함수를 배워요.
 - 이웃을 정해진 개수만 뽑아요. 그래서 아주 큰 그래프에서도 미니배치로 학습할 수 있어요.
 - GCN은 전이적이고 GraphSAGE는 귀납적이에요. 그래서 학습 때 못 본 노드도 벡터를 만들 수 있어요.

ENG) First compute the neighbour mean. The node itself is not included.

KOR) 먼저 이웃 평균을 구한다. 자기 자신은 넣지 않는다.

ENG) A: neighbours B, D, E. $(3 + -2 + 2)/3 = 1$

KOR) A: 이웃 B, D, E. $(3 + -2 + 2)/3 = 1$

ENG) B: neighbours A, C, D. $(0 + 4 + -2)/3 = 0.6667$

KOR) B: 이웃 A, C, D. $(0 + 4 + -2)/3 = 0.6667$

ENG) C: neighbours B, D. $(3 + -2)/2 = 0.5$

KOR) C: 이웃 B, D. $(3 + -2)/2 = 0.5$

ENG) D: neighbours A, B, C. $(0 + 3 + 4)/3 = 2.3333$

KOR) D: 이웃 A, B, C. $(0 + 3 + 4)/3 = 2.3333$

ENG) E: neighbours A. $(0)/1 = 0$

KOR) E: 이웃 A. $(0)/1 = 0$

ENG) Concatenate the node's own value with the neighbour mean. This is not an addition.

KOR) 자기 값을 앞에, 이웃 평균을 뒤에 이어 붙인다. 더하는 것이 아니다.

ENG) ReLU clips the negatives: $-2 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-2 \rightarrow 0$

ENG) The graph embedding is the mean per position: first $7 / 5$, second $4.5 / 5$

KOR) 그래프 임베딩은 자리별 평균이다. 첫 자리 합 $7 / 5$, 둘째 자리 합 $4.5 / 5$

ENG) b) $h(A) = [0, 1]$, $h(B) = [3, 0.6667]$, $h(C) = [4, 0.5]$, $h(D) = [0, 2.3333]$, $h(E) = [2, 0]$

ENG) c) $hG = [1.8, 0.9]$

ENG)

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 는 두 개의 군집 G_1 과 G_2 를 포함하며, 각 노드 옆의 수가 초기 특징이다.

ClusterGCN 모형에서는 이웃 중 자신과 같은 군집에 속한 노드만 평균에 포함한다.

- 미니배치 학습이란 무엇이며 ClusterGCN이 이를 필요로 하는 이유를 서술하시오.
- 층 $k = 1$ 에서 모든 노드의 출력 표현을 계산하시오.

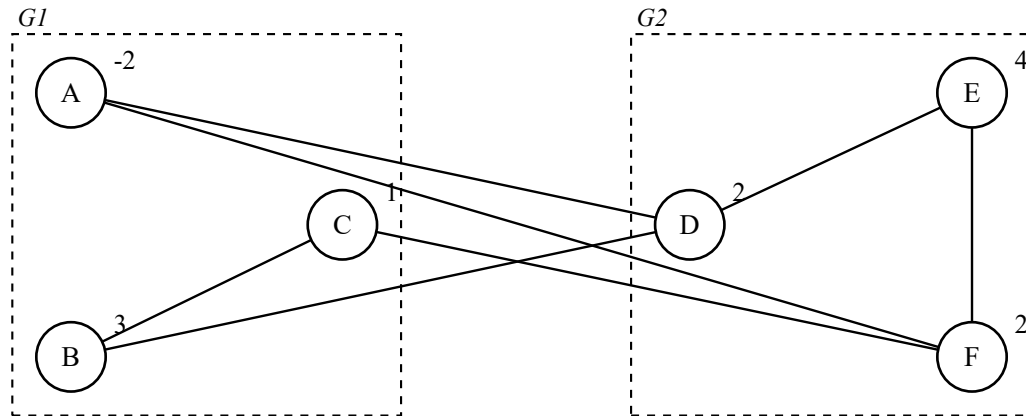


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 미니배치 학습은 한 번에 자료의 일부만 써서 파라미터를 고쳐요.
 - 큰 그래프는 메모리에 안 들어가요. 그래서 ClusterGCN은 안쪽이 촘촘한 덩어리로 나뉘요.
 - 잘 나누면 선 대부분이 배치 안에 남아서 경계에서 잃는 정보가 적어요.

ENG) Edges that cross a group are treated as absent. Cross them out first.

KOR) 묶음을 넘는 간선은 없는 것으로 본다. 먼저 그 간선에 사선을 그어 지운다.

ENG) A: no neighbour inside the group, so the mean is 0.

KOR) A: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) B: neighbours C. $(1)/1 = 1$

KOR) B: 이웃 C. $(1)/1 = 1$

ENG) C: neighbours B. $(3)/1 = 3$

KOR) C: 이웃 B. $(3)/1 = 3$

ENG) D: neighbours E. $(4)/1 = 4$

KOR) D: 이웃 E. $(4)/1 = 4$

ENG) E: neighbours D, F. $(2 + 2)/2 = 2$

KOR) E: 이웃 D, F. $(2 + 2)/2 = 2$

ENG) F: neighbours E. $(4)/1 = 4$

KOR) F: 이웃 E. $(4)/1 = 4$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU to both entries.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다. ReLU 는 두 자리 모두에 걸린다.

ENG) b) $h(A) = [0, 0]$, $h(B) = [3, 1]$, $h(C) = [1, 3]$, $h(D) = [2, 4]$, $h(E) = [4, 2]$, $h(F) = [2, 4]$

ENG)

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 에는 추출된 부분그래프 G_1 이 주어져 있으며, 각 노드 옆의 수가 초기 특징이다.

GraphSAINT 모형에서는 이웃 중 같은 부분그래프에 속한 노드만 평균에 포함한다.

- GraphSAINT의 추출 전략이 노드 단위 또는 층 단위 추출 방식과 어떻게 다른지 서술하시오.
- 층 $k = 1$ 에서 노드 A, B, C, D의 출력 표현을 계산하시오.

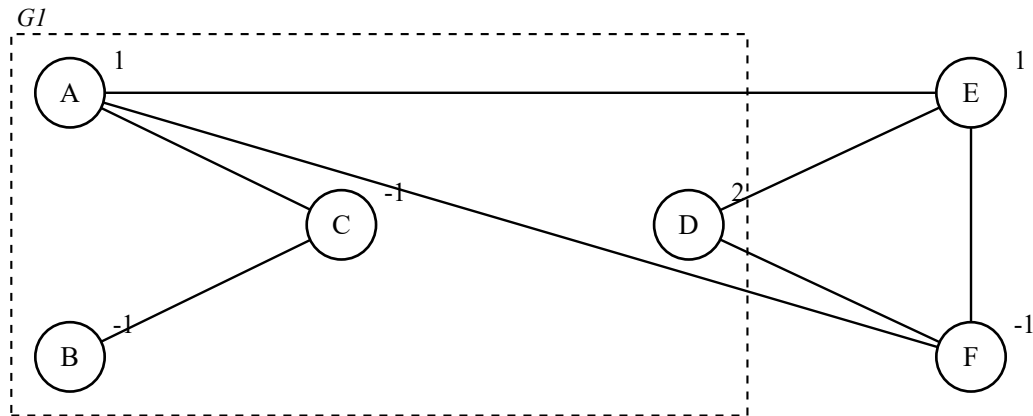


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE와 FastGCN은 층을 쌓으면서 뽑아요. 그래서 층마다 뽑기를 되풀이해요.
 - GraphSAINT는 부분 그래프를 먼저 뽑고 그 위에 GNN을 통째로 올려요.
 - 그래서 깊어져도 비용이 안 늘고, 정규화 계수로 뽑기의 치우침을 바로잡아요.

ENG) Edges that leave the sampled subgraph are discarded.

KOR) 뽑힌 부분그래프 밖으로 나가는 간선은 버린다.

ENG) A: neighbours C. $(-1)/1 = -1$

KOR) A: 이웃 C. $(-1)/1 = -1$

ENG) B: neighbours C. $(-1)/1 = -1$

KOR) B: 이웃 C. $(-1)/1 = -1$

ENG) C: neighbours A, B. $(1 + -1)/2 = 0$

KOR) C: 이웃 A, B. $(1 + -1)/2 = 0$

ENG) D: no neighbour inside the group, so the mean is 0.

KOR) D: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다.

ENG) b) $h(A) = [1, 0]$, $h(B) = [0, 0]$, $h(C) = [0, 0]$, $h(D) = [2, 0]$

ENG)

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

JK 신경망의 층 (k) 출력은 $\sigma(\tilde{A} H^{(0)})$ 부터 $\sigma(\tilde{A} H^{(k-1)})$ 까지의 원소별 최댓값으로 계산된다고 가정한다.

- Jumping Knowledge 신경망이 깊은 GCN 의 과평활 문제를 어떻게 완화하는지 설명하시오.
- $\tilde{A}$ 와 $H^{(1)}$ 을 계산하시오.
- 층 $k = 2$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -3 \\ -3 \\ -3 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 에서는 노드가 이웃 평균을 거듭 내서 표현들이 다 비슷해져요. 이것이 과평활이에요.
 - JKNet 은 모든 층의 표현을 들고 있다가 끝에서 최댓값, 이어 붙이기, LSTM 으로 합쳐요.
 - 알맞은 깊이는 노드마다 달라요. 그래서 노드마다 자기에게 맞는 깊이를 고를 수 있어요.

ENG) Degrees: $d(A)=4, d(B)=3, d(C)=1, d(D)=2, d(E)=2$

KOR) 차수를 센다. $d(A)=4, d(B)=3, d(C)=1, d(D)=2, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]$

ENG) First build $H1 = \text{ReLU}(\tilde{A}H0)$

KOR) 먼저 $H1$ 을 만든다. $H1 = \text{ReLU}(\tilde{A}H0)$

ENG) A: neighbours B, C, D, E. $(-3 + -3 + -3 + 3)/4 = -1.5$

KOR) A: 이웃 B, C, D, E. $(-3 + -3 + -3 + 3)/4 = -1.5$

ENG) B: neighbours A, D, E. $(4 + -3 + 3)/3 = 1.3333$

KOR) B: 이웃 A, D, E. $(4 + -3 + 3)/3 = 1.3333$

ENG) C: neighbours A. $(4)/1 = 4$

KOR) C: 이웃 A. $(4)/1 = 4$

ENG) D: neighbours A, B. $(4 + -3)/2 = 0.5$

KOR) D: 이웃 A, B. $(4 + -3)/2 = 0.5$

ENG) E: neighbours A, B. $(4 + -3)/2 = 0.5$

KOR) E: 이웃 A, B. $(4 + -3)/2 = 0.5$

ENG) ReLU clips the negatives: $-1.5 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1.5 \rightarrow 0$

ENG) $H1 = [0, 1.3333, 4, 0.5, 0.5]$

ENG) For $k = 2$ there are two terms. First: $\text{ReLU}(\tilde{A}H0) = [0, 1.3333, 4, 0.5, 0.5]$

KOR) $k=2$ 이므로 조각이 두 개다. 첫째 $\text{ReLU}(\tilde{A}H0) = [0, 1.3333, 4, 0.5, 0.5]$

ENG) Second: $\text{ReLU}(\tilde{A}H1) = [1.5833, 0.3333, 0, 0.6667, 0.6667]$

KOR) 둘째 $\text{ReLU}(\tilde{A}H1) = [1.5833, 0.3333, 0, 0.6667, 0.6667]$

ENG) The max is taken element-wise, not by choosing one whole vector.

KOR) max 는 가로줄마다 큰 쪽을 고른다. 벡터 하나를 고르는 것이 아니다.

ENG) c) $H2 = [1.5833, 1.3333, 4, 0.6667, 0.6667]$

ENG)

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCNII 모형의 층 (k) 출력은 $H^{(k)} = \sigma[((1-\beta)I_n) \cdot ((1-\alpha)\tilde{A} H^{(k-1)} + \alpha H^{(0)})]$ 로 계산된다고 가정한다. I_n 은 단위행렬이며 α 와 β 는 문제에 주어진 값을 사용한다.

- GCNII 가 GCN 의 깊이 한계를 극복하기 위해 도입한 두 가지 주요 구성 요소를 쓰시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 3 \\ 3 \\ 0 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 두 장치는 초기 잔차 연결과 항등 사상이에요.
 - 초기 잔차 연결은 층마다 입력 $H^{(0)}$ 의 일부를 알파만큼 다시 더해요.
 - 항등 사상은 가중치 행렬을 단위행렬에 가깝게 뒹요. 그래서 깊어져도 신호가 안 망가져요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=3$, $d(D)=2$, $d(E)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=3$, $d(D)=2$, $d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$

ENG) Start with the inner bracket. $H_0 = [3, -1.3333, -1.3333, 3, 3]$

KOR) 안쪽 괄호부터 푼다. $H_0 = [3, -1.3333, -1.3333, 3, 3]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[2.4, -1.0667, -1.0667, 2.4, 2.4]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[2.4, -1.0667, -1.0667, 2.4, 2.4]$

ENG) Add $\alpha H_0 = 0.2 \times [-3, 3, 3, 0, -1] = [-0.6, 0.6, 0.6, 0, -0.2]$

KOR) $\alpha H_0 = 0.2 \times [-3, 3, 3, 0, -1] = [-0.6, 0.6, 0.6, 0, -0.2]$ 를 더한다.

ENG) Inner bracket = $[1.8, -0.4667, -0.4667, 2.4, 2.2]$

KOR) 안쪽 괄호 = $[1.8, -0.4667, -0.4667, 2.4, 2.2]$

ENG) Multiply by the outer $(1-\beta) = 0.6$. Dropping this factor scales the answer.

KOR) 바깥의 $(1-\beta) = 0.6$ 를 곱한다. 이 계수를 빼트리면 답이 배로 커진다.

ENG) ReLU clips the negatives: $-0.28 \rightarrow 0, -0.28 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.28 \rightarrow 0, -0.28 \rightarrow 0$

ENG) c) $H_1 = [1.08, 0, 0, 1.44, 1.32]$

ENG)

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

잔차 연결을 갖는 DeepGCN 의 층 (k) 출력은 $H^{(k)} = \sigma(\tilde{A} H^{(k-1)}) + H^{(k-1)}$ 로 계산된다고 가정한다. 덧셈은 ReLU 적용 이후에 이루어진다.

- 잔차 연결이 GCN 을 더 깊게 쌓을 수 있게 하는 이유를 설명하시오.
- 층 $k = 1$ 과 $k = 2$ 에서의 출력 표현을 계산하시오.
- 잔차 연결이 없는 보통의 GCN 출력과 $H^{(1)}$ 을 비교하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 은 기울기 소실과 과평활을 겪어요. 그래서 층을 더하면 오히려 나빠져요.
 - 잔차 연결은 입력을 그대로 앞으로 보내요. 그래서 층은 바뀌는 양만 배우면 돼요.
 - 바꿀 게 없으면 층은 0 을 내보내면 돼요. 기울기도 그 지름길로 되돌아 흘러요.

ENG) Degrees: $d(A)=2, d(B)=3, d(C)=3, d(D)=2$

KOR) 차수를 센다. $d(A)=2, d(B)=3, d(C)=3, d(D)=2$

ENG) Divide each row by its degree. $\text{Atilde} = [0, 0.5, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0.3333]; [0, 0.5, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.5, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0.3333]; [0, 0.5, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) $\tilde{H}_0 = [1.5, -0.3333, -0.6667, 1.5]$

ENG) Apply ReLU first: $[1.5, 0, 0, 1.5]$

KOR) ReLU 를 먼저 건다. $[1.5, 0, 0, 1.5]$

ENG) Then add $\tilde{H}_0$. The addition sits outside the ReLU, so negatives can survive.

KOR) 그 다음에 $\tilde{H}_0$ 을 더한다. 더하기는 ReLU 바깥이라 음수가 살아남을 수 있다.

ENG) For $k = 2$, feed H_1 back in and add H_1 , not $\tilde{H}_0$.

KOR) $k=2$ 에서는 H_1 을 다시 넣고, 더하는 것도 $\tilde{H}_0$ 이 아니라 H_1 이다.

ENG) $\tilde{H}_1 = [1.5, 0.6667, 0.3333, 1.5]$, after ReLU $[1.5, 0.6667, 0.3333, 1.5]$

KOR) $\tilde{H}_1 = [1.5, 0.6667, 0.3333, 1.5]$, ReLU 뒤 $[1.5, 0.6667, 0.3333, 1.5]$

ENG) b) $H_1 = [-1.5, 1, 2, 1.5]$

ENG) $H_2 = [0, 1.6667, 2.3333, 3]$

ENG) c) Plain GCN without residual: $H_1 = [1.5, 0, 0, 1.5]$ (the node's own value and negatives disappear)

KOR) c) 잔차 없는 GCN 의 $H_1 = [1.5, 0, 0, 1.5]$ (잔차가 없으면 자기 값과 음수가 사라져요)

4회차

ENG)

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

- a) Explain the key assumption behind relational classification, and state one limitation.
- b) Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- c) Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 노드 1과 5에는 이름표가 주어져 있으며($P(Y_1)=1, P(Y_5)=0$), 나머지 노드는 이름표가 없고 0.5로 초기화한다.

관계형 분류기는 이름표가 없는 각 노드를 이웃 노드들의 부류 확률에 대한 가중 평균으로 갱신한다. 분모는 해당 노드에 연결된 간선 가중치의 합이다.

- a) 관계형 분류가 전제하는 핵심 가정과 그 한계 하나를 서술하시오.
- b) 1차 반복 후 이름표가 없는 각 노드의 $P(Y_v)$ 를 계산하시오.
- c) 2차 반복 후의 $P(Y_v)$ 를 계산하시오. 단, 직전 반복의 값만 사용한다.

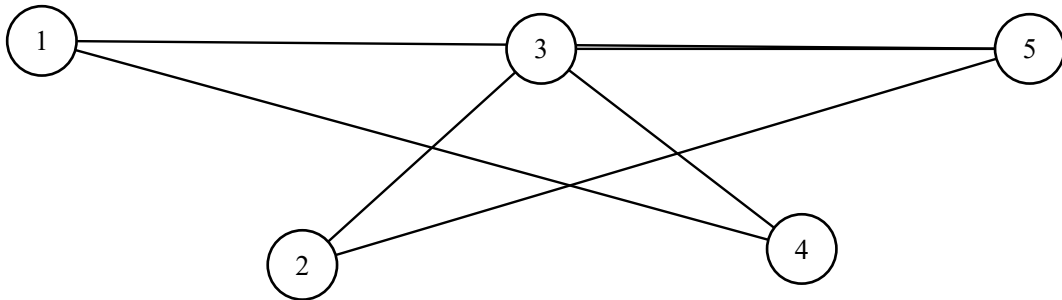


Figure. Graph G

풀이

ENG)

- a) Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.

KOR)

- a) 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 관계형 분류는 동질성을 가정해요. 이어진 노드끼리는 같은 이름표를 가지는 경향이 있다는 뜻이에요.
 - 이름표 없는 노드를 이웃들의 확률의 가중 평균으로 거듭 바뀌요.
 - 한계는 그래프 구조만 쓰고 노드의 특징은 무시한다는 것이에요.

ENG) Write the initial values: node 1 is 1, node 5 is 0, the rest start at 0.5.

KOR) 처음 값을 적는다. 이름표가 붙은 1번은 1, 5번은 0, 나머지는 0.5로 시작한다.

ENG) First iteration. Only the unlabelled nodes are replaced by the mean of their neighbours.

KOR) 1차 반복. 이름표가 없는 노드만 이웃의 평균으로 바꾼다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 3, 5. $(0.5 + 0)/2 = 0.25$

KOR) 2번: 이웃 3, 5. $(0.5 + 0)/2 = 0.25$

ENG) Node 3: neighbours 2, 4, 5. $(0.5 + 0.5 + 0)/3 = 0.3333$

KOR) 3번: 이웃 2, 4, 5. $(0.5 + 0.5 + 0)/3 = 0.3333$

ENG) Node 4: neighbours 1, 3. $(1 + 0.5)/2 = 0.75$

KOR) 4번: 이웃 1, 3. $(1 + 0.5)/2 = 0.75$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) b) $P(Y1) = 1$, $P(Y2) = 0.25$, $P(Y3) = 0.3333$, $P(Y4) = 0.75$, $P(Y5) = 0$

ENG) Second iteration. Use only the values from the first iteration, not the ones just computed.

KOR) 2차 반복. 방금 구한 값이 아니라 1차 반복의 값만 쓴다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 3, 5. $(0.3333 + 0)/2 = 0.1667$

KOR) 2번: 이웃 3, 5. $(0.3333 + 0)/2 = 0.1667$

ENG) Node 3: neighbours 2, 4, 5. $(0.25 + 0.75 + 0)/3 = 0.3333$

KOR) 3번: 이웃 2, 4, 5. $(0.25 + 0.75 + 0)/3 = 0.3333$

ENG) Node 4: neighbours 1, 3. $(1 + 0.3333)/2 = 0.6667$

KOR) 4번: 이웃 1, 3. $(1 + 0.3333)/2 = 0.6667$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) c) $P(Y1) = 1$, $P(Y2) = 0.1667$, $P(Y3) = 0.3333$, $P(Y4) = 0.6667$, $P(Y5) = 0$

ENG)

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \| \tilde{A}(:, u) \|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{D, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

KOR)

인접행렬 A 와 노드 특징 벡터 $h^{(0)}$ 가 주어진 무방향 그래프 G 를 고려한다.

FastGCN 은 GCN 한 층의 전체 이웃 합을 층 단위 몬테카를로 추정으로 대체한다. 각 층에서 t_l 개의 노드를 중요도 분포 $q(u) \propto \|A(:, u)\|^2$ 로부터 한 번 추출하며, 평균은 이웃 수가 아니라 추출 개수 t_l 로 나눈다.

- 층 단위 추출이 GraphSAGE 의 노드 단위 추출과 어떻게 다른지 서술하시오.
- 다섯 노드에 대한 중요도 분포 $q(u)$ 를 계산하시오.
- 주어진 추출 집합($t_l = 2$)에 대하여 $h^{(1)}(A)$ 의 추정값을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 3 \\ 3 \\ 2 \\ 0 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 뽑기는 노드마다 제 이웃을 뽑아서, 층이 깊어질수록 비용이 지수로 커져요.
 - FastGCN 은 층마다 정해진 개수를 한 번만 뽑고, 그 층의 모든 노드가 같이 써요.
 - 그래서 비용이 지수가 아니라 층 수에 비례해서만 늘어요.

ENG) Degrees: $d(A)=3, d(B)=2, d(C)=2, d(D)=3, d(E)=2$

KOR) 차수를 센다. $d(A)=3, d(B)=2, d(C)=2, d(D)=3, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0.5, 0.5, 0]; [0.5, 0.5, 0, 0, 0]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.5, 0, 0, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0.5, 0.5, 0]; [0.5, 0.5, 0, 0, 0]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.5, 0, 0, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) $q(u)$ uses the squared norm of column u of $\tilde{A}$, not of a row.

KOR) $q(u)$ 는 $\tilde{A}$ 의 **세로줄** 제곱합이다. 가로줄이 아니다.

ENG) Column A: squared sum = 0.6111

KOR) A 칸의 제곱합 = 0.6111

ENG) Column B: squared sum = 0.3611

KOR) B 칸의 제곱합 = 0.3611

ENG) Column C: squared sum = 0.3611

KOR) C 칸의 제곱합 = 0.3611

ENG) Column D: squared sum = 0.6111

KOR) D 칸의 제곱합 = 0.6111

ENG) Column E: squared sum = 0.2222

KOR) E 칸의 제곱합 = 0.2222

ENG) Divide by the total 2.1667 to turn them into probabilities.

KOR) 전체 합 2.1667 로 나눠 확률로 만든다.

ENG) b) $q(A) = 0.2821, q(B) = 0.1667, q(C) = 0.1667, q(D) = 0.2821, q(E) = 0.1026$

ENG) For (c) look only at the sampled nodes: read $\tilde{A}(A, u)$ for each.

KOR) c) 는 뽑힌 노드만 본다. A 가로줄에서 그 칸의 $\tilde{A}$ 값을 읽는다.

ENG) $\tilde{A}(A, D) h(D) = 0.3333 \times 0 = 0$

KOR) $\tilde{A}(A, D) \times h(D) = 0.3333 \times 0 = 0$

ENG) $\tilde{A}(A, E) h(E) = 0.3333 \times 2 = 0.6667$

KOR) $\tilde{A}(A, E) \times h(E) = 0.3333 \times 2 = 0.6667$

ENG) Add them and divide by the number sampled (2), not by the degree: $(0 + 0.6667)/2 = 0.3333$

KOR) 더한 뒤 이웃 수가 아니라 **뽑은 개수 2** 로 나눈다. $(0 + 0.6667)/2 = 0.3333$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $h_1(A) = 0.3333$

ENG)

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = B$.
- Calculate $E_{p(u_j | B)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(B)$.

KOR)

문제 11 과 동일한 그래프와 특징 벡터를 고려한다.

ASGCN 은 GCN 한 층을 확률 분포에 대한 기댓값으로 다시 쓴다. 여기서 $N(v_i) = \sum_j \tilde{A}_{ij}$ 는 정규화 계수이고, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$ 는 부모 노드 v_i 가 주어졌을 때 노드 u_j 를 추출할 확률이다.

- ASGCN 의 제안 분포가 FastGCN 과 달리 무엇에 의존하는지 서술하시오.
- 주어진 부모 노드에 대한 $N(v_i)$ 와 $p(u_j | v_i)$ 를 계산하시오.
- 해당 기댓값과 그 결과로 얻어지는 $h^{(1)}$ 을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 0 \\ -1 \\ 2 \\ 4 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - FastGCN 의 뽑기 확률은 미리 정해져 있고 부모 노드와 상관이 없어요.
 - ASGCN 은 뽑기 확률을 윗층 노드에 맞춰 정해요.
 - 그래서 뽑은 노드가 거의 다 쓸모 있고, 추정값의 흔들림이 줄어요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=4$, $d(D)=2$, $d(E)=1$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=4$, $d(D)=2$, $d(E)=1$

ENG) Divide each row by its degree. Atilde = $[0, 0.5, 0.5, 0, 0]$; $[0.3333, 0, 0.3333, 0.3333, 0]$; $[0.25, 0.25, 0, 0.25, 0.25]$; $[0, 0.5, 0.5, 0, 0]$; $[0, 0, 1, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0.3333, 0.3333, 0]; [0.25, 0.25, 0, 0.25, 0.25]; [0, 0.5, 0.5, 0, 0]; [0, 0, 1, 0, 0]$ (가로줄 합이 모두 1)

ENG) The parent node is B, so only its row of $\tilde{A}$ matters.

KOR) 부모 노드는 B 다. $\tilde{A}$ 의 그 줄만 본다.

ENG) $N(B) = \text{sum of that row} = 1$. With row normalisation it is 1.

KOR) $N(B) = \text{그 가로줄의 합} = 1$. 행 정규화라 1 이다.

ENG) $p(A|B) = \tilde{A}(B,A) / N = 0.3333$

KOR) $p(A|B) = \tilde{A}(B,A) / N = 0.3333 / 1 = 0.3333$

ENG) $p(C|B) = \tilde{A}(B,C) / N = 0.3333$

KOR) $p(C|B) = \tilde{A}(B,C) / N = 0.3333 / 1 = 0.3333$

ENG) $p(D|B) = \tilde{A}(B,D) / N = 0.3333$

KOR) $p(D|B) = \tilde{A}(B,D) / N = 0.3333 / 1 = 0.3333$

ENG) b) $N(B) = 1$, $p(A|B) = 0.3333$, $p(C|B) = 0.3333$, $p(D|B) = 0.3333$

ENG) The expectation is the sum of probability times value: $0.3333 \times 0 + 0.3333 \times 2 + 0.3333 \times 4 = 2$

KOR) 기댓값은 확률 곱하기 값의 합이다. $0.3333 \times 0 + 0.3333 \times 2 + 0.3333 \times 4 = 2$

ENG) $N = 1$ and $W = 1$, so the value is unchanged. It equals the plain neighbour mean.

KOR) $N = 1$, $W = 1$ 이라 곱해도 그대로다. 결국 이웃 평균과 같은 값이다.

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $E = 2$, $h_1(B) = 2$

ENG)

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

KOR)

인접행렬 A 와 다층 퍼셉트론이 산출한 예측 행렬 H 가 주어졌다.

APPNP 는 순간이동 확률 α 를 사용하여 예측을 전파한다. 전파 단계 사이에는 비선형 함수를 적용하지 않는다.

- 개인화 PageRank 의 순간이동 기법을 설명하고, 그것이 과평활을 막는 이유를 서술하시오.
- $\tilde{A}$ 와 $Z^{(1)}$ 을 계산하시오.
- $Z^{(2)}$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 퍼뜨리는 걸음마다 확률 알파로 출발 노드로 되돌아가요. 이것이 순간이동 장치예요.
 - 매 걸음 일정 비율이 되돌아오니까 분포가 출발점 근처에 머물러요.
 - 그래서 많이 퍼뜨려도 과평활이 안 생기고, 퍼뜨리는 데에 추가 파라미터도 없어요.

ENG) Degrees: $d(A)=3, d(B)=1, d(C)=2, d(D)=2$

KOR) 차수를 센다. $d(A)=3, d(B)=1, d(C)=2, d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333]; [1, 0, 0, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0.3333, 0.3333]; [1, 0, 0, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.3333, 0.3333, 0.3333]; [1, 0, 0, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0.5, 0]$

ENG) Start from $Z_0 = H = [1, 0, 0, 0]$

KOR) $Z_0 = H$ 로 시작한다. $[1, 0, 0, 0]$

ENG) Propagate one step: $\tilde{Z}_0 = [0, 1, 0.5, 0.5]$

KOR) 한 걸음 퍼뜨린다. $\tilde{Z}_0 = [0, 1, 0.5, 0.5]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[0, 0.8, 0.4, 0.4]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[0, 0.8, 0.4, 0.4]$

ENG) Add $\alpha H = [0.2, 0, 0, 0]$. This is the teleport term.

KOR) $\alpha H = 0.2 \times [1, 0, 0, 0] = [0.2, 0, 0, 0]$ 를 더한다. 이것이 순간이동이다.

ENG) $Z_1 = [0.2, 0.8, 0.4, 0.4]$

ENG) One more step: $\tilde{Z}_1 = [0.5333, 0.2, 0.3, 0.3]$

KOR) 한 걸음 더. $\tilde{Z}_1 = [0.5333, 0.2, 0.3, 0.3]$

ENG) Multiply by $(1-\alpha)$ and add αH **again**. It is added at every step.

KOR) $(1-\alpha)$ 를 곱하고 **αH 를 또** 더한다. 매 걸음 더하는 것이 핵심이다.

ENG) c) $Z_2 = [0.6267, 0.16, 0.24, 0.24]$

ENG)

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.5$, $\gamma_1 = 0.4$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

KOR)

인접행렬 A 와 신경망이 산출한 은닉 특징 $H^{(0)}$ 이 주어졌다.

GPR-GNN 은 특징 추출과 위상 전파를 분리한다. $\tilde{A}$ 를 반복 적용하여 $H^{(k)}$ 를 얻고, 학습된 가중치 γ_k 로 가중합한다. 본 문항에서는 $K = 2$ 이며 $\gamma_0, \gamma_1, \gamma_2$ 가 주어진다.

- GPR-GNN 이 APPNP 와 달리 할 수 있는 것은 무엇이며, γ_k 가 음수가 될 수 있다는 점이 이질적 그래프에서 왜 중요한지 서술하시오.
- $H^{(1)}$ 과 $H^{(2)}$ 를 계산하시오.
- Z 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 3 \\ -2 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - APPNP 는 계수를 알파 곱하기 (1-알파)의 k제곱으로 못 박아요. 늘 양수이고 줄어들어요.
 - GPR-GNN 은 그 무게를 학습하고, 음수도 될 수 있어요.
 - 음수 무게는 그 걸음을 빼요. 그래서 이질 그래프에서 고주파 필터처럼 동작할 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=1$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=1$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333]; [0.5, 0.5, 0, 0]; [0, 1, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333]; [0.5, 0.5, 0, 0]; [0, 1, 0, 0]$ (가로줄 합이 모두 1)

ENG) H_0 stays as it is: $[-1, 3, -2, 3]$

KOR) H_0 은 그대로 둔다. $[-1, 3, -2, 3]$

ENG) $H_1 = \text{Atilde } H_0 = [0.5, 0, 1, 3]$

ENG) $H_2 = \text{Atilde } H_1 = [0.5, 1.5, 0.25, 0]$

ENG) b) $H_1 = [0.5, 0, 1, 3]$, $H_2 = [0.5, 1.5, 0.25, 0]$

ENG) Now take the weighted sum with gamma. There is no ReLU during propagation.

KOR) 이제 gamma 로 가중합한다. 전파 중에 ReLU 는 없다.

ENG) $\text{gamma}_0 H_0 = 0.5 \times [-1, 3, -2, 3] = [-0.5, 1.5, -1, 1.5]$

KOR) $\text{gamma}_0 \times H_0 = 0.5 \times [-1, 3, -2, 3] = [-0.5, 1.5, -1, 1.5]$

ENG) $\text{gamma}_1 H_1 = 0.4 \times [0.5, 0, 1, 3] = [0.2, 0, 0.4, 1.2]$

KOR) $\text{gamma}_1 \times H_1 = 0.4 \times [0.5, 0, 1, 3] = [0.2, 0, 0.4, 1.2]$

ENG) $\text{gamma}_2 H_2 = -0.2 \times [0.5, 1.5, 0.25, 0] = [-0.1, -0.3, -0.05, 0]$

KOR) $\text{gamma}_2 \times H_2 = -0.2 \times [0.5, 1.5, 0.25, 0] = [-0.1, -0.3, -0.05, 0]$

ENG) gamma_2 is negative, so that hop is subtracted. Dropping the sign gives a wrong answer.

KOR) gamma_2 가 음수라 그 걸음은 빼는 효과다. 부호를 빼뜨리면 틀린다.

ENG) Add the three vectors element-wise.

KOR) 세 벡터를 자리별로 더한다.

ENG) c) $Z = [-0.4, 1.2, -0.65, 2.7]$

ENG)

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

MixHop 한 층은 인접행렬의 여러 거듭제곱 $\hat{A}^j$ ($j \in P = \{0, 1, 2\}$)를 각각 계산한 뒤 열 방향으로 이어 붙인다.

본 문항에서 가중치 행렬은 모두 1 로 둔다.

a) 두 걸음 델타 연산자에 관한 MixHop 의 정리 1 을 서술하고, 보통의 GCN 이 층을 무한히 쌓아도 이를 표현할 수 없는 이유를 설명하시오.

b) $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$, $\hat{A}^2 H^{(0)}$ 을 계산하시오.

c) $H^{(1)}$ 과 델타 연산자 $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ -2 \\ -2 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 정리 1 은 보통 GCN 이 두 걸음 델타 연산자를 표현할 수 없다고 말해요. 층을 무한히 쌓아도요.
 - GCN 층을 쌓으면 A 에 1제곱 곱하기 W 꼴로 뭉쳐져서, 중간 걸음들이 따로 남지 않아요.
 - MixHop 은 거듭제곱마다 따로 구해서 나란히 붙여요. 그래서 뒤 층이 하나에서 다른 하나를 뺄 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) A^0 is the identity, so it is just $H_0: [-2, -2, -2, 3]$

KOR) A_0 은 단위행렬이라 H_0 그대로다. $[-2, -2, -2, 3]$

ENG) $A^1 H_0 = [-2, 0.5, 0.5, -2]$ (one hop, the neighbour mean)

KOR) $A_1 H_0 = [-2, 0.5, 0.5, -2]$ (한 걸음, 이웃 평균)

ENG) $A^2 H_0 = \tilde{A}(\tilde{A} H_0) = [0.5, -2, -2, 0.5]$ (two hops)

KOR) $A_2 H_0 = \tilde{A}(\tilde{A} H_0) = [0.5, -2, -2, 0.5]$ (두 걸음)

ENG) ReLU is applied to each block separately, not after concatenation.

KOR) 조각마다 따로 ReLU 를 건다. 이어 붙인 뒤가 아니다.

ENG) After ReLU: $[0, 0, 0, 3] \parallel [0, 0.5, 0.5, 0] \parallel [0.5, 0, 0, 0.5]$

KOR) ReLU 뒤: $[0, 0, 0, 3] \parallel [0, 0.5, 0.5, 0] \parallel [0.5, 0, 0, 0.5]$

ENG) The delta operator subtracts the two-hop block from the one-hop block.

KOR) 델타 연산자는 한 걸음에서 두 걸음을 뺀 것이다.

ENG) b) $A_0 H = [-2, -2, -2, 3]$, $A_1 H = [-2, 0.5, 0.5, -2]$, $A_2 H = [0.5, -2, -2, 0.5]$

ENG) c) $H_1 =$ concatenation of the three blocks: $[0, 0, 0, 3] \parallel [0, 0.5, 0.5, 0] \parallel [0.5, 0, 0, 0.5]$

KOR) c) $H_1 =$ 세 조각을 이어 붙여요: $[0, 0, 0, 3] \parallel [0, 0.5, 0.5, 0] \parallel [0.5, 0, 0, 0.5]$

ENG) $\Delta = [-0.5, 0.5, 0.5, -0.5]$

ENG)

7. (PairNorm) After one GCN layer the node features of a four-node graph are PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

KOR)

GCN 한 층을 통과한 뒤의 노드 특징 행렬 X 가 주어졌다. 행은 노드, 열은 특징 차원이다.

PairNorm 은 중심화 단계와 크기 조정 단계로 정규화한다. 중심화는 노드들에 대한 평균을 빼는 것이고, 크기 조정은 중심화된 벡터들의 노름 제곱 평균의 제곱근으로 나누는 것이다. $s = 1, n = 4$ 로 한다.

- 노드 단위 과평활과 특징 단위 과평활을 각각 설명하고, PairNorm 이 층에 걸쳐 일정하게 유지하는 양이 무엇인지 서술하시오.
- 중심화된 특징을 계산하시오.
- 크기 조정의 분모와 최종 x_i 를 계산하시오.

$$X = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 4 & 0 \\ -2 & 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 과평활은 이어진 노드들의 특징이 너무 비슷해지는 것이에요.
 - 특징 단위 과평활은 그래프 전체에서 모든 노드 특징이 구별이 안 되게 되는 것이에요.
 - PairNorm 은 층을 지나도 노드 사이 거리 제곱의 합을 일정하게 지켜요.

ENG) First take the mean over the nodes, separately for each feature column.

KOR) 먼저 노드들에 대한 평균을 구한다. 특징 차원마다 따로 구한다.

ENG) Column 1: $(-1 + 0 + 4 + -2)/4 = 0.25$

KOR) 첫 칸: $(-1 + 0 + 4 + -2)/4 = 0.25$

ENG) Column 2: $(0 + 0 + 0 + 0)/4 = 0$

KOR) 둘째 칸: $(0 + 0 + 0 + 0)/4 = 0$

ENG) Subtract the mean from every row. Check: each column now sums to zero.

KOR) 가로줄마다 평균을 뺀다. 검산: 각 칸의 합이 0 이 된다.

ENG) Node 1: squared norm = 1.5625

KOR) 노드 1 의 길이 제곱 = 1.5625

ENG) Node 2: squared norm = 0.0625

KOR) 노드 2 의 길이 제곱 = 0.0625

ENG) Node 3: squared norm = 14.0625

KOR) 노드 3 의 길이 제곱 = 14.0625

ENG) Node 4: squared norm = 5.0625

KOR) 노드 4 의 길이 제곱 = 5.0625

ENG) Mean = $(1.5625 + 0.0625 + 14.0625 + 5.0625)/4 = 5.1875$

KOR) 평균 = $(1.5625 + 0.0625 + 14.0625 + 5.0625)/4 = 5.1875$

ENG) Take the square root: $\sqrt{5.1875} = 2.2776$. Divide every row by this single number.

KOR) 제곱근을 씌운다. $\sqrt{5.1875} = 2.2776$. 이 **하나의 수** 로 모든 줄을 나눈다.

ENG) b) Mean = [0.25, 0], centered = [-1.25, 0]; [-0.25, 0]; [3.75, 0]; [-2.25, 0]

KOR) b) 평균 = [0.25, 0], 중심을 뺀 것 = [-1.25, 0]; [-0.25, 0]; [3.75, 0]; [-2.25, 0]

ENG) c) Scaling denominator = 2.2776, $x = [-0.5488, 0]; [-0.1098, 0]; [1.6465, 0]; [-0.9879, 0]$

KOR) c) 분모 = 2.2776, $x = [-0.5488, 0]; [-0.1098, 0]; [1.6465, 0]; [-0.9879, 0]$

ENG)

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (A, D) and (A, C) are dropped. Recalculate $H^{(1)}$.

KOR)

노드 A, B, C, D 로 이루어진 완전그래프 K_4 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

DropEdge 는 매 에폭마다 각 무방향 간선을 확률 $1-p$ 로 독립적으로 유지한다. 이때 $A^{(t)}$ 와 $D^{(t)}$ 는 살아남은 간선만으로 다시 구성한다. $p = 1/3$ 으로 한다.

- DropEdge 가 목표로 하는 두 가지 문제를 쓰고, Dropout 과 어떻게 다른지 서술하시오.
- 살아남은 간선 수의 기댓값과, 간선을 제거하지 않은 원래 그래프에서의 $H^{(1)}$ 을 계산하시오.
- 주어진 두 간선이 제거되었을 때의 $H^{(1)}$ 을 다시 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 0 \\ 3 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - DropEdge 는 과적합과 과평활을 노려요.
 - 판마다 다른 부분 그래프를 써요. 그래프에 자료를 늘리는 효과가 나요.
 - Dropout 은 특징 값을 0 으로 꺼요. DropEdge 는 인접행렬의 선을 지워요.

ENG) K_4 has 6 edges. Each survives with probability $1 - 1/3 = 2/3$, so the expectation is 4.

KOR) K_4 는 간선이 6 개다. 남을 확률이 $1 - 1/3 = 2/3$ 이므로 기댓값은 $6 \times 2/3 = 4$ 개다.

ENG) First the full graph: every node has degree 3.

KOR) 먼저 원래 그래프. 모든 노드의 차수가 3 다.

ENG) A: neighbours B, C, D. $(0 + 3 + 3)/3 = 2$

KOR) A: 이웃 B, C, D. $(0 + 3 + 3)/3 = 2$

ENG) B: neighbours A, C, D. $(-1 + 3 + 3)/3 = 1.6667$

KOR) B: 이웃 A, C, D. $(-1 + 3 + 3)/3 = 1.6667$

ENG) C: neighbours A, B, D. $(-1 + 0 + 3)/3 = 0.6667$

KOR) C: 이웃 A, B, D. $(-1 + 0 + 3)/3 = 0.6667$

ENG) D: neighbours A, B, C. $(-1 + 0 + 3)/3 = 0.6667$

KOR) D: 이웃 A, B, C. $(-1 + 0 + 3)/3 = 0.6667$

ENG) Now drop the two given edges and **recount the degrees**. This is where most mistakes happen.

KOR) 이제 주어진 두 간선을 지운다. **차수를 다시 센다**. 여기서 거의 다 틀린다.

ENG) Degrees after dropping: $d(A)=1$, $d(B)=3$, $d(C)=2$, $d(D)=2$

KOR) 지운 뒤 차수: $d(A)=1$, $d(B)=3$, $d(C)=2$, $d(D)=2$

ENG) A: neighbours B. $(0)/1 = 0$

KOR) A: 이웃 B. $(0)/1 = 0$

ENG) B: neighbours A, C, D. $(-1 + 3 + 3)/3 = 1.6667$

KOR) B: 이웃 A, C, D. $(-1 + 3 + 3)/3 = 1.6667$

ENG) C: neighbours B, D. $(0 + 3)/2 = 1.5$

KOR) C: 이웃 B, D. $(0 + 3)/2 = 1.5$

ENG) D: neighbours B, C. $(0 + 3)/2 = 1.5$

KOR) D: 이웃 B, C. $(0 + 3)/2 = 1.5$

ENG) b) Expected number of surviving edges = 4, $H1 = [2, 1.6667, 0.6667, 0.6667]$

KOR) b) 남은 선 수의 기댓값 = 4, $H1 = [2, 1.6667, 0.6667, 0.6667]$

ENG) c) $H1 = [0, 1.6667, 1.5, 1.5]$

ENG)

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

KOR)

다음 그림의 무방향 그래프 G 를 고려한다. 임의보행이 노드 A 에서 노드 B 로 이동하여 현재 B 에 있으며, $p = 2, q = 0.5$ 이다.

B 에서 다음 노드 x 로의 비정규화 전이확률은 $\pi_{Bx} = \alpha_{pq}(A, x)$ 이며, α 는 직전 노드 A 로부터의 최단경로 거리에 따라 결정된다.

- BFS 와 DFS 를 설명하고, q 가 작을 때 보행이 둘 중 어느 쪽처럼 동작하는지 서술하시오.
- 노드 B 에서 이동 가능한 모든 노드를 열거하고 비정규화 전이확률 π_{Bx} 를 계산하시오.
- 정규화된 전이확률을 계산하시오.

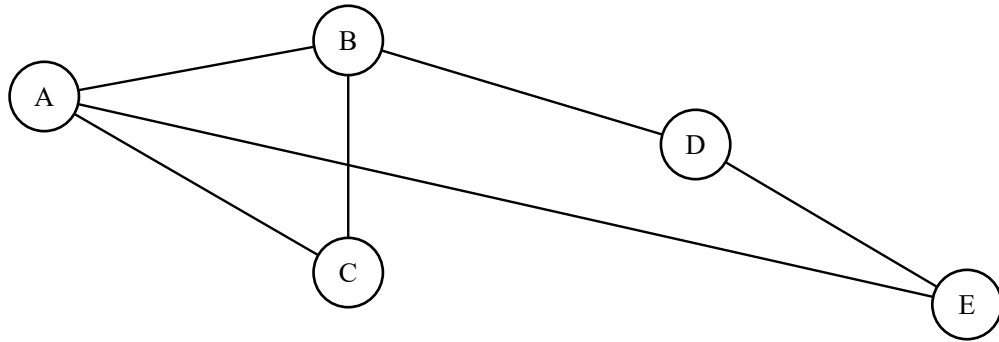


Figure 1. Graph G

풀이

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - BFS 는 바로 옆 이웃부터 훑어요. 그래서 임베딩이 노드의 구조적 역할을 담아요.
 - DFS 는 출발점에서 멀리 가요. 그래서 임베딩이 어느 동네 소속인지를 담아요.
 - q 가 작으면 $1/q$ 가 커져서 걸음이 바깥으로 나가요. DFS 처럼 움직여요.

ENG) The previous node is A and the walk is now at B. All distances are measured from A.

KOR) 직전 노드는 A, 지금 노드는 B 다. 거리는 모두 A 로부터 잴다.

ENG) A: $x = A$ is the previous node, $d = 0$, $\alpha = 1/p = 0.5$

KOR) A: $x = A$ 는 직전 노드라 $d = 0$. $\alpha = 1/p = 0.5$

ENG) C: x is also a neighbour of A, $d = 1$, $\alpha = 1 = 1$

KOR) C: A 와도 이웃이라 $d = 1$. $\alpha = 1 = 1$

ENG) D: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) D: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) Sum them: $0.5 + 1 + 2 = 3.5$

KOR) 세 값을 더한다. $0.5 + 1 + 2 = 3.5$

ENG) b) $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 1$, $\pi(B \rightarrow D) = 2$

ENG) c) $P(A) = 0.1429$, $P(C) = 0.2857$, $P(D) = 0.5714$

ENG)

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = -1, u'_C = 0.5, u'_D = 1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- What is second-order proximity, and how does it differ from first-order proximity?
- Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

KOR)

노드 임베딩 u 와 맥락 임베딩 u' 가 주어진 무방향 그래프 G 를 고려한다.

2차 근접도는 $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$ 로 정의한다. 지수 함수 값은 문제에 주어진 표를 사용한다.

- 2차 근접도란 무엇이며 1차 근접도와 어떻게 다른지 서술하시오.
- 주어진 기준 노드에 대하여 네 노드의 p_2 를 계산하시오.

풀이

ENG)

- Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 1차 근접도는 선으로 직접 이어진 두 노드 사이의 비슷함이에요.
 - 2차 근접도는 이웃을 많이 공유하는 두 노드의 비슷함이에요. 선이 없어도 돼요.
 - 그래서 LINE 은 노드마다 벡터를 두 개 뒹요. 자기 벡터와 맥락 벡터예요.

ENG) The base node is B with $u(B) = 0.5$. The other side uses the context vector u' .

KOR) 기준 노드는 B 이고 $u(B) = 0.5$ 다. 상대는 맥락 벡터 u' 를 쓴다.

ENG) A: $u'(A) u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

KOR) A: $u'(A) \times u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

ENG) B: $u'(B) u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

KOR) B: $u'(B) \times u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

ENG) C: $u'(C) u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

KOR) C: $u'(C) \times u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

ENG) D: $u'(D) u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

KOR) D: $u'(D) \times u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

ENG) The denominator is the sum of all four: $0.78 + 0.61 + 1.28 + 1.65 = 4.32$

KOR) 분모는 넷을 모두 더한 값이다. $0.78 + 0.61 + 1.28 + 1.65 = 4.32$

ENG) b) $p_2(A|B) = 0.1806$, $p_2(B|B) = 0.1412$, $p_2(C|B) = 0.2963$, $p_2(D|B) = 0.3819$

ENG)

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCN 모형의 층 (k) 에서의 은닉층은 $H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)})$ 로 계산된다고 가정한다. 여기서 $\tilde{A} = D^{-1}A$ 는 정규화된 인접행렬이고 σ 는 ReLU 이다.

- GCN 한 층에서 정보가 어떻게 전파되는지 설명하시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서 GCN 모형의 출력을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ -3 \\ -2 \\ 2 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 한 층에서 노드는 바로 옆 이웃의 특징을 모아 평균을 내요.
 - 행 정규화 행렬 $\tilde{A}$ 데는 D 의 역행렬 곱하기 A 이고, 그것이 평균을 내 줘요.
 - k 개 층을 지나면 노드는 k 걸음 안의 모든 노드 정보를 받아요.

ENG) Degrees: $d(A)=4, d(B)=2, d(C)=2, d(D)=2, d(E)=2$

KOR) 차수를 센다. $d(A)=4, d(B)=2, d(C)=2, d(D)=2, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0, 0.5]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0.5, 0, 0]; [0.5, 0.5, 0, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0, 0.5]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0.5, 0, 0]; [0.5, 0.5, 0, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0, 0.5]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0.5, 0, 0]; [0.5, 0.5, 0, 0, 0]$

ENG) Multiply $\tilde{A}$ by H_0 . Each entry is the mean of that node's neighbours.

KOR) $\tilde{A}$ 를 H_0 에 곱한다. 노드마다 이웃의 평균이다.

ENG) A: neighbours B, C, D, E. $(-3 + -2 + 2 + -2)/4 = -1.25$

KOR) A: 이웃 B, C, D, E. $(-3 + -2 + 2 + -2)/4 = -1.25$

ENG) B: neighbours A, E. $(-1 + -2)/2 = -1.5$

KOR) B: 이웃 A, E. $(-1 + -2)/2 = -1.5$

ENG) C: neighbours A, D. $(-1 + 2)/2 = 0.5$

KOR) C: 이웃 A, D. $(-1 + 2)/2 = 0.5$

ENG) D: neighbours A, C. $(-1 + -2)/2 = -1.5$

KOR) D: 이웃 A, C. $(-1 + -2)/2 = -1.5$

ENG) E: neighbours A, B. $(-1 + -3)/2 = -2$

KOR) E: 이웃 A, B. $(-1 + -3)/2 = -2$

ENG) ReLU clips the negatives: $-1.25 \rightarrow 0, -1.5 \rightarrow 0, -1.5 \rightarrow 0, -2 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1.25 \rightarrow 0, -1.5 \rightarrow 0, -1.5 \rightarrow 0, -2 \rightarrow 0$

ENG) c) $H_1 = [0, 0, 0.5, 0, 0]$

ENG)

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 각 노드 옆의 수가 초기 특징이다.

집계 함수가 평균인 GraphSAGE 모형에서, 노드 i 의 층 k 특징은 이웃의 평균을 자신의 특징에 이어 붙인 뒤 ReLU를 적용하여 갱신된다.

- GraphSAGE의 핵심 착상은 무엇이며 기존 GCN과 어떻게 다른지 서술하시오.
- $k = 1$ 에서 각 노드의 특징을 계산하시오.
- $k = 1$ 일 때 평균 전역 풀링으로 그래프 수준 임베딩 h_G 를 계산하시오.

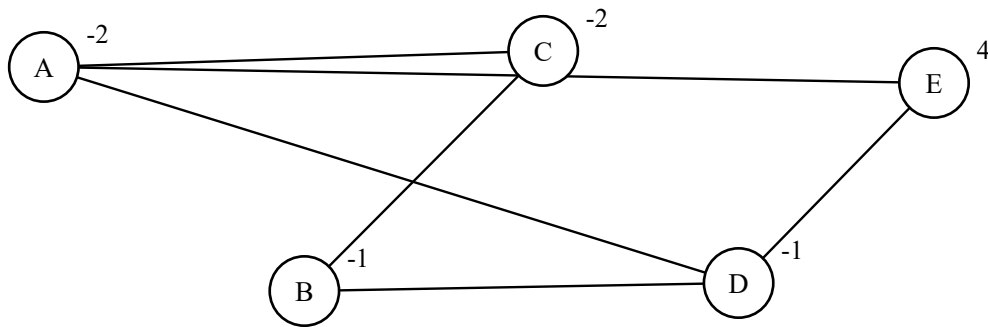


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE는 노드마다 고정된 벡터를 배우는 대신 모으는 함수를 배워요.
 - 이웃을 정해진 개수만 뽑아요. 그래서 아주 큰 그래프에서도 미니배치로 학습할 수 있어요.
 - GCN은 전이적이고 GraphSAGE는 귀납적이에요. 그래서 학습 때 못 본 노드도 벡터를 만들 수 있어요.

ENG) First compute the neighbour mean. The node itself is not included.

KOR) 먼저 이웃 평균을 구한다. 자기 자신은 넣지 않는다.

ENG) A: neighbours C, D, E. $(-2 + -1 + 4)/3 = 0.3333$

KOR) A: 이웃 C, D, E. $(-2 + -1 + 4)/3 = 0.3333$

ENG) B: neighbours C, D. $(-2 + -1)/2 = -1.5$

KOR) B: 이웃 C, D. $(-2 + -1)/2 = -1.5$

ENG) C: neighbours A, B. $(-2 + -1)/2 = -1.5$

KOR) C: 이웃 A, B. $(-2 + -1)/2 = -1.5$

ENG) D: neighbours A, B, E. $(-2 + -1 + 4)/3 = 0.3333$

KOR) D: 이웃 A, B, E. $(-2 + -1 + 4)/3 = 0.3333$

ENG) E: neighbours A, D. $(-2 + -1)/2 = -1.5$

KOR) E: 이웃 A, D. $(-2 + -1)/2 = -1.5$

ENG) Concatenate the node's own value with the neighbour mean. This is not an addition.

KOR) 자기 값을 앞에, 이웃 평균을 뒤에 이어 붙인다. 더하는 것이 아니다.

ENG) ReLU clips the negatives: $-2 \rightarrow 0, -1 \rightarrow 0, -1.5 \rightarrow 0, -2 \rightarrow 0, -1.5 \rightarrow 0, -1 \rightarrow 0, -1.5 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-2 \rightarrow 0, -1 \rightarrow 0, -1.5 \rightarrow 0, -2 \rightarrow 0, -1.5 \rightarrow 0, -1 \rightarrow 0, -1.5 \rightarrow 0$

ENG) The graph embedding is the mean per position: first $-2 / 5$, second $-3.8333 / 5$

KOR) 그래프 임베딩은 자리별 평균이다. 첫 자리 합 $-2 / 5$, 둘째 자리 합 $-3.8333 / 5$

ENG) b) $h(A) = [0, 0.3333], h(B) = [0, 0], h(C) = [0, 0], h(D) = [0, 0.3333], h(E) = [4, 0]$

ENG) c) $hG = [0.8, 0.1333]$

ENG)

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 는 두 개의 군집 G_1 과 G_2 를 포함하며, 각 노드 옆의 수가 초기 특징이다.

ClusterGCN 모형에서는 이웃 중 자신과 같은 군집에 속한 노드만 평균에 포함한다.

- 미니배치 학습이란 무엇이며 ClusterGCN 이 이를 필요로 하는 이유를 서술하시오.
- 층 $k = 1$ 에서 모든 노드의 출력 표현을 계산하시오.

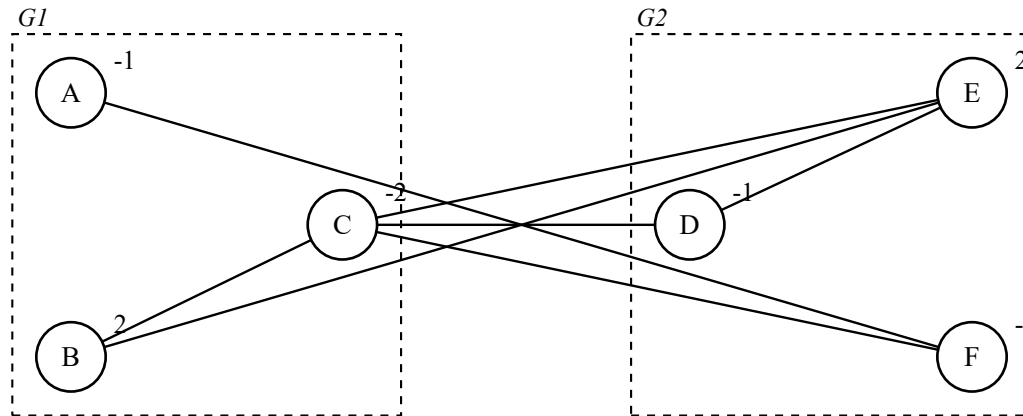


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 미니배치 학습은 한 번에 자료의 일부만 써서 파라미터를 고쳐요.
 - 큰 그래프는 메모리에 안 들어가요. 그래서 ClusterGCN 은 안쪽이 촘촘한 덩어리로 나뉘요.
 - 잘 나누면 선 대부분이 배치 안에 남아서 경계에서 잃는 정보가 적어요.

ENG) Edges that cross a group are treated as absent. Cross them out first.

KOR) 묶음을 넘는 간선은 없는 것으로 본다. 먼저 그 간선에 사선을 그어 지운다.

ENG) A: no neighbour inside the group, so the mean is 0.

KOR) A: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) B: neighbours C. $(-2)/1 = -2$

KOR) B: 이웃 C. $(-2)/1 = -2$

ENG) C: neighbours B. $(2)/1 = 2$

KOR) C: 이웃 B. $(2)/1 = 2$

ENG) D: neighbours E. $(2)/1 = 2$

KOR) D: 이웃 E. $(2)/1 = 2$

ENG) E: neighbours D. $(-1)/1 = -1$

KOR) E: 이웃 D. $(-1)/1 = -1$

ENG) F: no neighbour inside the group, so the mean is 0.

KOR) F: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU to both entries.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다. ReLU 는 두 자리 모두에 걸린다.

ENG) b) $h(A) = [0, 0]$, $h(B) = [2, 0]$, $h(C) = [0, 2]$, $h(D) = [0, 2]$, $h(E) = [2, 0]$, $h(F) = [0, 0]$

ENG)

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 에는 추출된 부분그래프 G_1 이 주어져 있으며, 각 노드 옆의 수가 초기 특징이다.

GraphSAINT 모형에서는 이웃 중 같은 부분그래프에 속한 노드만 평균에 포함한다.

- GraphSAINT의 추출 전략이 노드 단위 또는 층 단위 추출 방식과 어떻게 다른지 서술하시오.
- 층 $k = 1$ 에서 노드 A, B, C, D의 출력 표현을 계산하시오.

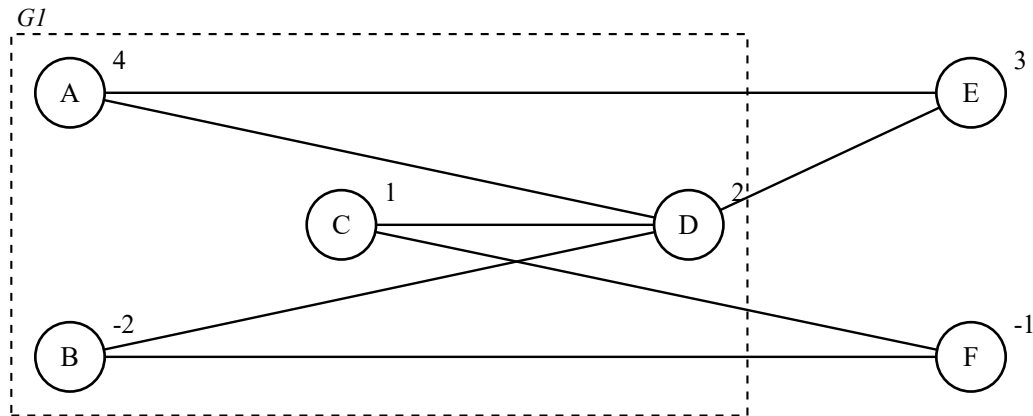


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE와 FastGCN은 층을 쌓으면서 뽑아요. 그래서 층마다 뽑기를 되풀이해요.
 - GraphSAINT는 부분 그래프를 먼저 뽑고 그 위에 GNN을 통째로 올려요.
 - 그래서 깊어져도 비용이 안 늘고, 정규화 계수로 뽑기의 치우침을 바로잡아요.

ENG) Edges that leave the sampled subgraph are discarded.

KOR) 뽑힌 부분그래프 밖으로 나가는 간선은 버린다.

ENG) A: neighbours D. $(2)/1 = 2$

KOR) A: 이웃 D. $(2)/1 = 2$

ENG) B: neighbours D. $(2)/1 = 2$

KOR) B: 이웃 D. $(2)/1 = 2$

ENG) C: neighbours D. $(2)/1 = 2$

KOR) C: 이웃 D. $(2)/1 = 2$

ENG) D: neighbours A, B, C. $(4 + -2 + 1)/3 = 1$

KOR) D: 이웃 A, B, C. $(4 + -2 + 1)/3 = 1$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다.

ENG) b) $h(A) = [4, 2]$, $h(B) = [0, 2]$, $h(C) = [1, 2]$, $h(D) = [2, 1]$

ENG)

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

JK 신경망의 층 (k) 출력은 $\sigma(\tilde{A} H^{(0)})$ 부터 $\sigma(\tilde{A} H^{(k-1)})$ 까지의 원소별 최댓값으로 계산된다고 가정한다.

- Jumping Knowledge 신경망이 깊은 GCN 의 과평활 문제를 어떻게 완화하는지 설명하시오.
- $\tilde{A}$ 와 $H^{(1)}$ 을 계산하시오.
- 층 $k = 2$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 0 \\ 0 \\ -2 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 에서는 노드가 이웃 평균을 거듭 내서 표현들이 다 비슷해져요. 이것이 과평활이에요.
 - JKNet 은 모든 층의 표현을 들고 있다가 끝에서 최댓값, 이어 붙이기, LSTM 으로 합쳐요.
 - 알맞은 깊이는 노드마다 달라요. 그래서 노드마다 자기에게 맞는 깊이를 고를 수 있어요.

ENG) Degrees: $d(A)=3, d(B)=3, d(C)=2, d(D)=2, d(E)=2$

KOR) 차수를 센다. $d(A)=3, d(B)=3, d(C)=2, d(D)=2, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.5, 0.5, 0, 0, 0]; [0.5, 0, 0.5, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.5, 0.5, 0, 0, 0]; [0.5, 0, 0.5, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.5, 0.5, 0, 0, 0]; [0.5, 0, 0.5, 0, 0]$

ENG) First build $H1 = \text{ReLU}(\tilde{A}H0)$

KOR) 먼저 $H1$ 을 만든다. $H1 = \text{ReLU}(\tilde{A}H0)$

ENG) A: neighbours B, D, E. $(0 + -2 + 3)/3 = 0.3333$

KOR) A: 이웃 B, D, E. $(0 + -2 + 3)/3 = 0.3333$

ENG) B: neighbours A, C, D. $(2 + 0 + -2)/3 = 0$

KOR) B: 이웃 A, C, D. $(2 + 0 + -2)/3 = 0$

ENG) C: neighbours B, E. $(0 + 3)/2 = 1.5$

KOR) C: 이웃 B, E. $(0 + 3)/2 = 1.5$

ENG) D: neighbours A, B. $(2 + 0)/2 = 1$

KOR) D: 이웃 A, B. $(2 + 0)/2 = 1$

ENG) E: neighbours A, C. $(2 + 0)/2 = 1$

KOR) E: 이웃 A, C. $(2 + 0)/2 = 1$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) $H1 = [0.3333, 0, 1.5, 1, 1]$

ENG) For $k = 2$ there are two terms. First: $\text{ReLU}(\tilde{A}H0) = [0.3333, 0, 1.5, 1, 1]$

KOR) $k=2$ 이므로 조각이 두 개다. 첫째 $\text{ReLU}(\tilde{A}H0) = [0.3333, 0, 1.5, 1, 1]$

ENG) Second: $\text{ReLU}(\tilde{A}H1) = [0.6667, 0.9444, 0.5, 0.1667, 0.9167]$

KOR) 둘째 $\text{ReLU}(\tilde{A}H1) = [0.6667, 0.9444, 0.5, 0.1667, 0.9167]$

ENG) The max is taken element-wise, not by choosing one whole vector.

KOR) max 는 가로줄마다 큰 쪽을 고른다. 벡터 하나를 고르는 것이 아니다.

ENG) c) $H2 = [0.6667, 0.9444, 1.5, 1, 1]$

ENG)

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.5$, $\beta = 0.5$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCNII 모형의 층 (k) 출력은 $H^{(k)} = \sigma[((1-\beta)I_n) \cdot ((1-\alpha)\tilde{A} H^{(k-1)} + \alpha H^{(0)})]$ 로 계산된다고 가정한다. I_n 은 단위행렬이며 α 와 β 는 문제에 주어진 값을 사용한다.

- GCNII 가 GCN 의 깊이 한계를 극복하기 위해 도입한 두 가지 주요 구성 요소를 쓰시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 4 \\ -2 \\ 3 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 두 장치는 초기 잔차 연결과 항등 사상이에요.
 - 초기 잔차 연결은 층마다 입력 $H^{(0)}$ 의 일부를 알파만큼 다시 더해요.
 - 항등 사상은 가중치 행렬을 단위행렬에 가깝게 뒹요. 그래서 깊어져도 신호가 안 망가져요.

ENG) Degrees: $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=3$, $d(E)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=3$, $d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0, 0.5, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0, 0.5, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0, 0.5, 0.5, 0]$

ENG) Start with the inner bracket. $\tilde{H}_0 = [3.5, 1.3333, 1.5, 2, 0.5]$

KOR) 안쪽 괄호부터 푼다. $\tilde{H}_0 = [3.5, 1.3333, 1.5, 2, 0.5]$

ENG) Multiply by $(1-\alpha) = 0.5$: $[1.75, 0.6667, 0.75, 1, 0.25]$

KOR) $(1-\alpha) = 0.5$ 를 곱한다. $[1.75, 0.6667, 0.75, 1, 0.25]$

ENG) Add $\alpha \tilde{H}_0 = 0.5 \times [3, 4, -2, 3, -1] = [1.5, 2, -1, 1.5, -0.5]$

KOR) $\alpha \tilde{H}_0 = 0.5 \times [3, 4, -2, 3, -1] = [1.5, 2, -1, 1.5, -0.5]$ 를 더한다.

ENG) Inner bracket = $[3.25, 2.6667, -0.25, 2.5, -0.25]$

KOR) 안쪽 괄호 = $[3.25, 2.6667, -0.25, 2.5, -0.25]$

ENG) Multiply by the outer $(1-\beta) = 0.5$. Dropping this factor scales the answer.

KOR) 바깥의 $(1-\beta) = 0.5$ 를 곱한다. 이 계수를 빼뜨리면 답이 배로 커진다.

ENG) ReLU clips the negatives: $-0.125 \rightarrow 0, -0.125 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.125 \rightarrow 0, -0.125 \rightarrow 0$

ENG) c) $H_1 = [1.625, 1.3333, 0, 1.25, 0]$

ENG)

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

잔차 연결을 갖는 DeepGCN 의 층 (k) 출력은 $H^{(k)} = \sigma(\tilde{A} H^{(k-1)}) + H^{(k-1)}$ 로 계산된다고 가정한다. 덧셈은 ReLU 적용 이후에 이루어진다.

- 잔차 연결이 GCN 을 더 깊게 쌓을 수 있게 하는 이유를 설명하시오.
- 층 $k = 1$ 과 $k = 2$ 에서의 출력 표현을 계산하시오.
- 잔차 연결이 없는 보통의 GCN 출력과 $H^{(1)}$ 을 비교하시오.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ 0 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 은 기울기 소실과 과평활을 겪어요. 그래서 층을 더하면 오히려 나빠져요.
 - 잔차 연결은 입력을 그대로 앞으로 보내요. 그래서 층은 바뀌는 양만 배우면 돼요.
 - 바꿀 게 없으면 층은 0 을 내보내면 돼요. 기울기도 그 지름길로 되돌아 흘러요.

ENG) Degrees: $d(A)=2, d(B)=3, d(C)=2, d(D)=3$

KOR) 차수를 센다. $d(A)=2, d(B)=3, d(C)=2, d(D)=3$

ENG) Divide each row by its degree. $\text{Atilde} = [0, 0.5, 0, 0.5]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.5, 0, 0.5]; [0.3333, 0, 0.3333, 0.3333]; [0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) $\tilde{H}_0 = [-2, 0.6667, -2, 0.6667]$

ENG) Apply ReLU first: $[0, 0.6667, 0, 0.6667]$

KOR) ReLU 를 먼저 건다. $[0, 0.6667, 0, 0.6667]$

ENG) Then add H_0 . The addition sits outside the ReLU, so negatives can survive.

KOR) 그 다음에 H_0 을 더한다. 더하기는 ReLU 바깥이라 음수가 살아남을 수 있다.

ENG) For $k = 2$, feed H_1 back in and add H_1 , not H_0 .

KOR) $k=2$ 에서는 H_1 을 다시 넣고, 더하는 것도 H_0 이 아니라 H_1 이다.

ENG) $\tilde{H}_1 = [-1.3333, 0.8889, -1.3333, 0.8889]$, after ReLU $[0, 0.8889, 0, 0.8889]$

KOR) $\tilde{H}_1 = [-1.3333, 0.8889, -1.3333, 0.8889]$, ReLU 뒤 $[0, 0.8889, 0, 0.8889]$

ENG) b) $H_1 = [4, -1.3333, 0, -1.3333]$

ENG) $H_2 = [4, -0.4444, 0, -0.4444]$

ENG) c) Plain GCN without residual: $H_1 = [0, 0.6667, 0, 0.6667]$ (the node's own value and negatives disappear)

KOR) c) 잔차 없는 GCN 의 $H_1 = [0, 0.6667, 0, 0.6667]$ (잔차가 없으면 자기 값과 음수가 사라져요)

5회차

ENG)

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 노드 1과 5에는 이름표가 주어져 있으며($P(Y_1)=1, P(Y_5)=0$), 나머지 노드는 이름표가 없고 0.5로 초기화한다.

관계형 분류기는 이름표가 없는 각 노드를 이웃 노드들의 부류 확률에 대한 가중 평균으로 갱신한다. 분모는 해당 노드에 연결된 간선 가중치의 합이다.

- 관계형 분류가 전제하는 핵심 가정과 그 한계 하나를 서술하시오.
- 1차 반복 후 이름표가 없는 각 노드의 $P(Y_v)$ 를 계산하시오.
- 2차 반복 후의 $P(Y_v)$ 를 계산하시오. 단, 직전 반복의 값만 사용한다.

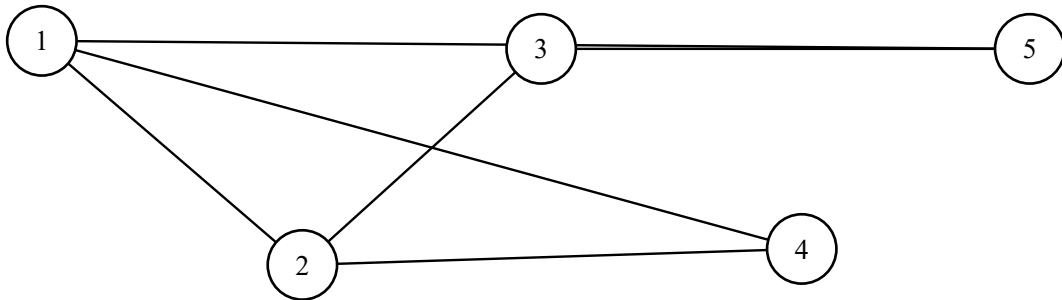


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 관계형 분류는 동질성을 가정해요. 이어진 노드끼리는 같은 이름표를 가지는 경향이 있다는 뜻이에요.
 - 이름표 없는 노드를 이웃들의 확률의 가중 평균으로 거듭 바뀌요.
 - 한계는 그래프 구조만 쓰고 노드의 특징은 무시한다는 것이에요.

ENG) Write the initial values: node 1 is 1, node 5 is 0, the rest start at 0.5.

KOR) 처음 값을 적는다. 이름표가 붙은 1번은 1, 5번은 0, 나머지는 0.5로 시작한다.

ENG) First iteration. Only the unlabelled nodes are replaced by the mean of their neighbours.

KOR) 1차 반복. 이름표가 없는 노드만 이웃의 평균으로 바꾼다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.5 + 0.5)/3 = 0.6667$

ENG) Node 3: neighbours 2, 5. $(0.5 + 0)/2 = 0.25$

KOR) 3번: 이웃 2, 5. $(0.5 + 0)/2 = 0.25$

ENG) Node 4: neighbours 1, 2. $(1 + 0.5)/2 = 0.75$

KOR) 4번: 이웃 1, 2. $(1 + 0.5)/2 = 0.75$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) b) $P(Y1) = 1$, $P(Y2) = 0.6667$, $P(Y3) = 0.25$, $P(Y4) = 0.75$, $P(Y5) = 0$

ENG) Second iteration. Use only the values from the first iteration, not the ones just computed.

KOR) 2차 반복. 방금 구한 값이 아니라 1차 반복의 값만 쓴다.

ENG) Node 1 is labelled, so it never changes. $P = 1$

KOR) 1번: 이름표가 붙어 있어 바뀌지 않는다. $P = 1$

ENG) Node 2: neighbours 1, 3, 4. $(1 + 0.25 + 0.75)/3 = 0.6667$

KOR) 2번: 이웃 1, 3, 4. $(1 + 0.25 + 0.75)/3 = 0.6667$

ENG) Node 3: neighbours 2, 5. $(0.6667 + 0)/2 = 0.3333$

KOR) 3번: 이웃 2, 5. $(0.6667 + 0)/2 = 0.3333$

ENG) Node 4: neighbours 1, 2. $(1 + 0.6667)/2 = 0.8333$

KOR) 4번: 이웃 1, 2. $(1 + 0.6667)/2 = 0.8333$

ENG) Node 5 is labelled, so it never changes. $P = 0$

KOR) 5번: 이름표가 붙어 있어 바뀌지 않는다. $P = 0$

ENG) c) $P(Y1) = 1$, $P(Y2) = 0.6667$, $P(Y3) = 0.3333$, $P(Y4) = 0.8333$, $P(Y5) = 0$

ENG)

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{B, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

KOR)

인접행렬 A 와 노드 특징 벡터 $h^{(0)}$ 가 주어진 무방향 그래프 G 를 고려한다.

FastGCN 은 GCN 한 층의 전체 이웃 합을 층 단위 몬테카를로 추정으로 대체한다. 각 층에서 t_l 개의 노드를 중요도 분포 $q(u) \propto \|A(:, u)\|^2$ 로부터 한 번 추출하며, 평균은 이웃 수가 아니라 추출 개수 t_l 로 나눈다.

- 층 단위 추출이 GraphSAGE 의 노드 단위 추출과 어떻게 다른지 서술하시오.
- 다섯 노드에 대한 중요도 분포 $q(u)$ 를 계산하시오.
- 주어진 추출 집합($t_l = 2$)에 대하여 $h^{(1)}(A)$ 의 추정값을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ 4 \\ 4 \\ -1 \\ -2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 뽑기는 노드마다 제 이웃을 뽑아서, 층이 깊어질수록 비용이 지수로 커져요.
 - FastGCN 은 층마다 정해진 개수를 한 번만 뽑고, 그 층의 모든 노드가 같이 써요.
 - 그래서 비용이 지수가 아니라 층 수에 비례해서만 늘어요.

ENG) Degrees: $d(A)=2, d(B)=1, d(C)=3, d(D)=3, d(E)=3$

KOR) 차수를 센다. $d(A)=2, d(B)=1, d(C)=3, d(D)=3, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.3333, 0.3333, 0.3333, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.3333, 0.3333, 0.3333, 0]$ (가로줄 합이 모두 1)

ENG) $q(u)$ uses the squared norm of column u of $\tilde{A}$, not of a row.

KOR) $q(u)$ 는 $\tilde{A}$ 의 **세로줄** 제곱합이다. 가로줄이 아니다.

ENG) Column A: squared sum = 0.2222

KOR) A 칸의 제곱합 = 0.2222

ENG) Column B: squared sum = 0.1111

KOR) B 칸의 제곱합 = 0.1111

ENG) Column C: squared sum = 0.4722

KOR) C 칸의 제곱합 = 0.4722

ENG) Column D: squared sum = 0.4722

KOR) D 칸의 제곱합 = 0.4722

ENG) Column E: squared sum = 1.2222

KOR) E 칸의 제곱합 = 1.2222

ENG) Divide by the total 2.5 to turn them into probabilities.

KOR) 전체 합 2.5 로 나눠 확률로 만든다.

ENG) $b) q(A) = 0.0889, q(B) = 0.0444, q(C) = 0.1889, q(D) = 0.1889, q(E) = 0.4889$

ENG) For (c) look only at the sampled nodes: read $\tilde{A}(A, u)$ for each.

KOR) c) 는 뽑힌 노드만 본다. A 가로줄에서 그 칸의 $\tilde{A}$ 값을 읽는다.

ENG) $\tilde{A}(A, B) h(B) = 0 \times 4 = 0$

KOR) $\tilde{A}(A, B) \times h(B) = 0 \times 4 = 0$

ENG) $\tilde{A}(A, E) h(E) = 0 \times -2 = 0$

KOR) $\tilde{A}(A, E) \times h(E) = 0 \times -2 = 0$

ENG) Add them and divide by the number sampled (2), not by the degree: $(0 + 0)/2 = 0$

KOR) 더한 뒤 이웃 수가 아니라 **뽑은 개수 2** 로 나눈다. $(0 + 0)/2 = 0$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $h_1(A) = 0$

ENG)

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = E$.
- Calculate $E_{p(u_j | E)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(E)$.

KOR)

문제 11 과 동일한 그래프와 특징 벡터를 고려한다.

ASGCN 은 GCN 한 층을 확률 분포에 대한 기댓값으로 다시 쓴다. 여기서 $N(v_i) = \sum_j \tilde{A}_{ij}$ 는 정규화 계수이고, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$ 는 부모 노드 v_i 가 주어졌을 때 노드 u_j 를 추출할 확률이다.

- ASGCN 의 제안 분포가 FastGCN 과 달리 무엇에 의존하는지 서술하시오.
- 주어진 부모 노드에 대한 $N(v_i)$ 와 $p(u_j | v_i)$ 를 계산하시오.
- 해당 기댓값과 그 결과로 얻어지는 $h^{(1)}$ 을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 4 \\ 4 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - FastGCN 의 뽑기 확률은 미리 정해져 있고 부모 노드와 상관이 없어요.
 - ASGCN 은 뽑기 확률을 윗층 노드에 맞춰 정해요.
 - 그래서 뽑은 노드가 거의 다 쓸모 있고, 추정값의 흔들림이 줄어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=4$, $d(E)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=4$, $d(E)=2$

ENG) Divide each row by its degree. Atilde = $[0, 0, 0, 0.5, 0.5]$; $[0, 0, 0.5, 0.5, 0]$; $[0, 0.5, 0, 0.5, 0]$; $[0.25, 0.25, 0.25, 0, 0.25]$; $[0.5, 0, 0, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5, 0]; [0, 0.5, 0, 0.5, 0]; [0.25, 0.25, 0.25, 0, 0.25]; [0.5, 0, 0, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) The parent node is E, so only its row of $\tilde{A}$ matters.

KOR) 부모 노드는 E 다. $\tilde{A}$ 의 그 줄만 본다.

ENG) $N(E) = \text{sum of that row} = 1$. With row normalisation it is 1.

KOR) $N(E) = \text{그 가로줄의 합} = 1$. 행 정규화라 1 이다.

ENG) $p(A|E) = \tilde{A}(E,A) / N = 0.5$

KOR) $p(A|E) = \tilde{A}(E,A) / N = 0.5 / 1 = 0.5$

ENG) $p(D|E) = \tilde{A}(E,D) / N = 0.5$

KOR) $p(D|E) = \tilde{A}(E,D) / N = 0.5 / 1 = 0.5$

ENG) b) $N(E) = 1$, $p(A|E) = 0.5$, $p(D|E) = 0.5$

ENG) The expectation is the sum of probability times value: $0.5 \times 4 + 0.5 \times 2 = 3$

KOR) 기댓값은 확률 곱하기 값의 합이다. $0.5 \times 4 + 0.5 \times 2 = 3$

ENG) $N = 1$ and $W = 1$, so the value is unchanged. It equals the plain neighbour mean.

KOR) $N = 1$, $W = 1$ 이라 곱해도 그대로다. 결국 이웃 평균과 같은 값이다.

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) c) $E = 3$, $h1(E) = 3$

ENG)

4. (APPP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

APPP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

KOR)

인접행렬 A 와 다층 퍼셉트론이 산출한 예측 행렬 H 가 주어졌다.

APPP 는 순간이동 확률 α 를 사용하여 예측을 전파한다. 전파 단계 사이에는 비선형 함수를 적용하지 않는다.

- 개인화 PageRank 의 순간이동 기법을 설명하고, 그것이 과평활을 막는 이유를 서술하시오.
- $\tilde{A}$ 와 $Z^{(1)}$ 을 계산하시오.
- $Z^{(2)}$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 퍼뜨리는 걸음마다 확률 알파로 출발 노드로 되돌아가요. 이것이 순간이동 장치예요.
 - 매 걸음 일정 비율이 되돌아오니까 분포가 출발점 근처에 머물러요.
 - 그래서 많이 퍼뜨려도 과평활이 안 생기고, 퍼뜨리는 데에 추가 파라미터도 없어요.

ENG) Degrees: $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

KOR) 차수를 센다. $d(A)=2, d(B)=2, d(C)=2, d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$

ENG) Start from $Z_0 = H = [0, 1, 0, 0]$

KOR) $Z_0 = H$ 로 시작한다. $[0, 1, 0, 0]$

ENG) Propagate one step: $\tilde{Z}_0 = [0.5, 0, 0, 0.5]$

KOR) 한 걸음 퍼뜨린다. $\tilde{Z}_0 = [0.5, 0, 0, 0.5]$

ENG) Multiply by $(1-\alpha) = 0.8$: $[0.4, 0, 0, 0.4]$

KOR) $(1-\alpha) = 0.8$ 를 곱한다. $[0.4, 0, 0, 0.4]$

ENG) Add $\alpha H = [0, 0.2, 0, 0]$. This is the teleport term.

KOR) $\alpha H = 0.2 \times [0, 1, 0, 0] = [0, 0.2, 0, 0]$ 를 더한다. 이것이 순간이동이다.

ENG) $Z_1 = [0.4, 0.2, 0, 0.4]$

ENG) One more step: $\tilde{Z}_1 = [0.1, 0.4, 0.4, 0.1]$

KOR) 한 걸음 더. $\tilde{Z}_1 = [0.1, 0.4, 0.4, 0.1]$

ENG) Multiply by $(1-\alpha)$ and add αH **again**. It is added at every step.

KOR) $(1-\alpha)$ 를 곱하고 **αH 를 또** 더한다. 매 걸음 더하는 것이 핵심이다.

ENG) c) $Z_2 = [0.08, 0.52, 0.32, 0.08]$

ENG)

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.2$, $\gamma_2 = -0.3$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

KOR)

인접행렬 A 와 신경망이 산출한 은닉 특징 $H^{(0)}$ 이 주어졌다.

GPR-GNN 은 특징 추출과 위상 전파를 분리한다. $\tilde{A}$ 를 반복 적용하여 $H^{(k)}$ 를 얻고, 학습된 가중치 γ_k 로 가중합한다. 본 문항에서는 $K = 2$ 이며 $\gamma_0, \gamma_1, \gamma_2$ 가 주어진다.

- GPR-GNN 이 APPNP 와 달리 할 수 있는 것은 무엇이며, γ_k 가 음수가 될 수 있다는 점이 이질적 그래프에서 왜 중요한지 서술하시오.
- $H^{(1)}$ 과 $H^{(2)}$ 를 계산하시오.
- Z 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 1 \\ -2 \\ 2 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - APPNP 는 계수를 알파 곱하기 (1-알파)의 k제곱으로 못 박아요. 늘 양수이고 줄어들어요.
 - GPR-GNN 은 그 무게를 학습하고, 음수도 될 수 있어요.
 - 음수 무게는 그 걸음을 빼요. 그래서 이질 그래프에서 고주파 필터처럼 동작할 수 있어요.

ENG) Degrees: d(A)=2, d(B)=2, d(C)=3, d(D)=1

KOR) 차수를 센다. d(A)=2, d(B)=2, d(C)=3, d(D)=1

ENG) Divide each row by its degree. Atilde = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0] (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. Atilde = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0] (가로줄 합이 모두 1)

ENG) H_0 stays as it is: $[-1, 1, -2, 2]$

KOR) H_0 은 그대로 둔다. $[-1, 1, -2, 2]$

ENG) $H_1 = \text{Atilde } H_0 = [-0.5, -1.5, 0.6667, -2]$

ENG) $H_2 = \text{Atilde } H_1 = [-0.4167, 0.0833, -1.3333, 0.6667]$

ENG) b) $H_1 = [-0.5, -1.5, 0.6667, -2]$, $H_2 = [-0.4167, 0.0833, -1.3333, 0.6667]$

ENG) Now take the weighted sum with gamma. There is no ReLU during propagation.

KOR) 이제 gamma 로 가중합한다. 전파 중에 ReLU 는 없다.

ENG) $\text{gamma}_0 H_0 = 0.7 \times [-1, 1, -2, 2] = [-0.7, 0.7, -1.4, 1.4]$

KOR) $\text{gamma}_0 \times H_0 = 0.7 \times [-1, 1, -2, 2] = [-0.7, 0.7, -1.4, 1.4]$

ENG) $\text{gamma}_1 H_1 = 0.2 \times [-0.5, -1.5, 0.6667, -2] = [-0.1, -0.3, 0.1333, -0.4]$

KOR) $\text{gamma}_1 \times H_1 = 0.2 \times [-0.5, -1.5, 0.6667, -2] = [-0.1, -0.3, 0.1333, -0.4]$

ENG) $\text{gamma}_2 H_2 = -0.3 \times [-0.4167, 0.0833, -1.3333, 0.6667] = [0.125, -0.025, 0.4, -0.2]$

KOR) $\text{gamma}_2 \times H_2 = -0.3 \times [-0.4167, 0.0833, -1.3333, 0.6667] = [0.125, -0.025, 0.4, -0.2]$

ENG) gamma_2 is negative, so that hop is subtracted. Dropping the sign gives a wrong answer.

KOR) gamma_2 가 음수라 그 걸음은 빼는 효과다. 부호를 빼뜨리면 틀린다.

ENG) Add the three vectors element-wise.

KOR) 세 벡터를 자리별로 더한다.

ENG) c) $Z = [-0.675, 0.375, -0.8667, 0.8]$

ENG)

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

MixHop 한 층은 인접행렬의 여러 거듭제곱 $\hat{A}^j$ ($j \in P = \{0, 1, 2\}$)를 각각 계산한 뒤 열 방향으로 이어 붙인다. 본 문항에서 가중치 행렬은 모두 1로 둔다.

- 두 걸음 델타 연산자에 관한 MixHop의 정리 1을 서술하고, 보통의 GCN이 층을 무한히 쌓아도 이를 표현할 수 없는 이유를 설명하시오.
- $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$, $\hat{A}^2 H^{(0)}$ 을 계산하시오.
- $H^{(1)}$ 과 델타 연산자 $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ 를 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -3 \\ 1 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 정리 1은 보통 GCN이 두 걸음 델타 연산자를 표현할 수 없다고 말해요. 층을 무한히 쌓아도요.
 - GCN 층을 쌓으면 A 에 1제곱 곱하기 W 꼴로 뭉쳐져서, 중간 걸음들이 따로 남지 않아요.
 - MixHop은 거듭제곱마다 따로 구해서 나란히 붙여요. 그래서 뒤 층이 하나에서 다른 하나를 뺄 수 있어요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=2$, $d(D)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
(each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$ (가로 줄 합이 모두 1)

ENG) A^0 is the identity, so it is just $H_0: [3, -3, 1, 0]$

KOR) A_0 은 단위행렬이라 H_0 그대로다. $[3, -3, 1, 0]$

ENG) $A^1 H_0 = [-1, 1.5, 1.5, -1]$ (one hop, the neighbour mean)

KOR) $A_1 H_0 = [-1, 1.5, 1.5, -1]$ (한 걸음, 이웃 평균)

ENG) $A^2 H_0 = \tilde{A}(\tilde{A} H_0) = [1.5, -1, -1, 1.5]$ (two hops)

KOR) $A_2 H_0 = \tilde{A}(\tilde{A} H_0) = [1.5, -1, -1, 1.5]$ (두 걸음)

ENG) ReLU is applied to each block separately, not after concatenation.

KOR) 조각마다 따로 ReLU 를 건다. 이어 붙인 뒤가 아니다.

ENG) After ReLU: $[3, 0, 1, 0] \parallel [0, 1.5, 1.5, 0] \parallel [1.5, 0, 0, 1.5]$

KOR) ReLU 뒤: $[3, 0, 1, 0] \parallel [0, 1.5, 1.5, 0] \parallel [1.5, 0, 0, 1.5]$

ENG) The delta operator subtracts the two-hop block from the one-hop block.

KOR) 델타 연산자는 한 걸음에서 두 걸음을 뺀 것이다.

ENG) b) $A_0 H = [3, -3, 1, 0]$, $A_1 H = [-1, 1.5, 1.5, -1]$, $A_2 H = [1.5, -1, -1, 1.5]$

ENG) c) $H_1 =$ concatenation of the three blocks: $[3, 0, 1, 0] \parallel [0, 1.5, 1.5, 0] \parallel [1.5, 0, 0, 1.5]$

KOR) c) $H_1 =$ 세 조각을 이어 붙여요: $[3, 0, 1, 0] \parallel [0, 1.5, 1.5, 0] \parallel [1.5, 0, 0, 1.5]$

ENG) $\Delta = [-1.5, 1.5, 1.5, -1.5]$

ENG)

7. (PairNorm) After one GCN layer the node features of a four-node graph are PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

KOR)

GCN 한 층을 통과한 뒤의 노드 특징 행렬 X 가 주어졌다. 행은 노드, 열은 특징 차원이다.

PairNorm 은 중심화 단계와 크기 조정 단계로 정규화한다. 중심화는 노드들에 대한 평균을 빼는 것이고, 크기 조정은 중심화된 벡터들의 노름 제곱 평균의 제곱근으로 나누는 것이다. $s = 1, n = 4$ 로 한다.

- 노드 단위 과평활과 특징 단위 과평활을 각각 설명하고, PairNorm 이 층에 걸쳐 일정하게 유지하는 양이 무엇인지 서술하시오.
- 중심화된 특징을 계산하시오.
- 크기 조정의 분모와 최종 x_i 를 계산하시오.

$$X = \begin{bmatrix} -2 & 4 \\ 1 & -1 \\ -1 & 1 \\ -1 & 3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 노드 단위 과평활은 이어진 노드들의 특징이 너무 비슷해지는 것이에요.
 - 특징 단위 과평활은 그래프 전체에서 모든 노드 특징이 구별이 안 되게 되는 것이에요.
 - PairNorm 은 층을 지나도 노드 사이 거리 제곱의 합을 일정하게 지켜요.

ENG) First take the mean over the nodes, separately for each feature column.

KOR) 먼저 노드들에 대한 평균을 구한다. 특징 차원마다 따로 구한다.

ENG) Column 1: $(-2 + 1 + -1 + -1)/4 = -0.75$

KOR) 첫 칸: $(-2 + 1 + -1 + -1)/4 = -0.75$

ENG) Column 2: $(4 + -1 + 1 + 3)/4 = 1.75$

KOR) 둘째 칸: $(4 + -1 + 1 + 3)/4 = 1.75$

ENG) Subtract the mean from every row. Check: each column now sums to zero.

KOR) 가로줄마다 평균을 뺀다. 검산: 각 칸의 합이 0 이 된다.

ENG) Node 1: squared norm = 6.625

KOR) 노드 1 의 길이 제곱 = 6.625

ENG) Node 2: squared norm = 10.625

KOR) 노드 2 의 길이 제곱 = 10.625

ENG) Node 3: squared norm = 0.625

KOR) 노드 3 의 길이 제곱 = 0.625

ENG) Node 4: squared norm = 1.625

KOR) 노드 4 의 길이 제곱 = 1.625

ENG) Mean = $(6.625 + 10.625 + 0.625 + 1.625)/4 = 4.875$

KOR) 평균 = $(6.625 + 10.625 + 0.625 + 1.625)/4 = 4.875$

ENG) Take the square root: $\text{sqrt}(4.875) = 2.2079$. Divide every row by this single number.

KOR) 제곱근을 씌운다. $\text{sqrt}(4.875) = 2.2079$. 이 **하나의 수** 로 모든 줄을 나눈다.

ENG) b) Mean = $[-0.75, 1.75]$, centered = $[-1.25, 2.25]$; $[1.75, -2.75]$; $[-0.25, -0.75]$; $[-0.25, 1.25]$

KOR) b) 평균 = $[-0.75, 1.75]$, 중심을 뺀 것 = $[-1.25, 2.25]$; $[1.75, -2.75]$; $[-0.25, -0.75]$; $[-0.25, 1.25]$

ENG) c) Scaling denominator = 2.2079, $x = [-0.5661, 1.019]$; $[0.7926, -1.2455]$; $[-0.1132, -0.3397]$; $[-0.1132, 0.5661]$

KOR) c) 분모 = 2.2079, $x = [-0.5661, 1.019]$; $[0.7926, -1.2455]$; $[-0.1132, -0.3397]$; $[-0.1132, 0.5661]$

ENG)

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (C, D) and (A, D) are dropped. Recalculate $H^{(1)}$.

KOR)

노드 A, B, C, D 로 이루어진 완전그래프 K_4 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

DropEdge 는 매 에폭마다 각 무방향 간선을 확률 $1-p$ 로 독립적으로 유지한다. 이때 $A^{(t)}$ 와 $D^{(t)}$ 는 살아남은 간선만으로 다시 구성한다. $p = 1/3$ 으로 한다.

- DropEdge 가 목표로 하는 두 가지 문제를 쓰고, Dropout 과 어떻게 다른지 서술하시오.
- 살아남은 간선 수의 기댓값과, 간선을 제거하지 않은 원래 그래프에서의 $H^{(1)}$ 을 계산하시오.
- 주어진 두 간선이 제거되었을 때의 $H^{(1)}$ 을 다시 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 2 \\ 1 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - DropEdge 는 과적합과 과평활을 노려요.
 - 판마다 다른 부분 그래프를 써요. 그래프에 자료를 늘리는 효과가 나요.
 - Dropout 은 특징 값을 0 으로 꺼요. DropEdge 는 인접행렬의 선을 지워요.

ENG) K_4 has 6 edges. Each survives with probability $1 - 1/3 = 2/3$, so the expectation is 4.

KOR) K_4 는 간선이 6 개다. 남을 확률이 $1 - 1/3 = 2/3$ 이므로 기댓값은 $6 \times 2/3 = 4$ 개다.

ENG) First the full graph: every node has degree 3.

KOR) 먼저 원래 그래프. 모든 노드의 차수가 3 다.

ENG) A: neighbours B, C, D. $(2 + 1 + 0)/3 = 1$

KOR) A: 이웃 B, C, D. $(2 + 1 + 0)/3 = 1$

ENG) B: neighbours A, C, D. $(-2 + 1 + 0)/3 = -0.3333$

KOR) B: 이웃 A, C, D. $(-2 + 1 + 0)/3 = -0.3333$

ENG) C: neighbours A, B, D. $(-2 + 2 + 0)/3 = 0$

KOR) C: 이웃 A, B, D. $(-2 + 2 + 0)/3 = 0$

ENG) D: neighbours A, B, C. $(-2 + 2 + 1)/3 = 0.3333$

KOR) D: 이웃 A, B, C. $(-2 + 2 + 1)/3 = 0.3333$

ENG) Now drop the two given edges and **recount the degrees**. This is where most mistakes happen.

KOR) 이제 주어진 두 간선을 지운다. **차수를 다시 센다**. 여기서 거의 다 틀린다.

ENG) Degrees after dropping: $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=1$

KOR) 지운 뒤 차수: $d(A)=2$, $d(B)=3$, $d(C)=2$, $d(D)=1$

ENG) A: neighbours B, C. $(2 + 1)/2 = 1.5$

KOR) A: 이웃 B, C. $(2 + 1)/2 = 1.5$

ENG) B: neighbours A, C, D. $(-2 + 1 + 0)/3 = -0.3333$

KOR) B: 이웃 A, C, D. $(-2 + 1 + 0)/3 = -0.3333$

ENG) C: neighbours A, B. $(-2 + 2)/2 = 0$

KOR) C: 이웃 A, B. $(-2 + 2)/2 = 0$

ENG) D: neighbours B. $(2)/1 = 2$

KOR) D: 이웃 B. $(2)/1 = 2$

ENG) b) Expected number of surviving edges = 4, $H1 = [1, 0, 0, 0.3333]$

KOR) b) 남은 선 수의 기댓값 = 4, $H1 = [1, 0, 0, 0.3333]$

ENG) c) $H1 = [1.5, 0, 0, 2]$

ENG)

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

KOR)

다음 그림의 무방향 그래프 G 를 고려한다. 임의보행이 노드 A 에서 노드 B 로 이동하여 현재 B 에 있으며, $p = 2, q = 0.5$ 이다.

B 에서 다음 노드 x 로의 비정규화 전이확률은 $\pi_{Bx} = \alpha_{pq}(A, x)$ 이며, α 는 직전 노드 A 로부터의 최단경로 거리에 따라 결정된다.

- BFS 와 DFS 를 설명하고, q 가 작을 때 보행이 둘 중 어느 쪽처럼 동작하는지 서술하시오.
- 노드 B 에서 이동 가능한 모든 노드를 열거하고 비정규화 전이확률 π_{Bx} 를 계산하시오.
- 정규화된 전이확률을 계산하시오.

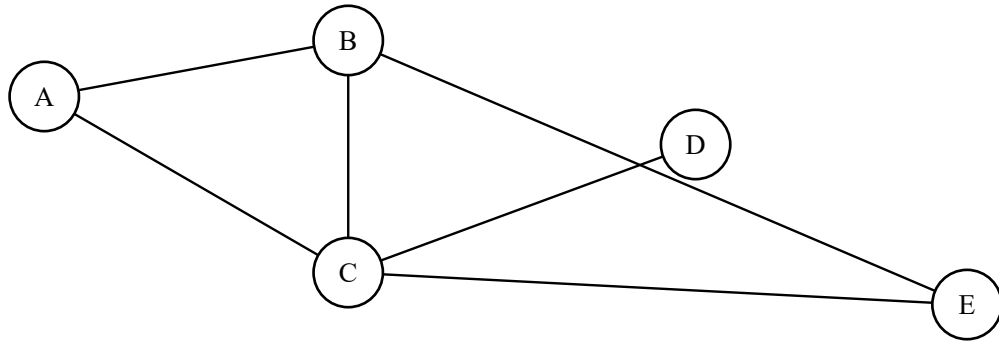


Figure 1. Graph G

풀이

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - BFS 는 바로 옆 이웃부터 훑어요. 그래서 임베딩이 노드의 구조적 역할을 담아요.
 - DFS 는 출발점에서 멀리 가요. 그래서 임베딩이 어느 동네 소속인지를 담아요.
 - q 가 작으면 $1/q$ 가 커져서 걸음이 바깥으로 나가요. DFS 처럼 움직여요.

ENG) The previous node is A and the walk is now at B. All distances are measured from A.

KOR) 직전 노드는 A, 지금 노드는 B 다. 거리는 모두 A 로부터 잴다.

ENG) A: $x = A$ is the previous node, $d = 0$, $\alpha = 1/p = 0.5$

KOR) A: $x = A$ 는 직전 노드라 $d = 0$. $\alpha = 1/p = 0.5$

ENG) C: x is also a neighbour of A, $d = 1$, $\alpha = 1 = 1$

KOR) C: A 와도 이웃이라 $d = 1$. $\alpha = 1 = 1$

ENG) E: x is two hops from A, $d = 2$, $\alpha = 1/q = 2$

KOR) E: A 에서 두 걸음이라 $d = 2$. $\alpha = 1/q = 2$

ENG) Sum them: $0.5 + 1 + 2 = 3.5$

KOR) 세 값을 더한다. $0.5 + 1 + 2 = 3.5$

ENG) b) $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 1$, $\pi(B \rightarrow E) = 2$

ENG) c) $P(A) = 0.1429$, $P(C) = 0.2857$, $P(E) = 0.5714$

ENG)

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = 0.5, u'_B = 1, u'_C = -1, u'_D = -0.5.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- What is second-order proximity, and how does it differ from first-order proximity?
- Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

KOR)

노드 임베딩 u 와 맥락 임베딩 u' 가 주어진 무방향 그래프 G 를 고려한다.

2차 근접도는 $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$ 로 정의한다. 지수 함수 값은 문제에 주어진 표를 사용한다.

- 2차 근접도란 무엇이며 1차 근접도와 어떻게 다른지 서술하시오.
- 주어진 기준 노드에 대하여 네 노드의 p_2 를 계산하시오.

풀이

ENG)

- Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 1차 근접도는 선으로 직접 이어진 두 노드 사이의 비슷함이에요.
 - 2차 근접도는 이웃을 많이 공유하는 두 노드의 비슷함이에요. 선이 없어도 돼요.
 - 그래서 LINE 은 노드마다 벡터를 두 개 뒹요. 자기 벡터와 맥락 벡터예요.

ENG) The base node is B with $u(B) = 0.5$. The other side uses the context vector u' .

KOR) 기준 노드는 B 이고 $u(B) = 0.5$ 다. 상대는 맥락 벡터 u' 를 쓴다.

ENG) A: $u'(A) u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

KOR) A: $u'(A) \times u(B) = 0.5 \times 0.5 = 0.25$, $\exp = 1.28$

ENG) B: $u'(B) u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

KOR) B: $u'(B) \times u(B) = 1 \times 0.5 = 0.5$, $\exp = 1.65$

ENG) C: $u'(C) u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

KOR) C: $u'(C) \times u(B) = -1 \times 0.5 = -0.5$, $\exp = 0.61$

ENG) D: $u'(D) u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

KOR) D: $u'(D) \times u(B) = -0.5 \times 0.5 = -0.25$, $\exp = 0.78$

ENG) The denominator is the sum of all four: $1.28 + 1.65 + 0.61 + 0.78 = 4.32$

KOR) 분모는 넷을 모두 더한 값이다. $1.28 + 1.65 + 0.61 + 0.78 = 4.32$

ENG) b) $p_2(A|B) = 0.2963$, $p_2(B|B) = 0.3819$, $p_2(C|B) = 0.1412$, $p_2(D|B) = 0.1806$

ENG)

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCN 모형의 층 (k) 에서의 은닉층은 $H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)})$ 로 계산된다고 가정한다. 여기서 $\tilde{A} = D^{-1}A$ 는 정규화된 인접행렬이고 σ 는 ReLU 이다.

- GCN 한 층에서 정보가 어떻게 전파되는지 설명하시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서 GCN 모형의 출력을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ 4 \\ 0 \\ -1 \\ -3 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 한 층에서 노드는 바로 옆 이웃의 특징을 모아 평균을 내요.
 - 행 정규화 행렬 $\tilde{A}$ 데는 D 의 역행렬 곱하기 A 이고, 그것이 평균을 내 줘요.
 - k 개 층을 지나면 노드는 k 걸음 안의 모든 노드 정보를 받아요.

ENG) Degrees: $d(A)=2, d(B)=3, d(C)=3, d(D)=2, d(E)=2$

KOR) 차수를 센다. $d(A)=2, d(B)=3, d(C)=3, d(D)=2, d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\text{Atilde} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$

ENG) Multiply Atilde by H_0 . Each entry is the mean of that node's neighbours.

KOR) Atilde 를 H_0 에 곱한다. 노드마다 이웃의 평균이다.

ENG) A: neighbours B, C. $(4 + 0)/2 = 2$

KOR) A: 이웃 B, C. $(4 + 0)/2 = 2$

ENG) B: neighbours A, D, E. $(0 + -1 + -3)/3 = -1.3333$

KOR) B: 이웃 A, D, E. $(0 + -1 + -3)/3 = -1.3333$

ENG) C: neighbours A, D, E. $(0 + -1 + -3)/3 = -1.3333$

KOR) C: 이웃 A, D, E. $(0 + -1 + -3)/3 = -1.3333$

ENG) D: neighbours B, C. $(4 + 0)/2 = 2$

KOR) D: 이웃 B, C. $(4 + 0)/2 = 2$

ENG) E: neighbours B, C. $(4 + 0)/2 = 2$

KOR) E: 이웃 B, C. $(4 + 0)/2 = 2$

ENG) ReLU clips the negatives: $-1.3333 \rightarrow 0, -1.3333 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1.3333 \rightarrow 0, -1.3333 \rightarrow 0$

ENG) c) $H_1 = [2, 0, 0, 2, 2]$

ENG)

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 5개)를 고려한다. 각 노드 옆의 수가 초기 특징이다.

집계 함수가 평균인 GraphSAGE 모형에서, 노드 i 의 층 k 특징은 이웃의 평균을 자신의 특징에 이어 붙인 뒤 ReLU를 적용하여 갱신된다.

- GraphSAGE의 핵심 착상은 무엇이며 기존 GCN과 어떻게 다른지 서술하시오.
- $k = 1$ 에서 각 노드의 특징을 계산하시오.
- $k = 1$ 일 때 평균 전역 풀링으로 그래프 수준 임베딩 h_G 를 계산하시오.

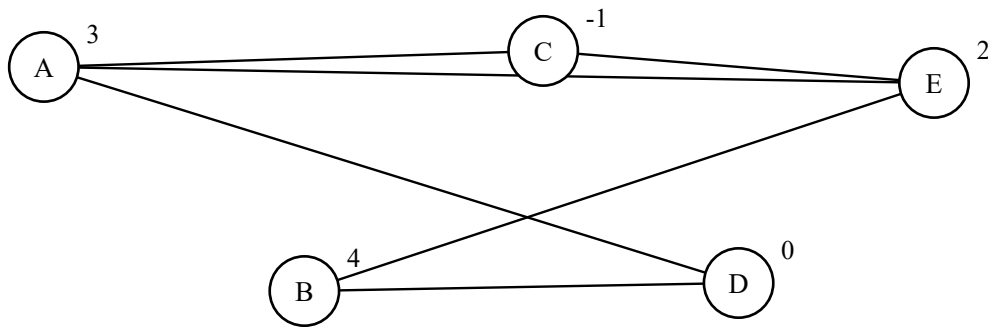


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE는 노드마다 고정된 벡터를 배우는 대신 모으는 함수를 배워요.
 - 이웃을 정해진 개수만 뽑아요. 그래서 아주 큰 그래프에서도 미니배치로 학습할 수 있어요.
 - GCN은 전이적이고 GraphSAGE는 귀납적이에요. 그래서 학습 때 못 본 노드도 벡터를 만들 수 있어요.

ENG) First compute the neighbour mean. The node itself is not included.

KOR) 먼저 이웃 평균을 구한다. 자기 자신은 넣지 않는다.

ENG) A: neighbours C, D, E. $(-1 + 0 + 2)/3 = 0.3333$

KOR) A: 이웃 C, D, E. $(-1 + 0 + 2)/3 = 0.3333$

ENG) B: neighbours D, E. $(0 + 2)/2 = 1$

KOR) B: 이웃 D, E. $(0 + 2)/2 = 1$

ENG) C: neighbours A, E. $(3 + 2)/2 = 2.5$

KOR) C: 이웃 A, E. $(3 + 2)/2 = 2.5$

ENG) D: neighbours A, B. $(3 + 4)/2 = 3.5$

KOR) D: 이웃 A, B. $(3 + 4)/2 = 3.5$

ENG) E: neighbours A, B, C. $(3 + 4 + -1)/3 = 2$

KOR) E: 이웃 A, B, C. $(3 + 4 + -1)/3 = 2$

ENG) Concatenate the node's own value with the neighbour mean. This is not an addition.

KOR) 자기 값을 앞에, 이웃 평균을 뒤에 이어 붙인다. 더하는 것이 아니다.

ENG) ReLU clips the negatives: $-1 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-1 \rightarrow 0$

ENG) The graph embedding is the mean per position: first $8 / 5$, second $9.3333 / 5$

KOR) 그래프 임베딩은 자리별 평균이다. 첫 자리 합 $8 / 5$, 둘째 자리 합 $9.3333 / 5$

ENG) b) $h(A) = [3, 0.3333]$, $h(B) = [4, 1]$, $h(C) = [0, 2.5]$, $h(D) = [0, 3.5]$, $h(E) = [2, 2]$

ENG) c) $hG = [1.8, 1.8667]$

ENG)

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 는 두 개의 군집 G_1 과 G_2 를 포함하며, 각 노드 옆의 수가 초기 특징이다.

ClusterGCN 모형에서는 이웃 중 자신과 같은 군집에 속한 노드만 평균에 포함한다.

- 미니배치 학습이란 무엇이며 ClusterGCN이 이를 필요로 하는 이유를 서술하시오.
- 층 $k = 1$ 에서 모든 노드의 출력 표현을 계산하시오.

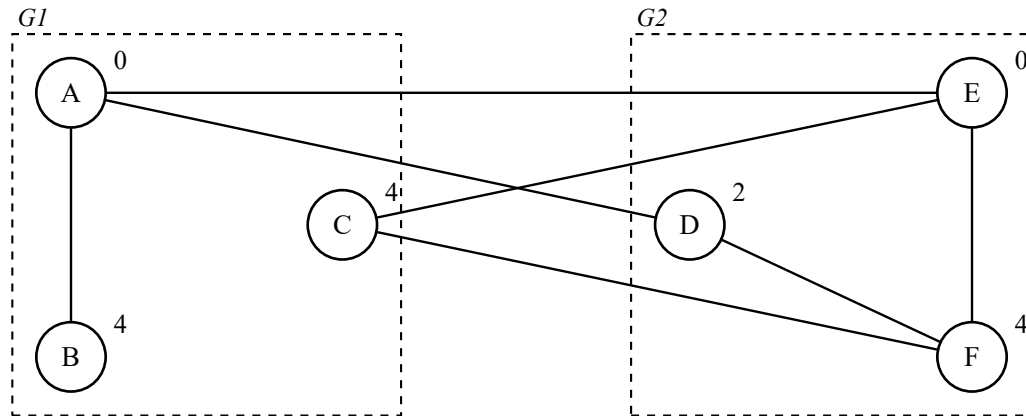


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 미니배치 학습은 한 번에 자료의 일부만 써서 파라미터를 고쳐요.
 - 큰 그래프는 메모리에 안 들어가요. 그래서 ClusterGCN은 안쪽이 촘촘한 덩어리로 나뉘요.
 - 잘 나누면 선 대부분이 배치 안에 남아서 경계에서 잃는 정보가 적어요.

ENG) Edges that cross a group are treated as absent. Cross them out first.

KOR) 묶음을 넘는 간선은 없는 것으로 본다. 먼저 그 간선에 사선을 그어 지운다.

ENG) A: neighbours B. $(4)/1 = 4$

KOR) A: 이웃 B. $(4)/1 = 4$

ENG) B: neighbours A. $(0)/1 = 0$

KOR) B: 이웃 A. $(0)/1 = 0$

ENG) C: no neighbour inside the group, so the mean is 0.

KOR) C: 같은 묶음 안에 이웃이 없다. 평균 0.

ENG) D: neighbours F. $(4)/1 = 4$

KOR) D: 이웃 F. $(4)/1 = 4$

ENG) E: neighbours F. $(4)/1 = 4$

KOR) E: 이웃 F. $(4)/1 = 4$

ENG) F: neighbours D, E. $(2 + 0)/2 = 1$

KOR) F: 이웃 D, E. $(2 + 0)/2 = 1$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU to both entries.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다. ReLU 는 두 자리 모두에 걸린다.

ENG) b) $h(A) = [0, 4]$, $h(B) = [4, 0]$, $h(C) = [4, 0]$, $h(D) = [2, 4]$, $h(E) = [0, 4]$, $h(F) = [4, 1]$

ENG)

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

KOR)

다음 그림의 무방향 그래프 G (노드 6개)를 고려한다. 그래프 G 에는 추출된 부분그래프 G_1 이 주어져 있으며, 각 노드 옆의 수가 초기 특징이다.

GraphSAINT 모형에서는 이웃 중 같은 부분그래프에 속한 노드만 평균에 포함한다.

- GraphSAINT의 추출 전략이 노드 단위 또는 층 단위 추출 방식과 어떻게 다른지 서술하시오.
- 층 $k = 1$ 에서 노드 A, B, C, D의 출력 표현을 계산하시오.

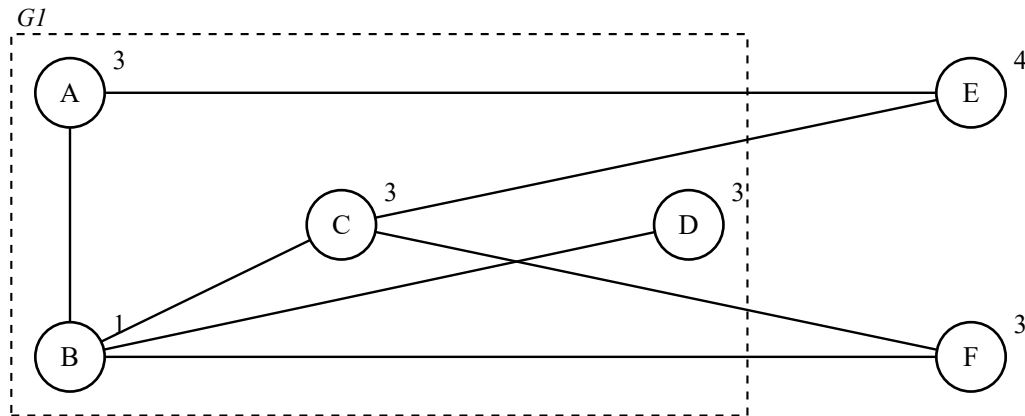


Figure. Graph G

풀이

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - GraphSAGE와 FastGCN은 층을 쌓으면서 뽑아요. 그래서 층마다 뽑기를 되풀이해요.
 - GraphSAINT는 부분 그래프를 먼저 뽑고 그 위에 GNN을 통째로 올려요.
 - 그래서 깊어져도 비용이 안 늘고, 정규화 계수로 뽑기의 치우침을 바로잡아요.

ENG) Edges that leave the sampled subgraph are discarded.

KOR) 뽑힌 부분그래프 밖으로 나가는 간선은 버린다.

ENG) A: neighbours B. $(1)/1 = 1$

KOR) A: 이웃 B. $(1)/1 = 1$

ENG) B: neighbours A, C, D. $(3 + 3 + 3)/3 = 3$

KOR) B: 이웃 A, C, D. $(3 + 3 + 3)/3 = 3$

ENG) C: neighbours B. $(1)/1 = 1$

KOR) C: 이웃 B. $(1)/1 = 1$

ENG) D: neighbours B. $(1)/1 = 1$

KOR) D: 이웃 B. $(1)/1 = 1$

ENG) Concatenate the neighbour mean after the node's own value, then apply ReLU.

KOR) 자기 값 뒤에 이웃 평균을 이어 붙이고 ReLU 를 건다.

ENG) b) $h(A) = [3, 1]$, $h(B) = [1, 3]$, $h(C) = [3, 1]$, $h(D) = [3, 1]$

ENG)

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

JK 신경망의 층 (k) 출력은 $\sigma(\tilde{A} H^{(0)})$ 부터 $\sigma(\tilde{A} H^{(k-1)})$ 까지의 원소별 최댓값으로 계산된다고 가정한다.

- Jumping Knowledge 신경망이 깊은 GCN 의 과평활 문제를 어떻게 완화하는지 설명하시오.
- $\tilde{A}$ 와 $H^{(1)}$ 을 계산하시오.
- 층 $k = 2$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -1 \\ 4 \\ 2 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 에서는 노드가 이웃 평균을 거듭 내서 표현들이 다 비슷해져요. 이것이 과평활이에요.
 - JKNet 은 모든 층의 표현을 들고 있다가 끝에서 최댓값, 이어 붙이기, LSTM 으로 합쳐요.
 - 알맞은 깊이는 노드마다 달라요. 그래서 노드마다 자기에게 맞는 깊이를 고를 수 있어요.

ENG) Degrees: $d(A)=3, d(B)=2, d(C)=3, d(D)=1, d(E)=3$

KOR) 차수를 센다. $d(A)=3, d(B)=2, d(C)=3, d(D)=1, d(E)=3$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0.3333, 0.3333, 0, 0.3333]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 1, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0.3333, 0.3333, 0, 0.3333]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 1, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0.3333, 0.3333, 0, 0.3333]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 1, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$

ENG) First build $H1 = \text{ReLU}(\tilde{A}H0)$

KOR) 먼저 $H1$ 을 만든다. $H1 = \text{ReLU}(\tilde{A}H0)$

ENG) A: neighbours B, C, E. $(-1 + 4 + -1)/3 = 0.6667$

KOR) A: 이웃 B, C, E. $(-1 + 4 + -1)/3 = 0.6667$

ENG) B: neighbours A, E. $(1 + -1)/2 = 0$

KOR) B: 이웃 A, E. $(1 + -1)/2 = 0$

ENG) C: neighbours A, D, E. $(1 + 2 + -1)/3 = 0.6667$

KOR) C: 이웃 A, D, E. $(1 + 2 + -1)/3 = 0.6667$

ENG) D: neighbours C. $(4)/1 = 4$

KOR) D: 이웃 C. $(4)/1 = 4$

ENG) E: neighbours A, B, C. $(1 + -1 + 4)/3 = 1.3333$

KOR) E: 이웃 A, B, C. $(1 + -1 + 4)/3 = 1.3333$

ENG) No negative value, so ReLU changes nothing.

KOR) ReLU 를 걸어도 음수가 없어 그대로다.

ENG) $H1 = [0.6667, 0, 0.6667, 4, 1.3333]$

ENG) For $k = 2$ there are two terms. First: $\text{ReLU}(\tilde{A}H0) = [0.6667, 0, 0.6667, 4, 1.3333]$

KOR) $k=2$ 이므로 조각이 두 개다. 첫째 $\text{ReLU}(\tilde{A}H0) = [0.6667, 0, 0.6667, 4, 1.3333]$

ENG) Second: $\text{ReLU}(\tilde{A}H1) = [0.6667, 1, 2, 0.6667, 0.4444]$

KOR) 둘째 $\text{ReLU}(\tilde{A}H1) = [0.6667, 1, 2, 0.6667, 0.4444]$

ENG) The max is taken element-wise, not by choosing one whole vector.

KOR) max 는 가로줄마다 큰 쪽을 고른다. 벡터 하나를 고르는 것이 아니다.

ENG) c) $H2 = [0.6667, 1, 2, 4, 1.3333]$

ENG)

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.5$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

GCNII 모형의 층 (k) 출력은 $H^{(k)} = \sigma[((1-\beta)I_n) \cdot ((1-\alpha)\tilde{A} H^{(k-1)} + \alpha H^{(0)})]$ 로 계산된다고 가정한다. I_n 은 단위행렬이며 α 와 β 는 문제에 주어진 값을 사용한다.

- GCNII 가 GCN 의 깊이 한계를 극복하기 위해 도입한 두 가지 주요 구성 요소를 쓰시오.
- $\tilde{A}$ 를 계산하시오.
- 층 $k = 1$ 에서의 출력 표현을 계산하시오.

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -3 \\ 3 \\ 4 \\ -1 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 두 장치는 초기 잔차 연결과 항등 사상이에요.
 - 초기 잔차 연결은 층마다 입력 $H^{(0)}$ 의 일부를 알파만큼 다시 더해요.
 - 항등 사상은 가중치 행렬을 단위행렬에 가깝게 뒹요. 그래서 깊어져도 신호가 안 망가져요.

ENG) Degrees: $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$, $d(E)=2$

KOR) 차수를 센다. $d(A)=2$, $d(B)=2$, $d(C)=3$, $d(D)=3$, $d(E)=2$

ENG) Divide each row by its degree. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0, 0.3333, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0, 0.3333, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$ (가로줄 합이 모두 1)

ENG) b) $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0, 0.3333, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$

ENG) Start with the inner bracket. $H_0 = [3.5, 1, 0.6667, 1, 0.5]$

KOR) 안쪽 괄호부터 푼다. $H_0 = [3.5, 1, 0.6667, 1, 0.5]$

ENG) Multiply by $(1-\alpha) = 0.5$: $[1.75, 0.5, 0.3333, 0.5, 0.25]$

KOR) $(1-\alpha) = 0.5$ 를 곱한다. $[1.75, 0.5, 0.3333, 0.5, 0.25]$

ENG) Add $\alpha H_0 = 0.5 \times [1, -3, 3, 4, -1] = [0.5, -1.5, 1.5, 2, -0.5]$

KOR) $\alpha H_0 = 0.5 \times [1, -3, 3, 4, -1] = [0.5, -1.5, 1.5, 2, -0.5]$ 를 더한다.

ENG) Inner bracket = $[2.25, -1, 1.8333, 2.5, -0.25]$

KOR) 안쪽 괄호 = $[2.25, -1, 1.8333, 2.5, -0.25]$

ENG) Multiply by the outer $(1-\beta) = 0.6$. Dropping this factor scales the answer.

KOR) 바깥의 $(1-\beta) = 0.6$ 를 곱한다. 이 계수를 빼뜨리면 답이 배로 커진다.

ENG) ReLU clips the negatives: $-0.6 \rightarrow 0, -0.15 \rightarrow 0$

KOR) ReLU 로 음수를 0 으로 누른다. $-0.6 \rightarrow 0, -0.15 \rightarrow 0$

ENG) c) $H_1 = [1.35, 0, 1.1, 1.5, 0]$

ENG)

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

KOR)

인접행렬 A 와 초기 노드 특징 행렬 $H^{(0)}$ 이 주어졌다.

잔차 연결을 갖는 DeepGCN 의 층 (k) 출력은 $H^{(k)} = \sigma(\tilde{A} H^{(k-1)}) + H^{(k-1)}$ 로 계산된다고 가정한다. 덧셈은 ReLU 적용 이후에 이루어진다.

- 잔차 연결이 GCN 을 더 깊게 쌓을 수 있게 하는 이유를 설명하시오.
- 층 $k = 1$ 과 $k = 2$ 에서의 출력 표현을 계산하시오.
- 잔차 연결이 없는 보통의 GCN 출력과 $H^{(1)}$ 을 비교하시오.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

풀이

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.

KOR)

- 는 영어로 쓴다. 위 세 문장을 그대로 적으면 된다.
 - 깊은 GCN 은 기울기 소실과 과평활을 겪어요. 그래서 층을 더하면 오히려 나빠져요.
 - 잔차 연결은 입력을 그대로 앞으로 보내요. 그래서 층은 바뀌는 양만 배우면 돼요.
 - 바꿀 게 없으면 층은 0 을 내보내면 돼요. 기울기도 그 지름길로 되돌아 흘러요.

ENG) Degrees: $d(A)=3, d(B)=3, d(C)=2, d(D)=2$

KOR) 차수를 센다. $d(A)=3, d(B)=3, d(C)=2, d(D)=2$

ENG) Divide each row by its degree. $\text{Atilde} = [0, 0.3333, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$ (each row sums to 1)

KOR) 가로줄마다 차수로 나눈다. $\text{Atilde} = [0, 0.3333, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$ (가로줄 합이 모두 1)

ENG) $\tilde{H}_0 = [1, -0.3333, -1, -1]$

ENG) Apply ReLU first: $[1, 0, 0, 0]$

KOR) ReLU 를 **먼저** 건다. $[1, 0, 0, 0]$

ENG) Then add $\tilde{H}_0$. The addition sits outside the ReLU, so negatives can survive.

KOR) 그 다음에 $\tilde{H}_0$ 을 더한다. 더하기는 ReLU 바깥이라 음수가 살아남을 수 있다.

ENG) For $k = 2$, feed H_1 back in and add H_1 , not $\tilde{H}_0$.

KOR) $k=2$ 에서는 H_1 을 다시 넣고, 더하는 것도 $\tilde{H}_0$ 이 아니라 H_1 이다.

ENG) $\tilde{H}_1 = [1, 0, -0.5, -0.5]$, after ReLU $[1, 0, 0, 0]$

KOR) $\tilde{H}_1 = [1, 0, -0.5, -0.5]$, ReLU 뒤 $[1, 0, 0, 0]$

ENG) b) $H_1 = [-2, 1, 2, 0]$

ENG) $H_2 = [-1, 1, 2, 0]$

ENG) c) Plain GCN without residual: $H_1 = [1, 0, 0, 0]$ (the node's own value and negatives disappear)

KOR) c) 잔차 없는 GCN 의 $H_1 = [1, 0, 0, 0]$ (잔차가 없으면 자기 값과 음수가 사라져요)